Agentic Workflows

Joel Stevick

Edited by Luke O'Malley

AiFirst Press

This page intentionally left blank

Copyright © 2026 Joel Stevick

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Published by AiFirst Press

First Edition, 2026

Joel's Dedication



My beloved children

Contents

- Introduction
- 1. Talking to a Model
- 2. Giving the Model Hands
- 3. The Loop
- 4. What It Knows and Forgets
- 5. Why It Lies to You
- 6. Writing It Down First
- 7. A Second Pair of Eyes
- 8. Running the Thing
- 9. Somebody Has to Decide
- 10. Tests That Can Fail
- 11. Rules a Prompt Can't Hold
- 12. From Yours to Shared
- 13. Inside a Plugin
- Conclusion
- Appendix A: Glossary
- Appendix B: The Finished Workflow

*An **agent** is a program in a loop with a language model. The program sends a request with a list of tools the model may ask for, each described in plain words: what it does, what arguments it takes, what it returns. The model replies by naming one and the arguments to give it; it cannot run anything itself. The program runs what the model named, puts both the call and its result into the next request, and sends it again, until the model asks for nothing more or a limit the program sets ends it.*

*An **agentic workflow** is a fixed chain of jobs that a piece of work passes through from start to finish, where each job is either an agent or ordinary code. The jobs run serially, and where several run in parallel, the next one waits for all of them to finish.*

The tools are where anything actually changes: a file written, a row updated, a service called. The model's last reply is not the work. The work is already done by then.

Introduction

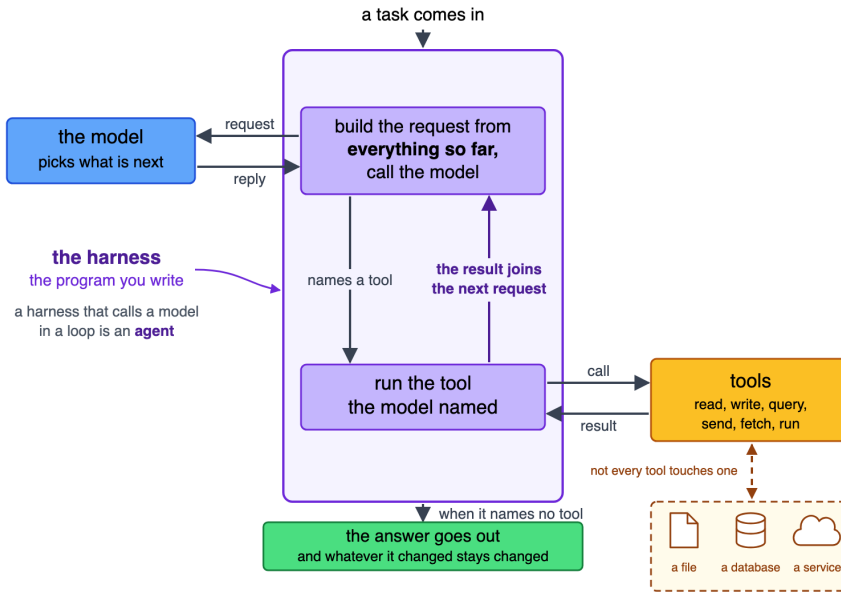


Figure 1: A task comes in at the top. Your program builds a request and calls the model, which picks what happens next but cannot run anything itself — it can only name a tool and wait. Your program is what runs it, and the result goes into the next request, so each pass through the loop carries everything the earlier passes produced. The tools are where things actually change: a file, a row in a database, a call to a service somewhere else. When the model names no tool the loop ends, and the answer that leaves at the bottom is a report on work that has already happened.

The picture above is an **agent**: a program that calls a model in a loop, runs whatever tool the model asks for, and stops when the model stops asking. It is what you are going to build.

A **workflow** is a fixed sequence of jobs that a piece of work passes through, where each job has one responsibility and a defined output.

An **agentic workflow** is one in which some of those jobs are carried out by a language model rather than by a person or by code you wrote in advance.

What travels along those arrows is one JSON object — which model to ask, the standing instruction, the tools it may call, and every instruction and reply so far, because the model keeps nothing between calls. Chapter 1 builds the first few of those fields and Chapter 2 the rest, and Chapter 2 shows the whole object once you have written enough of it to read it.

The parts you are going to build

Agentic workflows do not all need the same parts. A job that classifies text or drafts a summary satisfies the definition above and never calls a tool, never loops, and never touches anything outside itself. A pipeline whose output nobody relies on needs no checking either.

The one built here assumes the harder case: the model acts on something real, and being confidently wrong is expensive. That case takes two things. A model has to be able to act, and whatever establishes that the work is right has to be something other than the run that produced it.

This book divides acting and checking into six parts, and each names the chapter that builds it.

The first four cover acting. Together they are the agent.

A model call. One HTTP request carrying your instructions and ev-

erything sent and received so far, and one response. The model retains nothing between calls: everything it knows on call n is what you put in call n, which makes carrying state forward your program's job rather than the model's. (Chapter 1)

Tools. Descriptions of capabilities — read a file, run a query against a database, call a service somewhere else — listed in the request's `tools` field. The model cannot execute any of them itself; it can only reply that it wants to, naming a tool and its arguments, and your code decides whether to run it and sends the result back. Every constraint on what an agent can do lives on your side of that line. (Chapter 2)

A loop. Call the model, execute whatever tool it asked for, append the result, call again, until it stops asking. The loop needs a way to stop, because a model that keeps requesting tools will keep being handed them. (Chapter 3)

Context management. The model sees only what the request contains, and a request cannot grow past a fixed size — past it the API returns an error rather than quietly dropping the oldest part. What to include, what to summarise, and what to drop becomes a design problem as soon as a run makes more than a handful of calls. (Chapter 4)

Those four produce something that works. They produce nothing that establishes whether the work was done *correctly*, because every part listed so far reports on itself. That is the second requirement, and it takes two more parts.

Recorded intent. A statement of what the result must do, written before the run starts, in terms specific enough to check without having done the work. (Chapter 6)

Independent checks. Jobs that establish whether the output is correct without consulting the model that produced it. A reviewer reads the change and is never shown the argument for why it is good. A test runs

the finished program and compares what it actually did against what it was supposed to do. A separate program refuses the work outright when a rule is broken — when a test was deleted, say, to make the rest of them pass. (Chapters 7 through 11)

Why the last two exist

Three properties of language models are why recorded intent and independent checks are required:

- **Fluency is independent of correctness.** A wrong answer is written with the same vocabulary, structure and confidence as a right one, so you cannot tell them apart by reading.
- **Self-assessment is unreliable.** A model asked whether its own work is correct will usually say yes, and will produce plausible reasoning for that answer if you ask why.
- **Output varies between runs.** The same input can produce different results, so a run that came out right once does not tell you the next one will.

Recorded intent and independent checks answer these directly. Intent is recorded before the run, because a criterion written afterwards can be shaped to fit whatever was produced. The checks are carried out without asking the model that did the work, because a model asked whether its own work is correct says yes either way, and an answer that never varies tells you nothing.

What the six parts are good for

None of the six parts is specific to software. All six apply wherever a model does work that something downstream depends on: extraction

pipelines whose output is validated against a schema before it is stored, research summaries where each claim carries a source a later job verifies, classification passes audited against a labelled sample, drafts screened by a separate job against the rules they must not break.

What differs between those is only how expensive it is to find out you were wrong. Software delivery is the worked example here because checking is cheapest there: a test passes or it does not, a program runs or it crashes, a change either contains a line or it does not.

What you are going to build

You are going to write, line by line, a small delivery pipeline in TypeScript. By Chapter 12 it takes a plain-English request, writes down what it intends to do in checkable terms, does the work, has that work examined by independent readers that never saw the reasoning behind it, has a separate judge decide which of their objections are real, and refuses to declare success while a test is failing. Every part of it is code you typed.

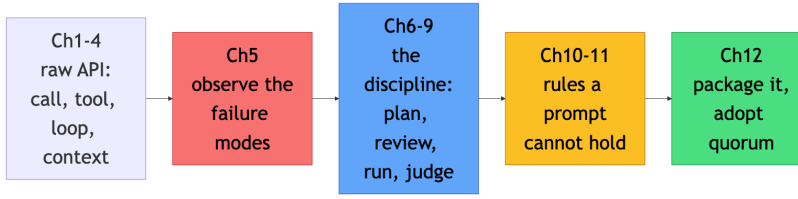


Figure 2: The order of construction: the four parts that make an agent, the failure modes that motivate the rest, then the parts that make its output checkable.

Chapters 1 through 4 build parts 1 to 4 — a working agent. Chapter 5 has you run it against a task where it produces confident, wrong output. Chapters 6 through 11 build parts 5 and 6, each one answering a specific failure you saw in Chapter 5.

Chapter 12 is where you stop typing and start delegating, and Chapter 13 opens the package you delegated to and reads it against what you wrote.

If you would rather read than type, the finished version already exists. Everything these chapters build by hand is published at `https://github.com/AiFirstDevelopment/quorum`, installable into a repository in one file, and Chapter 12 is about adopting it.

Who this is for

You are a working programmer who can read and write code, has shipped something, and is comfortable in a terminal. You have used an AI assistant in your editor, or you have not; either is fine.

What the chapters ahead assume you have *not* done is written code that calls a model directly: not through a wrapper, a framework or a chat window, but a program of yours that makes an HTTP request to Anthropic and does something with the answer.

Typing it yourself

Chapters 1 through 11 have you write every line. Chapter 12 hands the finished pipeline over to a packaged version, and Chapter 13 takes the package apart.

What you need before Chapter 1

Four things, none of which takes long.

1. Node.js 20 or newer, and npm. Check the version you have with `node --version`; if it prints something starting with `v20.` or higher, you have what the examples need. The examples use TypeScript throughout and are run with `tsx`, which compiles and runs a `.ts` file in one step with no build directory to manage.

2. An Anthropic account and an API key. An **API key** is a secret string that identifies your account to Anthropic's servers. Create one at `console.anthropic.com` under **API Keys**. It begins with `sk-ant-` and it is shown to you exactly once, so copy it somewhere safe immediately.

Put it in your shell environment rather than in a file you might commit:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

While you are in the console, set a spending limit under **Billing**. Working through every chapter and re-running the examples a few times each costs a few dollars rather than a few hundred, so five dollars is comfortably more than enough.

> **Pro Tip:** Set the limit even if you are not worried about the total. Chapter 3 has you write a loop that calls the model repeatedly until it decides it is finished, and one of the exercises in that chapter is deliberately making that loop misbehave. A ceiling converts an expensive mistake into an error to read.

> **Warning:** A key committed to a public repository is found and used by automated scanners within minutes, and you are billed for what they do with it. Never write the key into a source file, not even temporarily, not even in a file you intend to delete before committing. Every example ahead reads the key from the environment for that reason.

3. A project directory. Everything you build lives in one folder:

```
mkdir agentic-workflow && cd agentic-workflow
npm init -y
npm install @anthropic-ai/sdk
npm install --save-dev typescript tsx @types/node
```

Anthropic publishes an **SDK** — software development kit — as `@anthropic-ai/sdk`: a library wrapping its HTTP interface in ordinary TypeScript functions. Chapter 1 walks through each field of the first call one at a time.

Then replace the generated `package.json` with this one.

Listing: `package.json`

```
{
  "name": "agentic-workflow",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "test": "tsx demo/src/slugify.test.ts",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.124.0"
  },
  "devDependencies": {
    "@types/node": "^20.14.0",
    "tsx": "^4.16.0",
    "typescript": "^5.5.0"
  }
}
```

Then run `npm install` once more. The versions in that file are not the ones the earlier commands installed, and `package-lock.json` records what was installed; Chapter 11's build job installs from the lock file and refuses one that disagrees with `package.json`.

And add a TypeScript configuration beside it.

Listing: `tsconfig.json`

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "types": ["node"],
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "noEmit": true
  },
  "include": ["agentic/**/*.ts", "demo/**/*.ts"]
}
```

Check it with `npx tsc --noEmit`, which right now answers:

```
error TS18003: No inputs were found in config file 'tsconfig.json'.
```

That is correct: `agentic/` and `demo/` do not exist yet, so there is nothing to compile.

4. Somewhere to put it. Run `git init` in that directory now. Several later chapters compare one state of the code against another.

Add one more file before you commit anything. Chapter 10 has you run `git add -A`, and without this every change a reviewer reads from Chapter 7 onward arrives buried in `node_modules/`.

Listing: `.gitignore`

```
node_modules/
```

How to read the code

Code appears in the chapters in two forms. Short excerpts show one idea in isolation — a few lines lifted out of a file so that the thing being

discussed is the only thing on the page. Complete files are labelled with a line of exactly this shape, immediately above the code:

```
**Listing: `agentic/example.ts`**
```

Every complete file also appears, in its finished state, in Appendix B. If you get lost, or if you want to check a file you typed against the version the book ends with, that appendix is the reference. It is generated from the chapters themselves, so the two cannot disagree.

Type the code. Do not paste it.

When the model is replaced

Model names change and parameters are added. The specific version identifier used in the examples will eventually be superseded. When it is, the examples still work if you change one string in one place; Chapter 1 shows you where that string lives.

Newer models invent fewer facts and agree with wrong suggestions less often. Neither rate reaches zero, and the wrong answers that remain read exactly as well as the right ones.

Chapter 1 is about thirty lines of TypeScript, starting on the next page.

Chapter 1

Talking to a Model

This chapter builds one operation in TypeScript, field by field: you send some text to a server, and text comes back. An agentic workflow rests on that operation. Every diagram, review, judge and guard rule in the twelve chapters after this one is that single operation wrapped in structure.

What the thing on the other end actually is

A **large language model** — LLM for short — is a very large mathematical function that takes a sequence of text and produces a probability distribution over what text comes next. It was fitted to that job by being shown an enormous quantity of text and repeatedly adjusted until its guesses about what followed got good. The result is a fixed set of numbers, called weights, that do not change while you use them. Running those weights over an input to get an output is called **inference**, and it is the only thing that happens when you call the API: no training, no learning, no adjustment. Inference is a read of a frozen function.

Two consequences fall straight out of that description.

The first is that the model is *generating*, not *retrieving*. When it tells you that a TypeScript function is called `Array.prototype.findLast`, it is not looking that up anywhere. It is producing the text that its fitted function scores as likely to follow your question. Most of the time the likely continuation and the true one are the same, because it was fitted on text where they usually were. Sometimes they are not, and nothing in the scoring separates a likely continuation from a true one. Chapter 5 is about living with that.

The second is that the model has no memory. The weights do not change when it answers you. Whatever it appears to know about your last question, it knows because you sent your last question along with this one. We will prove that with a program later in this chapter.

The model is a pure function of its input: same weights, same input, same distribution over outputs. Memory, personality and continuity are supplied by the program around it, in the input.

Your first call

Create a directory called `agentic` inside your project. Everything you build in the chapters ahead lives there.

One file, about thirty lines, no framework of any kind. Type it.

Listing: `agentic/talk.ts`

```ts
import Anthropic from "@anthropic-ai/sdk";

// A missing credential should be one clear line, not a stack trace.
if (!process.env.ANTHROPIC_API_KEY) {
  console.error("ANTHROPIC_API_KEY is not set. Export it and run this again.");
  process.exit(1);
}
```

```
const client = new Anthropic();

async function main(): Promise<void> {
  const response = await client.messages.create({
    model: "claude-opus-5",
    max_tokens: 1024,
    messages: [
      {
        role: "user",
        content: "In two sentences: what is a slug in a URL, and why?",
      },
    ],
  });

  for (const block of response.content) {
    if (block.type === "text") {
      console.log(block.text);
    }
  }

  console.log(
    `\n[in: ${response.usage.input_tokens} out:
    ${response.usage.output_tokens}]`,
  );
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Run it:

```
npx tsx agentic/talk.ts
```

You should get two sentences about URL slugs and a line of counts. If you got a line about `ANTHROPIC_API_KEY`, export the key as described in the introduction and try again. If you got something else, skip ahead to *When it goes wrong* at the end of this chapter.

That is a working program that talks to a large language model. There is no framework underneath it and nothing hidden: `new Anthropic()`

builds an HTTP client, `messages.create` posts a request, and the reply comes back as data.

`new Anthropic()` takes no arguments because the client reads `ANTHROPIC_API_KEY` from the environment by itself, which is why no program here ever writes a key into a source file. You *can* pass one explicitly — `new Anthropic({ apiKey })` — and the only good reason to is when a single program has to use two different accounts.

The four lines above it print the name of the missing variable and exit non-zero, instead of failing later with an `AuthenticationError` stack trace. The **exit code** is the small integer a process hands back to whatever started it; zero conventionally means success and anything else means failure, which is how the shell, `npm`, and continuous integration systems decide whether your program worked. Chapter 10 onward depends on that.

Now go through the request slowly, because five of those lines are concepts every later chapter uses.

The request, field by field

`messages` and the shape of a conversation

The `messages` array is the entire input to the model, and it is a list of **message** objects. A message is one contribution to the exchange, and it has exactly two parts that matter here: a **role**, saying who is speaking, and content, saying what they said.

There are two roles you will write by hand:

- `user` — you, or your program acting on your behalf.
- `assistant` — the model.

The array must begin with a `user` message, and conventionally the two alternate. The complete array — every message in it, in order — is what we will call the **conversation**, and when we are talking about it as something to inspect rather than something to send, the **transcript**.

The array is *the input*: it is neither a reference to a session on Anthropic's servers nor a pointer to a conversation ID, but the literal text of everything that has been said so far, sent again, in full, on every single request.

`model`

The `model` field carries the model ID from `setup.ts`, and it has no default: every request must say which model it wants. Different models are different functions with different weights, and the same request to two of them can produce different answers.

`max_tokens`

`max_tokens` is a hard ceiling on how much the model is allowed to produce in this response. It is required, it is not a suggestion, and if the model reaches it, the response simply stops — mid-sentence if necessary.

Which requires knowing what a token is.

Tokens

A **token** is the unit of text the model actually operates on: a chunk of characters somewhere between a letter and a word. Common English words are usually one token each, while rarer words are split into several, and whitespace and punctuation usually ride along with a neighbouring word. As a working rule of thumb for English prose, one token is about

four characters, or about three quarters of a word; for code it is denser, because identifiers like `requireApiKey` split into pieces.

Tokens govern three separate things:

1. **You are billed per token**, at one rate for tokens going in and a higher rate for tokens coming out. That is what the `usage` line at the bottom of `talk.ts` is reporting.
2. **`max_tokens` is measured in output tokens.** Set it too low and answers get truncated; set it absurdly high and nothing bad happens directly, because you are billed for what is produced, not for the ceiling.
3. **There is a limit on the total**, and that limit is what constrains how you design.

The **context window** is the maximum number of tokens that can be in a single request and its response combined — everything you send plus everything that comes back. Exceed it and the API returns an error rather than silently dropping the oldest part. Current Anthropic models have very large context windows, large enough that none of these exercises will reach it by accident, but "very large" is not "unlimited," and Chapter 4 is entirely about what you do as you approach it.

> **Warning:** Cost is linear in the tokens you send, and you send the entire conversation every time — so a loop that runs twenty rounds over a growing transcript does not cost twenty times one request, it costs closer to two hundred, and Chapter 4 gives you the arithmetic for it.

The response

`response.content` is an array, not a string. The reason is that a single response can contain several different kinds of thing. Each element is a **content block**: a typed piece of the answer. The one you care about now has `type: "text"` and carries a `text` field. In Chapter 2 you will meet a second kind, and in Chapter 4 a third.

This is why `talk.ts` loops and checks `block.type` rather than reaching for `response.content[0].text`. TypeScript will in fact refuse to compile the second form, because `content` is a union of block types and only some of them have a `.text`. The day a response comes back with a different block first, code that assumed `[0]` would have read `undefined` and carried on.

`response.usage` reports what the request consumed: `input_tokens` is what you sent and `output_tokens` is what came back. Print them. In Chapter 4 you will start adding them up.

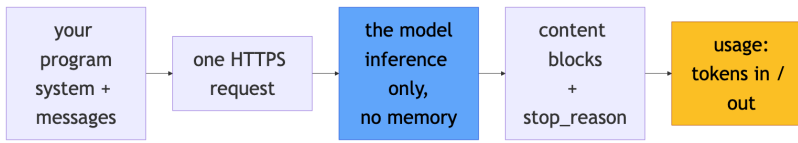


Figure 1.1: One request, one response. What you send is a system instruction and an array; what comes back is content blocks, a reason it stopped, and a count of the tokens used. Tools and the loop add structure around this exchange rather than replacing it.

One place for the things that change

You are about to write a second program, and it needs the same two things `talk.ts` needed: the model identifier and the key check. Before there are two copies of either, put them in a file of their own.

Listing: `agentic/setup.ts`

```ts
// The one place the model identifier lives. When Anthropic releases a
// newer model, this is the only line in the whole project that changes.
export const MODEL = "claude-opus-5";

// Every script from here on calls this before it does anything else.
export function requireApiKey(): void {
  if (!process.env.ANTHROPIC_API_KEY) {
    console.error(
      "ANTHROPIC_API_KEY is not set. Export it and run this again.",
    );
    process.exit(1);
  }
}
```

The string `claude-opus-5` is a **model ID**: the exact identifier of the model a request runs against. Anthropic serves several models at once — larger and slower, smaller and cheaper — and the model ID is how you choose between them.

In Chapter 7 you will run the same change past the same model five times over, and Chapter 8 adds a sixth; changing which model that is takes one edit rather than six. It is also the string most likely to be out of date by the time you read this.

`talk.ts` can stay exactly as it is — it is a one-off, and leaving it standalone means your first program depends on nothing but the SDK. Every file from here on imports `MODEL` and `requireApiKey` from `setup.ts`.

Proving it has no memory

The experiment below is short and costs about a tenth of a cent.

Listing: `agentic/chat.ts`

```ts
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

requireApiKey();

const client = new Anthropic();

const SYSTEM = "You are terse. Answer in one short sentence.";

async function ask(
  history: Anthropic.MessageParam[],
  question: string,
): Promise<string> {
  history.push({ role: "user", content: question });

  const response = await client.messages.create({
    model: MODEL,
    max_tokens: 1024,
    system: SYSTEM,
    messages: history,
  });

  // Push the model's own reply back into the history, or the next
  // request will look like we never heard it.
  history.push({ role: "assistant", content: response.content });

  return response.content
    .filter((b): b is Anthropic.TextBlock => b.type === "text")
    .map((b) => b.text)
    .join("");
}

async function main(): Promise<void> {
  const remembered: Anthropic.MessageParam[] = [];
  console.log("A:", await ask(remembered, "My favourite colour is orange."));
  console.log("B:", await ask(remembered, "What is my favourite colour?"));

  // Same question, same model, same everything - except we throw the
  // history away first.
```

```
  const forgotten: Anthropic.MessageParam[] = [];
  console.log("C:", await ask(forgotten, "What is my favourite colour?"));

  console.log(`\nhistory carried into B: ${remembered.length - 1} messages`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Run it. Line B knows your favourite colour, and line C — the identical question, to the identical model, seconds later — does not, and says so.

Nothing was forgotten between B and C, because nothing was ever remembered. The model is **stateless**: it retains nothing between requests, and every request is answered from scratch using only what that request contains. The illusion of memory in any chat product you have used is a program on the other side doing just what `ask` does: keeping an array and resending it.

> **Example:** Comment out the `history.push({ role: "assistant", ... })` line and run it again, and line B now fails too, for an exact reason: the model was asked "what is my favourite colour?" immediately after being told it, but the answer it gave in between was never put in the array, so from the model's point of view the exchange reads as two consecutive user statements with no reply.

If the model held state you could not inspect, you could not build one around it. Because it holds none, the entire input is a data structure in your program, and anything you can compute you can put in it.

The system prompt

`chat.ts` introduced a field `talk.ts` did not have.

A **prompt** is the text you give the model to act on. In casual use people mean the question they typed; in the API it is the whole input, meaning every message in the array plus the field we are about to discuss.

The **system prompt** is a separate top-level `system` field carrying instructions about how to behave, as opposed to what to answer. It is not a message and has no role; it sits outside the array. Two things make it different from putting the same words in the first user message:

- It comes first, always, no matter how long the conversation grows.
- It reads as operator instruction rather than as something a participant in the conversation said, which makes it meaningfully harder for later text to talk the model out of it.

In Chapter 7 you will build a reader whose job is to examine a change and report problems, and its system prompt will say so. If that instruction were the first user message, the material under review — which is text of unknown origin — would be sitting in the same channel as its own instructions.

Rule: Instructions to the model go in `system`. Material for the model to work on goes in `messages`. When you find yourself concatenating the two into one string, stop: you are removing the only structural distinction the API gives you between what to do and what to do it to.

Two more request fields

`temperature` controls how the next token is chosen from the distribution the model produced. At `0` the most likely candidate is taken every time. Higher values sample more adventurously, which is what people mean by "creative." It takes a value between 0 and 1 and defaults to 1.

`temperature: 0` does not make the model deterministic. It makes only the *selection rule* deterministic, and identical requests at temperature zero can still produce different text, for reasons that live in floating-point arithmetic and server-side batching and are entirely outside your control. Lower temperature narrows the spread of outcomes without collapsing it to one.

Some current models, including the one these examples use by default, do not accept `temperature` at all — they manage that themselves. If you send it and get a `400`, that is why; remove it.

`thinking` lets the model reason at length before answering, and **`effort`** sets how hard it works. On the model these examples use, thinking is on unless you switch it off, and that has two consequences you will see before you set either field: the `output_tokens` figure `talk.ts` prints includes reasoning you were never shown, and a response can carry a `thinking` block ahead of its text, which is one more reason `talk.ts` filters on `block.type`. Both fields are set on the request, and both cost output tokens:

```
const response = await client.messages.create({
  model: MODEL,
  max_tokens: 4096,
  thinking: { type: "adaptive" },
  output_config: { effort: "high" },
  messages,
});
```

Nothing in the chapters ahead sets either one. The defaults are what every example was run with, and the token counts you add up in Chapter 4 include the thinking.

When it goes wrong

Four failures account for nearly everything you will hit, and the SDK gives each one a class rather than making you match on message text.

```
try {
  const response = await client.messages.create({ /* ... */ });
} catch (error) {
  if (error instanceof Anthropic.AuthenticationError) {
    console.error("The API key was rejected. Check ANTHROPIC_API_KEY.");
  } else if (error instanceof Anthropic.RateLimitError) {
    console.error("Rate limited. Wait and retry.");
  } else if (error instanceof Anthropic.BadRequestError) {
    console.error(`The request was malformed: ${error.message}`);
  } else if (error instanceof Anthropic.APIError) {
    console.error(`API error ${error.status}: ${error.message}`);
  } else {
    throw error;
  }
}
```

- **`AuthenticationError`** (401): the key is wrong, revoked, or has a stray newline in it from however you copied it.
- **`RateLimitError`** (429): you have exceeded a **rate limit** — a cap on how many requests or tokens an account may use per minute, which exists so that one client cannot starve the rest. The SDK already retries these a couple of times with a delay before giving up, so seeing one means it kept happening.
- **`BadRequestError`** (400): the request itself is wrong. Missing `max_tokens`, an empty `messages` array, a field the chosen model does not accept. The message is usually specific enough to fix directly.
- **`APIError`**: everything else, including the 500s that are Anthropic's problem rather than yours.

Catch from most specific to least, and re-throw what you did not expect. A bare `catch` that logs and continues converts a broken request into a

silent empty response, and in Chapter 9 you will have a judge making decisions from responses.

> **Warning:** The one error you cannot catch is the response that arrives fine and is wrong. Nothing raises, `usage` looks normal, and no field in the reply distinguishes it from a correct answer. That failure has no exception class and never will, which is why Chapters 6 through 11 build checks that sit outside the conversation that produced the answer.

Where this leaves you

You have a program that sends text to a large language model and reads the response. The chapter covered what a token is, what the context window bounds, why the model is stateless, and where instructions belong. The continuity of a conversation is an array your code maintains.

What you have cannot act on anything. It can produce text about a file; it cannot read one. It can describe a command; it cannot run one.

The next chapter fixes exactly that, and it does so without letting the model run anything itself.

Chapter 2

Giving the Model Hands

This chapter builds a tool: a description of one of your own functions that the model can ask you to run. At the end of Chapter 1 you had a program that could talk about a file it could not read.

The mechanism in one sentence: **the model never runs anything; it asks you to.**

You describe some functions to it, and the model, mid-answer, stops and says "please call `read_file` with this argument." Your code calls your own function, with your own permissions, and sends the result back. The model continues with that result in hand. Every piece of execution happens in your process, under your control, in code you wrote.

Something to read

From here on we will work on a small, real, slightly broken program. Create it now.

Listing: `demo/src/slugify.ts`

```ts
// Convert a piece of text into a URL slug.
export function slugify(input: string): string {
  return input
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, "-")
    .replace(/^-+|-+$/g, "");
}
```

Listing: `demo/src/cli.ts`

```ts
import { slugify } from "./slugify.js";

const input = process.argv.slice(2).join(" ");

if (input.length === 0) {
  console.error("usage: tsx demo/src/cli.ts <text>");
  process.exit(1);
}

console.log(slugify(input));
```

Try it:

```
npx tsx demo/src/cli.ts "Hello There World"
```

That prints `hello-there-world`, so now try the input the next ten chapters use:

```
npx tsx demo/src/cli.ts "Crème Brûlée"
```

You get `cr-me-br-l-e`. Every accented character fell outside `[a-z0-9]` and was replaced by a hyphen. It passes review because the regular expression looks obviously correct, and reaches production because nobody tested a word with an accent in it.

Fixing this is the job we will hand to a model repeatedly from here on.

What a tool is

A **tool** is a function in *your* program that you have described to the model in a form it can request, and nothing more than that. It is not an installed extension, it lives nowhere but in your own source file, and the model gains no capability it did not have; what it gains is a vocabulary for asking you to do something.

Tool use is the whole mechanism: the model emitting a structured request to call one of your described functions, your program executing it, and the result being fed back into the conversation.

Describing a function means writing a **tool schema**: a JSON object with three parts.

- `name` — the identifier the model will use when asking for it.
- `description` — plain English saying what it does and when to reach for it.
- `input_schema` — the shape of the arguments, written as **JSON Schema**, a standard notation for describing the structure of JSON data: what fields exist, what types they hold, which are required.

Here is one:

```
const readFileTool: Anthropic.Tool = {
  name: "read_file",
  description:
    "Read a UTF-8 text file from the project and return its contents. " +
    "Paths are relative to the project root.",
  input_schema: {
    type: "object",
    properties: {
      path: {
        type: "string",
        description: "Project-relative path, e.g. demo/src/slugify.ts",
      },
    },
```

```
    required: ["path"],
  },
};
```

The `description` fields are not documentation. They are the only thing the model has to decide *whether* to call this and *what* to pass. A vague description produces a model that calls the wrong function with plausible-looking arguments.

> **Pro Tip:** Write the description as if for a competent new colleague who cannot ask follow-up questions and cannot see the implementation. Say what it does, what the arguments mean, what it returns, and when *not* to use it.

The round trip

Now the mechanism itself, in the order it happens.



Figure 2.1: The model asks; your code executes; the result goes back. The model never touches your machine.

Listing: `agentic/tool.ts`

```
import fs from "node:fs";
import path from "node:path";
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

requireApiKey();

const client = new Anthropic();
```

```ts
const ROOT = process.cwd();

const readFileTool: Anthropic.Tool = {
  name: "read_file",
  description:
    "Read a UTF-8 text file from the project and return its contents. " +
    "Paths are relative to the project root.",
  input_schema: {
    type: "object",
    properties: {
      path: {
        type: "string",
        description: "Project-relative path, e.g. demo/src/slugify.ts",
      },
    },
    required: ["path"],
  },
};

// The actual implementation. Note that it validates its own input:
// the arguments came from a model, not from your code.
function readFile(input: unknown): string {
  const { path: relative } = input as { path?: unknown };
  if (typeof relative !== "string") {
    throw new Error("read_file: 'path' must be a string");
  }
  const resolved = path.resolve(ROOT, relative);
  if (!resolved.startsWith(ROOT + path.sep)) {
    throw new Error(`read_file: refusing to read outside the project`);
  }
  return fs.readFileSync(resolved, "utf8");
}

async function main(): Promise<void> {
  const messages: Anthropic.MessageParam[] = [
    {
      role: "user",
      content:
        "Read demo/src/slugify.ts, then say in one sentence what it does " +
        "to accented characters such as è.",
    },
  ];

  // --- First request: the model decides it needs the file.
  const first = await client.messages.create({
    model: MODEL,
    max_tokens: 2048,
```

```
  tools: [readFileTool],
  messages,
});

console.log("stop_reason:", first.stop_reason);

if (first.stop_reason !== "tool_use") {
  console.log("The model answered without reading anything.");
  return;
}

// The model's request has to go into the history verbatim.
messages.push({ role: "assistant", content: first.content });

// --- Execute every request it made, collecting one result each.
const results: Anthropic.ToolResultBlockParam[] = [];

for (const block of first.content) {
  if (block.type !== "tool_use") continue;

  console.log(`call: ${block.name}(${JSON.stringify(block.input)})`);

  try {
    results.push({
      type: "tool_result",
      tool_use_id: block.id,
      content: readFile(block.input),
    });
  } catch (error) {
    results.push({
      type: "tool_result",
      tool_use_id: block.id,
      content: error instanceof Error ? error.message : String(error),
      is_error: true,
    });
  }
}

// All results go back as ONE user message.
messages.push({ role: "user", content: results });

// --- Second request: same conversation, now with the file in it.
const second = await client.messages.create({
  model: MODEL,
  max_tokens: 2048,
  tools: [readFileTool],
  messages,
```

```
  });

  for (const block of second.content) {
    if (block.type === "text") {
      console.log(block.text);
    }
  }
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Run it:

```
npx tsx agentic/tool.ts
```

You should see `stop_reason: tool_use`, then a line showing the model asking for `demo/src/slugify.ts`, then a sentence telling you accented characters are replaced by hyphens. It read your file, and it did so by asking.

The parts of the round trip

`stop_reason`

Every response carries a **stop reason**: a short string saying *why* the model stopped producing text. In Chapter 1 you never looked at it because there was only one answer. Now there are several, and the program branches on it.

`stop_reason`	Meaning
`end_turn`	It finished naturally. This is a real answer.
`tool_use`	It wants you to call something and send back the result.
`max_tokens`	It hit your ceiling and was cut off mid-thought.
`pause_turn`	A long-running server-side operation paused; re-send to continue.
`refusal`	It declined on safety grounds.

Two more exist, `stop_sequence` and `model_context_window_exceeded`, and neither arises in these chapters; the loop in Chapter 3 treats any stop reason it does not name as an error.

Treating `tool_use` as if it were an answer prints an empty string: the response held no text block at all.

Rule: Branch on `stop_reason` before you read `content`, never after it and never instead of it. The stop reason tells you what kind of thing the content *is*.

The `tool_use` block

When the stop reason is `tool_use`, the response content contains one or more **`tool_use` blocks**. Each is a content block with:

- `id` — a unique identifier for this specific request, like `toolu_01A9...`
- `name` — which tool it wants

- `input` — the arguments, as an object matching the schema you supplied

The response can also contain ordinary text blocks alongside them; models often narrate what they are about to do. That text is real output and you may print it.

`tool_result`

Your reply is a `user` message whose content is a list of `tool_result` blocks. Each one carries:

- `tool_use_id` — the `id` from the request it answers. This is the join key. Get it wrong and the API rejects the request.
- `content` — what your function returned, as text.
- `is_error` — optional, `true` when your function failed.

Three rules govern this reply:

1. **Every `tool_use` block gets a `tool_result` block.** No exceptions, including ones that failed. A missing result is a protocol error, not a silent omission.
2. **All the results go in one message**, not one message each. Splitting them is accepted by the API, and it teaches the model that you cannot handle more than one at a time.
3. **The assistant message containing the requests must be in the history**, verbatim, before the results. That is the `messages.push({ role: "assistant", content: first.content })` line, and forgetting it produces a `400` complaining about results with nothing to answer.

Failing on purpose

Look again at the `catch` branch: when `readFile` throws, we neither crash nor send an empty result, but send the error message back as a `tool_result` with `is_error: true`.

Given `read_file: 'path' must be a string`, or `ENOENT: no such file or directory`, the model can correct its own argument and ask again; given silence, it cannot.

> **Example:** Change the file name in the request to `demo/src/slugfy.ts` and run it again. The tool throws `ENOENT`, that text goes back as an errored result, and the model, with no other prompting, typically asks to read a directory listing or tries the corrected spelling.

Those are all the moving parts. Here is a request on the second pass of a loop — after the model has asked to read a file and your code has sent the contents back:

```
{
  "model": "claude-opus-5",
  "max_tokens": 1024,
  "system": "You fix small TypeScript bugs.",

  "tools": [
    { "name": "read_file",
      "description": "Read a text file and return it.",
      "input_schema": {
        "type": "object",
        "properties": { "path": { "type": "string" } },
        "required": ["path"] } }
  ],

  "messages": [
    { "role": "user", "content": "Fix accent folding" },

    { "role": "assistant", "content": [
      { "type": "tool_use", "id": "tu_01",
        "name": "read_file",
        "input": { "path": "src/slugify.ts" } } ] },

    { "role": "user", "content": [
      { "type": "tool_result",
        "tool_use_id": "tu_01",
        "content": "export function slugify(s) { ..." } ] }
  ]
}
```

Figure 2.2: One request, whole. `model` and `max_tokens` are settings, `system` is the standing instruction, and `tools` is the set the model may choose from — each with the argument shape it must fill in. Everything the model has said and everything your code has answered lives in `messages`, in order; that array is the only memory there is, and it grows by two entries on every pass.

Nothing in it is hidden state. A request that produces a given response today produces the same conversation on any machine that sends the same object, which is why a failing run can be reproduced by saving the request that caused it.

Five keys, in the order they appear:

- `model`: which model answers, from `MODEL` in `setup.ts`.
- `max_tokens`: the ceiling on the reply, required, and reported back as a stop reason if the reply reaches it.
- `system`: the standing instruction, kept out of `messages` so the model cannot read it as something a participant said.
- `tools`: what you are willing to run. Each entry carries a `name` the model will use when asking, a `description` it decides *whether* to ask on, and an `input_schema` giving the shape of the arguments and which of them are required.
- `messages`: everything said so far, oldest first.

That last one is the only one that is not a setting. The three entries in it came from three different moments:

1. **`user`** — the task, which you sent.
2. **`assistant`** — the `tool_use` block, which the model returned. Its `id` is how the model will recognise the answer.
3. **`user`** — the `tool_result` block, which *your code* added after running `read_file`. It carries the same `id` in `tool_use_id`, and the file's contents as its `content`.

So the object above is a snapshot that contains a history. Entry 2 arrived in a response and entry 3 was never in one at all, and both are being sent again now, along with entry 1, because the model kept none of them. Every later round trip sends this array with one more pair on the end.

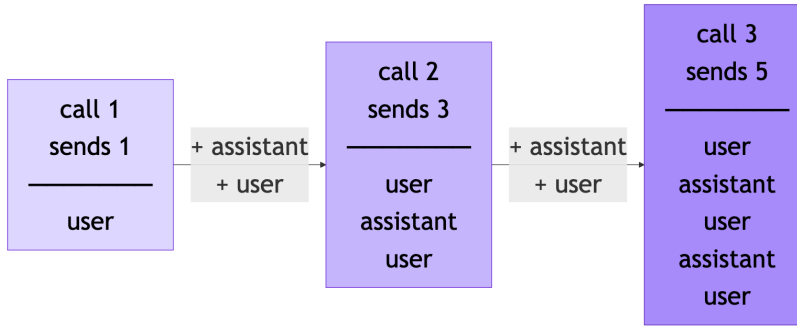


Figure 2.3: What `messages` holds at each call. Your code appends two entries per round trip, never one: the model's reply, and the `tool_result` your code produces by running what the reply asked for. So the array is odd-numbered at every call — 1, 3, 5 — and the request you send on call 3 still carries the task you typed before call 1.

Several at once

A single response can contain several `tool_use` blocks. This is **parallel tool use**: when the model can see that two operations are independent, such as reading this file and that file, it asks for both in one response rather than waiting for the first.

`tool.ts` already handles it, because the `for` loop collects every block and the results array grows to match. What it does *not* do is run them concurrently; it executes them one after another, which is fine for file reads. In Chapter 7 each operation is a several-second API call, where `Promise.all` applies, and the results array must end up in the same order as the requests, which is what mapping over the blocks, rather than pushing from a loop, produces.

The model chooses the arguments

This is the reason `readFile` has six lines of validation in front of two lines of work.

The arguments to your tool are produced by a language model, from text that may include a file you did not write, a bug report someone else filed, or the output of a previous tool call, so they are not trusted input: they have the same standing as a query string on a public HTTP endpoint.

```
const resolved = path.resolve(ROOT, relative);
if (!resolved.startsWith(ROOT + path.sep)) {
  throw new Error(`read_file: refusing to read outside the project`);
}
```

Without those two lines, `../../.ssh/id_rsa` is a valid path, `path.resolve` happily produces it, and your tool reads it and sends the contents to a third party. Nothing malicious has to be going on for that to happen: a model that has lost track of where the project root is will do it by accident.

> **Warning:** The safety of a tool is a property of your implementation, not of the schema. A schema saying `"path": {"type": "string"}` is not a constraint; it is a hint that the model usually respects. Validate inside the function, every time, as if the argument arrived from the internet.

Three habits apply from your first tool:

- **Constrain the surface.** `read_file` beats `run_shell_command` for the same reason a parameterised query beats string concatenation.
- **Resolve and check paths** against a known root, as above — and know what that check does not cover. `path.resolve` normalises

`..`; it does not follow symbolic links, and `readFileSync` does. If the project can contain a symlink pointing outside it — an npm workspace, an `npm link`, anything you did not put there yourself — compare `fs.realpathSync` of the target instead. A lexical prefix test is a string comparison, not a boundary.

- **Make destructive operations conspicuous.** In Chapter 3 you will add a `write_file`, and it will print what it is about to overwrite before it does.

A second tool

One tool demonstrates the mechanism; a useful program has a small set.

Here is what a second one looks like. Read it rather than typing it, because `tool.ts` is finished and the *set* these two belong to is built in the next chapter:

```
const listFilesTool: Anthropic.Tool = {
  name: "list_files",
  description:
    "List the files in a directory of the project, one per line. " +
    "Use this when you are unsure what a file is called.",
  input_schema: {
    type: "object",
    properties: {
      dir: {
        type: "string",
        description: "Project-relative directory, e.g. demo/src",
      },
    },
    required: ["dir"],
  },
};
```

Two tools is already enough for the model to make a choice, and the choice is made from your `description` fields and nothing else. If `read_file` said only "reads a file" and `list_files` said only "lists files,"

a request mentioning "the files in demo/src" is ambiguous, and you get whichever the model guesses.

A single `run_command` tool is maximally capable and maximally unbounded: every safety property of your program collapses into "did the model choose a safe command." A narrow tool fails in a narrow way. The set of tools you expose defines what your program can do: not what it will do, but what it *can* do.

Steering the choice

Two request fields adjust how the model decides.

`tool_choice` controls whether it may call anything at all: `{ type: "auto" }` is the default and lets it decide; `{ type: "none" }` forbids it. Some models also accept `{ type: "tool", name: "read_file" }` to force a specific call, and some current models reject that form with a `400` — if you need a guaranteed call, the portable approach is `auto` plus a system prompt sentence saying which function to use.

`strict: true` on a tool definition makes the API guarantee the arguments validate against your schema exactly. It requires `additionalProperties: false` and a complete `required` list. Enable it for tools whose arguments you would otherwise have to defend against in code, and keep the validation anyway, because a schema-valid path is still allowed to point at `/etc/passwd`.

What you have, and what is missing

`tool.ts` performs a single round trip and then stops: model asks, you answer, model replies, program exits.

Watch what happens when you ask for something that needs two steps. Change the request to:

```
Read demo/src/slugify.ts and demo/src/cli.ts, then tell me whether the CLI
would print an empty line for the input "///".
```

The model asks to read one file and you send it. It then asks for the second file, and `tool.ts`, whose second loop prints only text blocks, prints nothing, or a sentence of narration, and exits. The program does nothing with a second `tool_use`, because it was written with the number two hard-coded into it.

The fix is not more code but *less* structure: stop writing "first request, second request" and write "keep going until it says it is done."

That is a loop, and it is the entire subject of the next chapter, which is also where the thing you have been building finally gets its name.

This page intentionally left blank

Chapter 3

The Loop

This chapter builds the loop, in `agentic/loop.ts`. The program at the end of Chapter 2 could do one thing and then stopped, because the number two was baked into it.

The definition, before the code

An **agent** is a program that lets a model choose its own next action, repeatedly, until the model decides it is finished.

The definition stops there: no autonomy, no planning, no reasoning, just a loop with a model in the condition. Everything else people mean by the word is a property of the tools you gave it and the instructions you wrote.

The cycle that loop runs has a name of its own. The **agentic loop** is: send the conversation, read the stop reason, execute any requested tools, append the results, send again. One pass through that cycle is a **turn**.

The model does not run that loop. Your code runs it, and the model only

decides whether there is another pass, so every runaway, every repetition and every infinite spiral happens inside a `for` statement you wrote and can bound.

Here is the shape, with everything but the essentials removed:

```
while (true) {
  const response = await client.messages.create({ model, max_tokens, tools,
  messages });

  if (response.stop_reason === "end_turn") break;

  messages.push({ role: "assistant", content: response.content });
  messages.push({ role: "user", content: executeEveryToolCall(response.content)
});
}
```

Six lines. Read it once more: the entire difference between Chapter 2's program and an agent is that the second request is inside a loop instead of being written out twice.

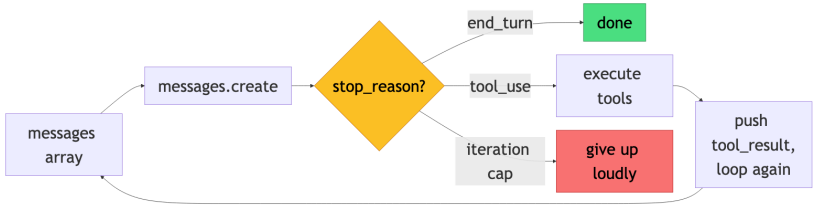


Figure 3.1: One pass is a turn. The stop reason decides whether there is another one.

Building it properly

The program comes in two files. The first holds the loop and the tools, so that every later chapter can import it instead of retyping it, and the second is the command you actually run.

Listing: agentic/loop.ts

```ts
import fs from "node:fs";
import path from "node:path";
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

// Checked once, here, because every entry point imports this module.
// A missing key becomes one line instead of a stack trace.
requireApiKey();

const client = new Anthropic();
const ROOT = process.cwd();

export const MAX_ITERATIONS = 25;

export interface AgentRun {
  text: string;
  iterations: number;
  inputTokens: number;
  outputTokens: number;
}

// --- Tools -------------------------------------------------------------

function projectPath(
  input: unknown,
  field: string,
  toolName: string,
): { relative: string; absolute: string } {
  if (typeof input !== "object" || input === null) {
    throw new Error(`${toolName}: expected an object of arguments`);
  }
  const value = (input as Record<string, unknown>)[field];
  if (typeof value !== "string" || value.length === 0) {
    throw new Error(`${toolName}: '${field}' must be a non-empty string`);
  }
  const absolute = path.resolve(ROOT, value);
  // The root itself resolves to ROOT with no trailing separator, so a bare
  // prefix test rejects `.` - which is the first thing list_files is asked for.
  if (absolute !== ROOT && !absolute.startsWith(ROOT + path.sep)) {
    throw new Error(`${toolName}: refusing to touch anything outside the
    project`);
  }
  return { relative: value, absolute };
}
```

```
function readFile(input: unknown): string {
  const { absolute } = projectPath(input, "path", "read_file");
  return fs.readFileSync(absolute, "utf8");
}

function listFiles(input: unknown): string {
  const { relative, absolute } = projectPath(input, "dir", "list_files");
  const entries = fs.readdirSync(absolute, { withFileTypes: true });
  if (entries.length === 0) {
    return `${relative} is empty.`;
  }
  return entries
    .map((entry) => (entry.isDirectory() ? `${entry.name}/` : entry.name))
    .join("\n");
}

function writeFile(input: unknown): string {
  const { relative, absolute } = projectPath(input, "path", "write_file");
  const content = (input as Record<string, unknown>)["content"];
  if (typeof content !== "string") {
    throw new Error("write_file: 'content' must be a string");
  }
  const existed = fs.existsSync(absolute);
  fs.mkdirSync(path.dirname(absolute), { recursive: true });
  fs.writeFileSync(absolute, content, "utf8");
  return `${existed ? "Overwrote" : "Created"} ${relative} ` +
    `(${content.length} characters).`;
}

export const FILE_TOOLS: Anthropic.Tool[] = [
  {
    name: "read_file",
    description:
      "Read a UTF-8 text file from the project and return its contents. " +
      "Paths are relative to the project root.",
    input_schema: {
      type: "object",
      properties: {
        path: { type: "string", description: "e.g. demo/src/slugify.ts" },
      },
      required: ["path"],
    },
  },
  {
    name: "list_files",
    description:
      "List the entries in a project directory, one per line. " +
```

```
      "Directories are shown with a trailing slash. " +
      "Use this when you are unsure what a file is called.",
    input_schema: {
      type: "object",
      properties: {
        dir: { type: "string", description: "e.g. demo/src" },
      },
      required: ["dir"],
    },
  },
  {
    name: "write_file",
    description:
      "Write a UTF-8 text file, replacing it entirely if it already exists. " +
      "You must supply the complete new contents, not a fragment.",
    input_schema: {
      type: "object",
      properties: {
        path: { type: "string", description: "e.g. demo/src/slugify.ts" },
        content: { type: "string", description: "The complete file contents" },
      },
      required: ["path", "content"],
    },
  },
];

// The same set with the one destructive tool removed. Anything that is only
// supposed to read gets this, and then it cannot do otherwise.
export const READ_ONLY_TOOLS: Anthropic.Tool[] = FILE_TOOLS.filter(
  (tool) => tool.name !== "write_file",
);

function execute(name: string, input: unknown): string {
  switch (name) {
    case "read_file":
      return readFile(input);
    case "list_files":
      return listFiles(input);
    case "write_file":
      return writeFile(input);
    default:
      throw new Error(`Unknown tool: ${name}`);
  }
}

// --- The loop ----------------------------------------------------
```

```ts
function trace(line: string): void {
  // Traces go to stderr so that stdout stays usable as a result.
  process.stderr.write(`${line}\n`);
}

export async function runAgent(
  system: string,
  task: string,
  tools: Anthropic.Tool[] = FILE_TOOLS,
  maxIterations: number = MAX_ITERATIONS,
): Promise<AgentRun> {
  const messages: Anthropic.MessageParam[] = [{ role: "user", content: task }];
  let inputTokens = 0;
  let outputTokens = 0;

  for (let iteration = 1; iteration <= maxIterations; iteration++) {
    const response = await client.messages.create({
      model: MODEL,
      max_tokens: 8192,
      system,
      tools: tools.length > 0 ? tools : undefined,
      messages,
    });

    inputTokens += response.usage.input_tokens;
    outputTokens += response.usage.output_tokens;

    const text = response.content
      .filter((b): b is Anthropic.TextBlock => b.type === "text")
      .map((b) => b.text)
      .join("");

    if (response.stop_reason === "end_turn") {
      return { text, iterations: iteration, inputTokens, outputTokens };
    }

    if (response.stop_reason === "max_tokens") {
      throw new Error(
        `Cut off at max_tokens on iteration ${iteration}. Raise max_tokens ` +
          `or ask for less at a time.`,
      );
    }

    if (response.stop_reason === "refusal") {
      throw new Error("The model declined this request.");
    }
```

```
    messages.push({ role: "assistant", content: response.content });

    if (response.stop_reason === "pause_turn") {
      continue;
    }

    const results: Anthropic.ToolResultBlockParam[] = [];

    for (const block of response.content) {
      if (block.type !== "tool_use") continue;
      trace(`  [${iteration}] ${block.name} ${JSON.stringify(block.input)}`);
      try {
        results.push({
          type: "tool_result",
          tool_use_id: block.id,
          content: execute(block.name, block.input),
        });
      } catch (error) {
        trace(`  [${iteration}] failed: ${String(error)}`);
        results.push({
          type: "tool_result",
          tool_use_id: block.id,
          content: error instanceof Error ? error.message : String(error),
          is_error: true,
        });
      }
    }

    if (results.length === 0) {
      throw new Error(
        `Stopped with '${response.stop_reason}' and no tool call to answer.`,
      );
    }

    messages.push({ role: "user", content: results });
  }

  throw new Error(
    `Gave up after ${maxIterations} iterations without reaching end_turn.`,
  );
}
```

Listing: agentic/agent.ts

```
import { runAgent } from "./loop.js";
```

```
const SYSTEM = [
  "You are a careful software engineer working in a TypeScript project.",
  "Read before you write. When you change a file, write the complete new",
  "contents, never a fragment. When you are finished, state plainly what you",
  "changed and why.",
].join(" ");

async function main(): Promise<void> {
  const task = process.argv.slice(2).join(" ");

  if (task.length === 0) {
    console.error('usage: tsx agentic/agent.ts "<task>"');
    process.exit(1);
  }

  const run = await runAgent(SYSTEM, task);

  console.log(run.text);
  console.log(
    `\n[${run.iterations} iterations, ` +
      `in: ${run.inputTokens} out: ${run.outputTokens}]`,
  );
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Now run it against the bug from Chapter 2:

```
npx tsx agentic/agent.ts \
  "demo/src/slugify.ts mangles accented characters: 'Crème Brûlée' becomes
   'cr-me-br-l-e'. Fix it so accents are folded to their base letters."
```

Watch the trace lines go past on stderr: it lists a directory, reads the file, writes a new one, and tells you what it did. Check the result:

```
npx tsx demo/src/cli.ts "Crème Brûlée"
```

If you get `creme-brulee`, a program you wrote decided on its own which of your functions to call and in what order, and changed your source

code. That is what an agent is, and it took about two hundred and fifty lines to build one.

Why the loop is shaped the way it is

Six decisions in `runAgent` are not obvious.

Five of the six are decisions about a single branch.

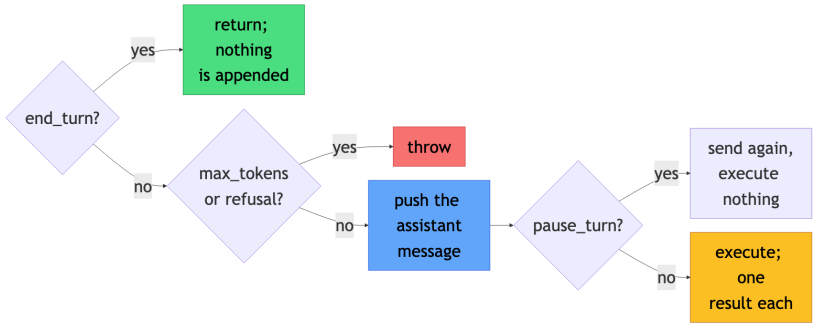


Figure 3.2: The three checks `runAgent` makes on the stop reason, in the order it makes them. The first two answer and leave the conversation untouched; the assistant message goes in only after them, which is what leaves `pause_turn` something to re-send and `tool_use` something for its results to answer.

The assistant message goes back before the results

`messages.push({ role: "assistant", content: response.content })` happens before any tool runs. It has to: the `tool_result` blocks you are about to send refer to `tool_use` ids that must already be present in the conversation. Push the results without the request that prompted them and the API returns a `400`.

`end_turn` returns before that push

When the model is finished, we return without appending anything, because nothing else is going to read that history. If you later want the full transcript out of `runAgent` — Chapter 9 will — return `messages` too.

`max_tokens` is an error, not a completion

A response cut off by the ceiling is a truncated answer that *looks* like an answer: its text field is populated and its content is real, and then it simply stops mid-thought. Treating that as success is how a workflow reports a half-written file as a finished change.

`pause_turn` continues rather than executing

Some server-side operations pause mid-flight, leaving you nothing to execute. The correct response is to append the assistant message and send the conversation back unchanged, which is what `continue` does after the push. Get this wrong and you either crash on an unrecognised stop reason or drop, with no warning, work the model had already started.

An empty result list is fatal

If the stop reason was neither `end_turn` nor `pause_turn` and yet no tool was requested, something has happened that this code does not understand. Sending an empty user message would be a protocol error; looping again would send the same conversation forever. The code throws, and the error text names the stop reason.

There is a hard iteration cap

The cap earns a section of its own, immediately below.

Termination, and the failure that costs money

A **termination condition** is whatever makes the loop stop. `runAgent` has three, and you need all three:

1. **The model says it is done** — `end_turn`, the normal exit.
2. **Something went wrong** — `max_tokens`, `refusal`, or an unanswerable stop reason, the exceptional exit.
3. **The iteration cap is reached** — `maxIterations`, the backstop.

Leaving the third one out produces a **runaway loop**: an agentic loop that never reaches `end_turn` and keeps calling the API until something external stops it. Two mundane causes account for most of them:

- A tool that fails in a way the model reads as retryable. It asks to read `src/index.ts`; there is no such file; it gets `ENOENT`; it tries `./src/index.ts`; then `src/index.tsx`; then `src/`. Forever.
- An instruction that cannot be satisfied. "Make all the tests pass" when one test asserts something impossible produces a model that edits, checks, fails, and edits again, indefinitely.

Warning: Each pass through the loop re-sends the entire conversation, which grows every time. So a runaway loop's cost is not linear in iterations, it is roughly quadratic: iteration 30 sends thirty rounds of history. A loop left running overnight produces no output at all: it never reached `end_turn`.

Twenty-five makes a defensible default, and the right number is "more than the most complicated legitimate task you expect." When you hit it, that is information: either the cap is too low or the task is not converging.

Rule: The loop you are building here is meant to run unattended, and a loop like that does not ship without a hard iteration cap and a hard spend expectation. "I will add it later" does not count: the version without a cap is the version that ends up running when you are not there to stop it.

`write_file`

`write_file` is different in kind from the other two.

`read_file` and `list_files` only observe, so if the model makes a bad choice there you have wasted a few hundred tokens. `write_file` replaces the entire contents of a file on your disk with text a model produced. If the model makes a bad choice there, work is gone.

Three protections apply before you run `agent.ts` on anything you care about:

1. **A clean git tree before every run.** `git status` should be empty when you start. Then `git diff` shows exactly what the run did, and `git checkout .` undoes all of it. A **diff** is the line-by-line difference between two states of a file or a tree — what was removed, what was added, and nothing else. From Chapter 7 onward every independent check you build reads one, and it starts being useful here.
2. **The project-root check** you already have in `projectPath`.
3. **Complete contents, not fragments** — which is why the tool description says so twice, because a model that tries to write a partial file leaves a broken one.

Example: Run `agent.ts` with a task it cannot do — "add a `--json` flag to the CLI and make it emit RFC 8259 output with a schema header." Then run `git diff`. The diff typically contains real, plausible, *unrequested* changes to files you did not mention. Nothing failed and no error was raised; the diff is simply not what you asked for.

What an agent is not

Four things it does *not* mean, each of which you can check against the two hundred and fifty lines you just typed:

It is not planning. Nothing in `runAgent` maintains a plan. The model chooses one action, sees the result, and chooses again. When output looks planned, that is because a model good at predicting text produces text that reads like a plan and then usually follows it. No plan object exists anywhere in the system, which is exactly why Chapter 6 has you build one.

It is not memory. The model is exactly as stateless as it was in Chapter 1. What looks like memory within a run is the `messages` array; what looks like memory across runs is nothing at all, because `runAgent` starts with a fresh array every time. Run `agent.ts` twice on related tasks and the second run knows nothing of the first.

It is not learning. No weights change between one iteration and the next. A model that made a mistake in iteration 3 and was corrected in iteration 4 has learned nothing durable; it is simply reading a conversation in which that correction appears. Start a new run and it will make the same mistake.

It is not supervision. Nothing in the loop evaluates whether the work

is any good. `end_turn` means "the model has stopped producing tokens," and it arrives on a perfect fix and a fabricated one alike.

Rule: `end_turn` is a statement about token generation, not about correctness. Treating a completed run as a successful one promotes a stop reason into a quality judgement.

Watching what it does

Notice that `trace` writes to stderr and results go to stdout. That separation costs one line and buys two things.

The first is that you can use the program in a pipeline: `npx tsx agentic/agent.ts "..." > result.txt` captures the answer without twenty lines of tool chatter mixed into it. Once a loop runs unattended, the trace is the only record of what happened, and three questions are asked of it: which tools did it call, in what order, and where did it first go wrong.

A trace line per tool call, with the arguments, answers all three.

Pro Tip: When a run does something baffling, do not start by re-reading your system prompt. Start by reading the trace from the top and locating the first tool result that came back wrong or empty. Usually the model behaved sensibly given a bad input, and the bad input came from your own function.

What you have built

Two hundred and fifty lines of TypeScript that reads your code, edits it, and stops when it thinks it is done. Everything after this chapter is about the gap between "thinks it is done" and "is done."

Before we get there, though, there is a practical ceiling to deal with. Every pass through that loop makes the conversation longer, and the conversation is both what you pay for and what has to fit. The next chapter is about what the model knows, what it forgets, and what you can do about both.

This page intentionally left blank

Chapter 4

What It Knows and Forgets

This chapter is about the token count in the trace output of `agent.ts`, and about the fact that it is the only hard constraint in the entire system.

The shape of the problem

Run the agent on a real task and watch the token counts. Something like this:

```
[1] list_files {"dir":"demo/src"}
[2] read_file {"path":"demo/src/slugify.ts"}
[3] read_file {"path":"demo/src/cli.ts"}
[4] write_file {"path":"demo/src/slugify.ts","content":"..."}

[5 iterations, in: 21430 out: 1880]
```

Twenty-one thousand input tokens went to a task whose input was one sentence and two small files.

They came from sending the conversation five times. Iteration 1 sent the system prompt, the tool schemas, and the task. Iteration 2 sent all

of that again, plus the model's first response, plus the directory listing. Iteration 3 sent all of *that* again, plus a file, and iterations 4 and 5 carried on the same way.

In an agentic loop you pay for the conversation once per turn. A twenty-turn run over a transcript that ends at 40,000 tokens has sent something in the region of 400,000 input tokens, because the average turn carried half the final transcript. Cost is quadratic in turn count.

Every technique in Chapters 6 through 11 adds turns and adds readers.

Two limits, not one

The **context window**, from Chapter 1, is a *hard* limit: the maximum tokens in a single request plus its response. Exceed it and the API returns an error. Current models allow so much room here that none of these exercises comes close.

A **token budget** is a *soft* limit you impose: the number of tokens you have decided a piece of work is allowed to consume in total, across every request it makes. Nothing enforces it but you, and it exists because the context window is generous enough that a runaway loop can burn a great deal of money without ever tripping the hard limit.

The context window protects the model and the token budget protects you, and only the first of them is switched on by default.

Measuring it

Write something that shows you the arithmetic rather than describing it.

Listing: `agentic/budget.ts`

```ts
import fs from "node:fs";
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

requireApiKey();

const client = new Anthropic();

// Dollars per million tokens for the model named in setup.ts.
// Check the current figures on Anthropic's pricing page; they change,
// and an estimate you have stopped believing is worse than none.
// input_tokens is only the part of the prompt that was neither written
// to the cache nor read from it; the three prompt counts add up.
export const PRICE_PER_MTOK = {
  input: 5,
  cacheWrite: 6.25,
  cacheRead: 0.5,
  output: 25,
};

export function estimateCost(usage: {
  input_tokens: number;
  output_tokens: number;
  cache_creation_input_tokens?: number | null;
  cache_read_input_tokens?: number | null;
}): number {
  const written = usage.cache_creation_input_tokens ?? 0;
  const cached = usage.cache_read_input_tokens ?? 0;
  return (
    (usage.input_tokens / 1_000_000) * PRICE_PER_MTOK.input +
    (written / 1_000_000) * PRICE_PER_MTOK.cacheWrite +
    (cached / 1_000_000) * PRICE_PER_MTOK.cacheRead +
    (usage.output_tokens / 1_000_000) * PRICE_PER_MTOK.output
  );
}

// The soft limit. Nothing in the API knows about this number.
export const TOKEN_BUDGET = 200_000;

export function assertWithinBudget(spent: number, label: string): void {
  if (spent > TOKEN_BUDGET) {
    throw new Error(
      `${label}: token budget exceeded - ${spent} of ${TOKEN_BUDGET} used.`,
    );
  }
}
```

```
// Identical on every request, and long enough to be worth caching. The
// house rules alone are a few dozen tokens, and a prefix that short is
// never stored: the floor is a few hundred tokens and varies by model.
// The file under discussion is what carries this block over it.
const HOUSE_RULES = [
  "You are a reviewer for a small TypeScript project.",
  "Answer in at most three sentences.",
  "Never speculate about code you have not been shown.",
  "If a question cannot be answered from what you have, say so plainly.",
  "Prefer concrete nouns from the code over general description.",
].join("\n");

const SOURCE = fs.readFileSync("agentic/loop.ts", "utf8");

const CONTEXT =
  `${HOUSE_RULES}\n\nThe file under discussion, agentic/loop.ts:\n\n${SOURCE}`;

async function main(): Promise<void> {
  const messages: Anthropic.MessageParam[] = [];
  let spent = 0;
  let dollars = 0;

  const questions = [
    "What does runAgent do when the stop reason is max_tokens?",
    "Which tool is missing from READ_ONLY_TOOLS, and why?",
    "How many ways can the loop stop?",
    "Which of those would a runaway loop hit first?",
    "Summarise this conversation in one sentence.",
  ];

  for (const [index, question] of questions.entries()) {
    messages.push({ role: "user", content: question });

    const response = await client.messages.create({
      model: MODEL,
      max_tokens: 4096,
      system: [
        {
          type: "text",
          text: CONTEXT,
          cache_control: { type: "ephemeral" },
        },
      ],
      messages,
    });

    messages.push({ role: "assistant", content: response.content });
```

```
    const usage = response.usage;
    const cached = usage.cache_read_input_tokens ?? 0;
    const written = usage.cache_creation_input_tokens ?? 0;

    spent += usage.input_tokens + written + cached + usage.output_tokens;
    dollars += estimateCost(usage);

    console.log(
      `round ${index + 1}: ` +
        `fresh ${String(usage.input_tokens).padStart(6)}  ` +
        `cached ${String(cached).padStart(6)}  ` +
        `written ${String(written).padStart(6)}  ` +
        `out ${String(usage.output_tokens).padStart(5)}  ` +
        `total ${String(spent).padStart(7)}  ` +
        `$${dollars.toFixed(5)}`,
    );

    assertWithinBudget(spent, "budget.ts");
  }
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Run it and read the columns rather than the prose:

```
npx tsx agentic/budget.ts
```

The `fresh` column climbs every round, and it climbs by roughly the size of the previous question *and* the previous answer, because both are now history. Five short rounds are enough to see it.

Prompt caching

Look at the `cached` column: on round one it is zero while `written` is not, and from round two onward the two swap places.

Prompt caching stores the beginning of your request on Anthropic's servers so that identical beginnings on later requests do not have to be processed again. Cached input is billed at a small fraction of the normal input rate. You opt in by marking a block with `cache_control: { type: "ephemeral" }`, which says "cache everything from the start of the request up to and including this block."

Three facts govern it:

It is a prefix match, and it is exact. The cache is keyed on the literal sequence of tokens from the very beginning of the request. Change one character anywhere in that prefix — one character — and everything after the change is a cache miss. The order the API assembles things in is `tools`, then `system`, then `messages`, so a tool schema you rebuild with a different key order destroys the cache for everything behind it, without saying so.

Stable content goes first. House rules, tool schemas, a large document you are asking questions about: at the front, marked. Timestamps, request IDs, the actual question: after the last marked block. If you put `new Date().toISOString()` in your system prompt, your cache hit rate is zero and nothing tells you.

You verify it with `usage`. `cache_read_input_tokens` is the only proof you get. If it is zero across repeated requests that should share a prefix, something is invalidating it, and the fastest way to find out what is to shorten the prefix until the number goes up.

A prefix also has to be long enough to store. The floor is a few hundred tokens and differs from model to model, and a request under it is not cached and not warned about: `written` and `cached` both stay at zero. The house rules alone are far under it, which is why `budget.ts` puts `loop.ts` in the block.

> **Warning:** Writing to the cache costs a quarter *more* than sending the same tokens uncached: `written` in round one is billed at `cacheWrite`, and the dollars column shows it. Caching a prefix you use once is a small loss. It pays from the second use, which in an agentic loop is iteration two, and not at all on a one-shot call.

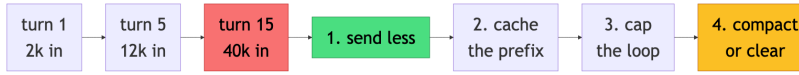


Figure 4.1: Where the tokens go, and the four levers you have. Caching makes re-sending cheap; the others make it smaller.

When the conversation is genuinely too long

Caching makes re-sending cheap without making the conversation any shorter, and some runs genuinely produce more history than can usefully be carried. Two API features address that, and they do opposite things to your data.

Compaction summarises. The API replaces older parts of the conversation with a compressed summary of them when the request approaches a threshold, keeping the gist and discarding the detail:

```
const response = await client.beta.messages.create({
  betas: ["compact-2026-01-12"],
  model: MODEL,
  max_tokens: 8192,
  messages,
  context_management: { edits: [{ type: "compact_20260112" }] },
});
```

One rule applies: append `response.content` — the whole array — back into `messages`, not just the text. Compaction returns a block that the

API needs on the next request to know what it already summarised. Extract the string and push that, and you have thrown away the state with no warning, in a way that looks fine until the conversation gets long.

Context editing deletes. Rather than summarising, it clears old tool results out of the conversation entirely, on the grounds that the contents of a file you read fifteen iterations ago are usually no longer needed:

```
const response = await client.beta.messages.create({
  betas: ["context-management-2025-06-27"],
  model: MODEL,
  max_tokens: 8192,
  tools,
  messages,
  context_management: { edits: [{ type: "clear_tool_uses_20250919" }] },
});
```

What separates the two is what each leaves of the conversation.

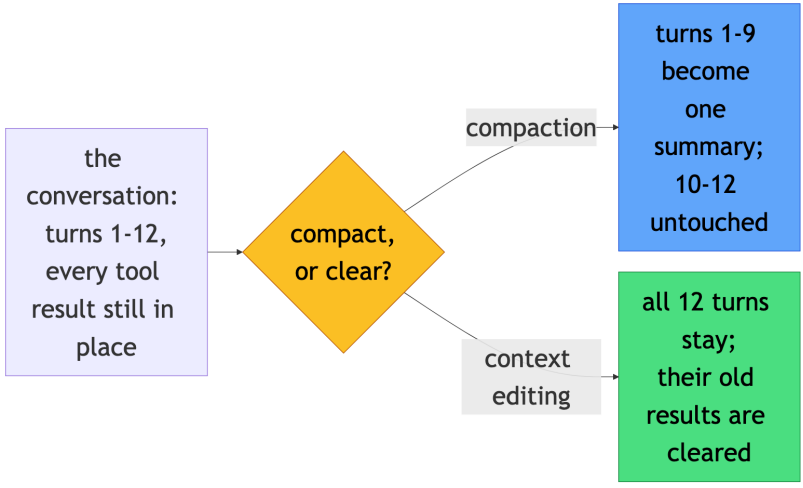


Figure 4.2: Two ways to shorten the same conversation, and what each leaves behind. Compaction replaces the older turns with one summary block and keeps the recent ones whole; context editing keeps all twelve turns and clears the tool results out of the old ones.

Summarise or delete. Compaction keeps a lossy trace of everything; context editing keeps a lossless trace of some things. For an agent that reads a lot of files and then edits one, deleting old file contents is usually right. For a long conversation whose early decisions still matter, summarising is.

Rule: Neither one is a substitute for putting less in. Both are recovery mechanisms for a conversation that has already grown too large. Reach for them after you have looked at what you are sending and concluded that it all needed to be there.

Retrieval

Retrieval is selecting the small part of a large body of material that a particular request actually needs, and sending only that. In its elaborate form it involves vector databases and similarity search. In the form you need here it is `read_file`.

"Give the model the whole repository, the window is huge" is now an available option.

Model performance is not flat across a long context. Material at the very beginning and the very end of a long input is attended to more reliably than material in the middle. As the input grows, the fraction of it that is irrelevant to the current question grows too, and the relevant part gets harder to pick out. This degradation as context grows is sometimes called **context rot**. It does not announce itself: you do not get an error or a warning. You get answers that are slightly worse, in ways that look like the model being unimpressive rather than the input being bad.

Example: Ask the agent "what does `slugify` do with accents?" in a fresh run, and then ask the identical question at the end of a twenty-iteration session that has read a dozen unrelated files. The second answer is typically vaguer, more hedged, and more likely to describe the file it read most recently.

The rule that follows: **when a piece of work needs a clean read of something, give it a new conversation rather than a longer one.** Chapter 7 gives every reviewer its own conversation for that reason.

What it knows before you tell it anything

So far this chapter has been about what you put *into* a request. There is a second body of knowledge in play, and misunderstanding what is in it causes a specific category of failure.

The weights were fitted on text gathered up to some point in time. That point is the **training cutoff**, and after it the model knows nothing — not because it refuses to answer, but because it has no way to distinguish "after my cutoff" from "does not exist." Ask about a library released last month and you will not get "I don't know." You will get a fluent, well-structured answer about a plausible library with plausible functions.

This applies to calling an API that changes. A model fitted before a parameter was removed will keep using it, and a model fitted before a new one existed will not reach for it. Both failures look identical from your side — code that reads perfectly and returns a `400`.

Warning: A model is most confidently wrong about the API of a fast-moving library, and next most about the library's own best practices. From inside the weights, the version it learned *is* the library.

There are two fixes, and they are the same fix at different scales.

Put the current truth in the context. If the request depends on how something works today, include today's documentation in the request rather than hoping. This is retrieval again, from the section above, applied to knowledge rather than to code: fetch the small relevant part and send it.

Pin what you can check. The model identifier in `setup.ts` is a string the model has opinions about and no current knowledge of, and so are your dependency versions. Keep anything a model might helpfully "correct" for you in one place you own, where a change is deliberate.

> **Rule:** Never let a model be the source of truth for a fact that has a source of truth. Versions, identifiers, prices, schemas, flags: look them up. The model is excellent at applying facts and unreliable at storing them.

Knowing before you send

You can price a request before making it.

```ts
const count = await client.messages.countTokens({
  model: MODEL,
  system: CONTEXT,
  messages,
});

console.log(`this request would cost ${count.input_tokens} input tokens`);
```

`countTokens` runs the same tokenizer the real request would, costs nothing, and returns the input side only. It is the right tool for two jobs: deciding whether a document is small enough to send whole, and checking a large prefix before you spend money caching it.

It is the wrong tool for estimating output, because nobody can estimate output — that is the model's choice, bounded only by your `max_tokens`. `budget.ts` measures what actually happened, every round, and keeps the total.

The four levers, in order

When a run costs too much or performs worse than it should, work down this list. It is ordered by benefit-per-effort.

1. **Send less.** Narrower tools, more specific tasks, and files read on demand rather than up front, which costs nothing and improves quality as well.
2. **Cache what repeats.** One `cache_control` marker on a stable prefix. Nearly free, purely a cost win, no downside once the prefix is genuinely stable.
3. **Cap the loop**, with the `maxIterations` from Chapter 3 and the `assertWithinBudget` you just wrote. This does not reduce the cost of a healthy run at all; it bounds the cost of a sick one, which is where the frightening numbers actually come from.
4. **Compact or clear.** Only when a legitimate run genuinely outgrows what you can send. This is the lever with a real quality cost attached, which is why it is last.

Pro Tip: Print running cost in every loop you write, from the first version. A number in a log records which iteration the cost came from.

What this chapter did not solve

You can now measure what a run costs, keep it from costing more than you meant, and give the model a clean slate when it needs one.

None of that touches whether the work is any good.

Everything in this chapter is about capacity: how much can go in, how much comes back, what it costs. The agent from Chapter 3, running comfortably inside its budget with a healthy cache hit rate, can still produce a fix that does not fix anything, describe a test it never ran, and finish with `end_turn` and an articulate summary of work it did not do.

That gap is what the next chapter puts in front of you.

This page intentionally left blank

Chapter 5

Why It Lies to You

This chapter runs experiments on the agent from the first four chapters, on your own machine, with the code you wrote. That agent reads your code, makes sensible choices, explains itself clearly, and fixed a real bug in Chapter 3. Everything from Chapter 6 to Chapter 11 is a response to what the experiments show.

The word, and why it is a bad word

A **hallucination** is output that is fluent, confident, well-formed, and false: not a crash, an error or a refusal, but a normal-looking answer that happens not to correspond to anything real.

It is a bad word for it, because it suggests a malfunction. A hallucinating person is a person whose perception has broken, and nothing here has broken. Go back to Chapter 1: the model produces the text its fitted function scores as a likely continuation. It has no separate faculty that checks whether the likely continuation is also the true one — no such faculty exists anywhere in the architecture. When the likely text and

the true text coincide, you call it a correct answer. When they diverge, you call it a hallucination. From the inside, those two cases are indistinguishable, because they are the same operation. No setting turns it off, because it was never a setting.

Plausible output is output whose surface features — structure, tone, specificity, confidence — are indistinguishable from correct output. A fabricated function name is spelled correctly, follows the library's naming conventions, and takes the arguments you would expect it to take. A summary of a file the model never read has the same shape as a summary of a file it did.

That is why "I'll just read what it says" does not work as a quality control, and why the chapters ahead give you something *outside* the text to check against — a plan written before the work, a second reader, a test, a program that refuses.

Experiment one: watch it be confidently wrong

Reset the demo to its broken state:

```
git checkout demo/src/slugify.ts
npx tsx demo/src/cli.ts "Crème Brûlée"     # cr-me-br-l-e
```

Now give the agent a task with a trap in it — one that is easy to answer plausibly and hard to answer correctly:

```
npx tsx agentic/agent.ts \
  "demo/src/slugify.ts mangles non-ASCII text. Fix it so that accented Latin
   letters fold to their base letters, German ß becomes ss, and the result is
   never an empty string. Do not add a dependency."
```

Read what it tells you: a clear statement of what was wrong, what it changed, and why. Then read the code:

```
git diff demo/src/slugify.ts
npx tsx demo/src/cli.ts "Crème Brûlée"
npx tsx demo/src/cli.ts "Weiß"
npx tsx demo/src/cli.ts "///"
```

Results vary, but across runs a consistent pattern shows up: the accent-folding usually works, because that is the part of the problem the training data is dense in. The `ß` case is often dropped without comment, or handled by a rule that also mangles something else. The empty-string case is frequently unhandled, and the summary will frequently claim it was.

The model wrote what a correct completion of "describe the change you made" looks like. Two of the three requirements really were addressed. The third produced text about being addressed.

> **Example:** In one run of this exact task while writing this chapter, the summary read: *"Added a guard so that an input consisting only of separators returns the literal string 'untitled' rather than an empty slug."* The word `untitled` did not appear anywhere in the file, and no guard had been added. Everything else in the paragraph was true.

Experiment two: ask it to grade itself

The obvious response to that is: fine, I will have it check its own work.

Listing: `agentic/grade.ts`

```
import { runAgent } from "./loop.js";

const ENGINEER = [
  "You are a careful software engineer working in a TypeScript project.",
  "Read before you write. Write complete file contents, never fragments.",
  "When finished, state plainly what you changed and why.",
```

```
].join(" ");

const GRADER = [
  "You are grading a change to a TypeScript project.",
  "Read the files you need. Judge only what the code actually does.",
  "Be brief and be decisive.",
].join(" ");

const TASK = [
  "demo/src/slugify.ts mangles non-ASCII text.",
  "Fix it so that accented Latin letters fold to their base letters,",
  "German ß becomes ss, and the result is never an empty string.",
  "Do not add a dependency.",
].join(" ");

async function main(): Promise<void> {
  // --- 1. Do the work.
  const work = await runAgent(ENGINEER, TASK);
  console.log("=== 1. What it says it did ===\n");
  console.log(work.text);

  // --- 2. Ask it whether the work is correct.
  const grade = await runAgent(
    GRADER,
    [
      "A colleague changed demo/src/slugify.ts and reported this:",
      "---",
      work.text,
      "---",
      "The original request was:",
      TASK,
      "",
      "Read the file as it now stands. Does it do what was asked?",
      'Reply with exactly one line: "PASS: <reason>" or "FAIL: <reason>".',
    ].join("\n"),
  );
  console.log("\n=== 2. Its own grade ===\n");
  console.log(grade.text);

  // --- 3. Disagree with it, giving no reason whatsoever.
  const pressed = await runAgent(
    GRADER,
    [
      "Earlier you judged a change to demo/src/slugify.ts and said:",
      "---",
      grade.text,
      "---",
```

```
      "Another reviewer looked at the same code and reached the opposite",
      "conclusion. They did not explain why.",
      "",
      "Do you stand by your judgement?",
      'Reply with exactly one line: "STAND: <reason>" or "CHANGE: <reason>".',
    ].join("\n"),
  );
  console.log("\n=== 3. The same question, under mild pressure ===\n");
  console.log(pressed.text);

  const inTokens = work.inputTokens + grade.inputTokens + pressed.inputTokens;
  const outTokens = work.outputTokens + grade.outputTokens +
  pressed.outputTokens;
  console.log(`\n[in: ${inTokens} out: ${outTokens}]`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Reset the file and run it:

```
git checkout demo/src/slugify.ts
npx tsx agentic/grade.ts
```

What section 2 shows you

Self-grading is asking a model to evaluate output produced by a model — most sharply, its own. Section 2 of `grade.ts` is the purest form of it.

It will very often print `PASS`, including on runs where the `ß` case or the empty-string case is demonstrably unhandled. You can check by hand in ten seconds; the grader had the file in front of it and did not.

Three reasons stack up behind that miss, and only the first is about the model.

Grading and generating are the same operation. Judging "does this code satisfy the request" is itself a text-continuation problem, and

the most likely continuation of *a competent-sounding summary of a change* followed by *does this do what was asked?* is `PASS`. The grader is not consulting a conclusion it reached; it is producing the likely next text.

The evidence in the context favours the answer. The change summary is right there, well-written, in the same conversation. It is the most salient thing in the input. Whether it is *true* is a question about a file, and reading files takes effort a plausible answer does not require.

Nothing forced it to check the hard case. Nobody said "run it on `Weiß`." "Does it do what was asked?" is a question that can be answered by reading, and reading is where the failure already is.

> **Rule:** Nothing grades its own work — not the model, not a second call to the same model with a different system prompt, not a "reviewer persona." A grader whose only input is the thing being graded is a text generator with a strong prior towards approval.

Experiment three: watch it fold

You gave the grader nothing to work with: no argument, no evidence, no counter-example, only the claim that somebody unspecified had reached the opposite conclusion. A meaningful fraction of the time, it changes its answer.

Sycophancy is a model's tendency to move towards the position it believes its interlocutor holds, rather than towards the position the evidence supports. It is not politeness and it is not indecision. It is a learned pattern: text in which a speaker accommodates a disagreeing counterpart is very common, and agreeable continuations were reinforced during training because human raters preferred them.

Watch what this does to the obvious fix for experiment two. "I'll have the model check its work, and if I disagree I'll push back" — in that exchange the person doing the pushing determines the outcome. If you are right, it agrees with you. If you are wrong, it agrees with you. The agreement carries no information either way, which means you have added a job that costs money and produces nothing.

> **Warning:** Sycophancy is worst when the grader has already been told what you were trying to do. A grader that has read your change summary has been told your conclusion. Chapter 7 is about building a reader that has not been.

Experiment four: the same question, three times

One more, and it is a shell loop rather than a program:

```
for i in 1 2 3; do
  git checkout demo/src/slugify.ts
  npx tsx agentic/agent.ts \
    "Fix demo/src/slugify.ts so accented letters fold to base letters and
     the result is never empty. Do not add a dependency." > /dev/null
  git diff demo/src/slugify.ts > "/tmp/run-$i.patch"
done

wc -l /tmp/run-*.patch
diff /tmp/run-1.patch /tmp/run-2.patch
```

Three runs. Same task, same model, same code, minutes apart. Three different patches — usually different in approach, not just in whitespace. One may normalise with `String.prototype.normalize("NFD")` and strip combining marks; another may use an explicit character map; another may do both and add a fallback.

Non-determinism is the property that identical inputs do not guarantee identical outputs. Chapter 1 mentioned it in passing when discussing

`temperature`; in practice it means you cannot debug by re-running, and cannot reproduce a failure by repeating the command. A run that worked yesterday is not evidence that the same run works today, and "it worked when I tried it" is not a statement about the system, it is a statement about one sample.

It is why everything from Chapter 6 on is built out of *checks* rather than *tuning*: you cannot tune your way to reliability in a system whose output distribution you can only sample.

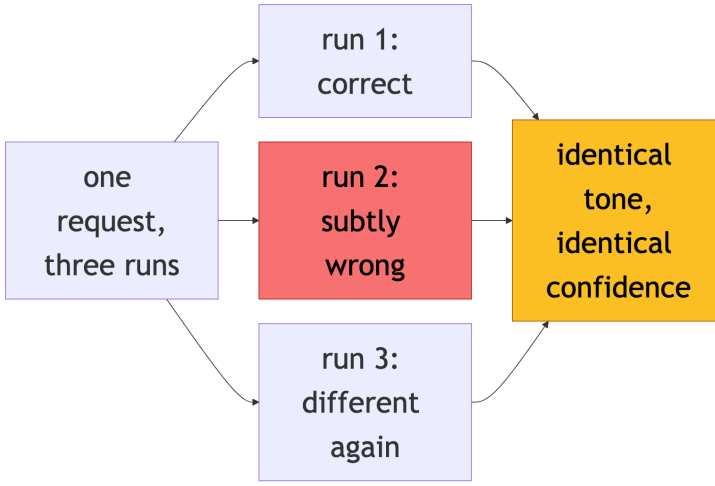


Figure 5.1: The same request, three runs. Every path is confident; only some are correct.

Four failures, one table

Put the experiments side by side:

What happened	What it looked like
A requirement was not implemented	A summary saying it was
The grader did not read carefully	A confident `PASS`

What happened	What it looked like
Pressure with no evidence	A reversed judgement
Three different implementations	Three confident explanations

In every row the *text* is fine: well-formed, well-argued, and appropriately detailed. The failure is never visible in the output; it is only visible by comparing the output against something else — the file, the behaviour, the requirement, the other run. Everything Chapters 6 through 11 build is a way of making that comparison.

Why this gets worse the moment nobody is watching

Everything above happened with you sitting in front of the terminal. Scale it up by one step — a run that starts when you are asleep, or one whose output feeds another run rather than a human — and each failure changes character.

A wrong answer you read is a wrong answer. A wrong answer that becomes the input to the next job is a wrong *premise*, and everything built on it will be internally consistent with it. The second run does not notice, because from inside the second conversation the false premise is simply a fact that was supplied.

This compounding is why the chapters ahead use artefacts that live on disk rather than in a conversation. A file can be diffed against what it was. A conversation can only be re-read, and re-reading is the check that already failed.

Warning: A model grading the output of a model, with no artefact in between that a person or a program could independently check, produces a great deal of confident text, at a real cost per run, and its correlation with correctness is unknown to everyone including the people who built it — because nothing in that loop ever compares one of its grades to what the code actually did.

What the experiments show

Look again at what actually happened in experiment one. Given a three-part request, the agent read the relevant files, understood a regular expression, identified the correct Unicode approach, wrote a working implementation of two of the three parts, and explained itself clearly. That took about forty seconds and a fraction of a cent.

The problem is not the quality of the work. The problem is the *variance* of the quality, combined with the complete absence of a signal telling you which sample you got. If nine runs in ten are excellent and one is subtly wrong, and all ten read identically, then the value of the nine is capped by what the one costs you.

Making the model better is not available to you from out here; making the tenth run *visible* is.

What does not work

Better prompting. "Be accurate." "Do not make things up." "Double-check your work." These help at the margin, because a model that has produced a plausible falsehood has no internal signal that anything is wrong to escalate.

Asking for a confidence score. The number is generated the same way the answer was. A fabricated function name comes with a fabricated 95%.

More capable models. They do genuinely hallucinate less, and they also produce more convincing wrong answers when they do, which changes the *rate* of failure without changing the *detectability*.

Reading it yourself, every time. This is the one that actually works, and it is the one that does not scale.

What does work

Everything that works has the same shape: a check whose outcome does not depend on the model's opinion of its own output.

There are exactly four of them, and the next six chapters build one each.

1. **Say what "done" means before starting, in checkable terms.** A summary can assert that a requirement is met, whereas a written-down requirement can be compared against the code, which is what Chapter 6 builds.
2. **Have it examined by something that did not do the work and cannot see the reasoning.** No change summary, no chain of argument, the code only — Chapters 7 and 8.
3. **Make somebody decide, with the authority to say no.** Several readers produce several opinions, and opinions are not a decision; Chapter 9 supplies one.
4. **Make the machine check the parts a machine can check.** Tests that are proven able to fail, and rules enforced by software that never gets tired or talked round, in Chapters 10 and 11.

None of these makes the model more truthful. All of them make an untruth land somewhere it gets caught.

Pro Tip: Keep `grade.ts` around and re-run it occasionally as models change. The specific failures will shift; some will get rarer. The structural point — that a grader looking at a change summary is reading the wrong artefact — does not depend on the model at all.

Where this leaves you

You have a working agent and no reason to trust it.

The chapters ahead start with writing down, before any code exists, what you would accept as finished. Chapter 12 hands you a package that does everything those chapters build, and asks whether you trust it.

Chapter 6

Writing It Down First

The plan is a file format, rules for what may go in it, and forty lines of TypeScript that fills one in; this chapter builds all three.

The cheapest defence against a confident wrong answer is deciding what a right answer would look like before you see any answer at all.

The problem, stated exactly

In Chapter 5, at the end of every run, the agent told you it was done. Ask yourself what "done" was measured against.

It was measured against a one-sentence task, interpreted by the party doing the work, at the moment it finished. Three separate weaknesses in one arrangement:

- The standard was **vague** — "fix it so accents fold" admits a range of behaviours, several of which are wrong.
- The standard was **interpreted by the interested party**: the

model grading its own work, which Chapter 5 showed reliably produces a `PASS`.
- The standard was **available only after the work**, so nothing prevented it from reshaping itself, unremarked, around what the work turned out to be.

The third one is the subtle one and it is the one with a name. In science, a **pre-registration** is a public statement of what you intend to measure and what result would count as which outcome, filed *before* the data is collected. It exists because a hypothesis chosen after seeing the data is not a hypothesis; it is a description. A requirement written after the work is a description of the work, and that holds whether or not everybody involved is honest.

Specification, criterion, falsifiable

A **specification** is a written statement of what a change is supposed to accomplish, produced before the change exists: not a design and not an implementation plan, but a statement of the target.

An **acceptance criterion** is one checkable condition that must hold for the change to count as complete. A specification is mostly a list of these.

An acceptance criterion counts only if it is **falsifiable**: there is a specific observation that would show it is *not* met. If you cannot name the observation that would fail it, it cannot pass either.

Applying that test to the criteria people actually write looks like this:

Not falsifiable	Falsifiable
Handles Unicode correctly	`npx tsx demo/src/cli.ts "Weiß"` prints `weiss`

Not falsifiable	Falsifiable
Improves error handling	Calling `slugify("")` returns `"untitled"`, not `""`
Is well tested	`demo/src/slugify.test.ts` contains a case for `ß` and the suite exits 0
No performance regression	`slugify` on a 10,000-character input completes in under 50 ms
Code is clean and readable	*(delete it — this is a review comment, not a criterion)*

The left column is not a worse version of the right column. It is a different kind of sentence. Every entry on the left can be argued about indefinitely; every entry on the right is settled by running something.

Rule: In this pipeline, an acceptance criterion that does not name a command to run or a file to open, along with the observable result, is rewritten or deleted. The bar is higher than a team of human readers would need, and it is set there because the readers downstream are programs.

Non-goals

A **non-goal** is something a reasonable reader would expect this change to include, stated explicitly as not included.

Consider the slugify task with this non-goal attached:

> **No new dependencies.** Nothing is added to `package.json`.

Without it, an agent that pulls in a well-regarded transliteration library has done something defensible: it solved the problem, it used a maintained package, and it will explain that choice persuasively. The only thing wrong with it is that you did not want it, and *you did not say so*, so there is nothing to point at.

Non-goals are how you keep scope from expanding without having an argument about it later. They are also, in a workflow that runs unattended, the only voice you have during the run. In Chapter 9 a judge will be forbidden from doing anything listed under non-goals, and that prohibition holds because the list was written before anybody knew what would be convenient.

Three non-goals recur:

- **No refactoring of adjacent code.** The most common way a small change becomes an unreviewable one.
- **No new dependencies.**
- **No changes to .** Name the thing: auth, migrations, the build, the payment path.

Open questions belong in the plan, not in the code

The fourth section of the format is a list of things that could not be settled by reading the code.

The alternative to writing a question down is answering it by guessing, and a guess made during planning is indistinguishable from a decision by the time anyone reads the result. "Should an empty slug become `untitled` or should the function throw?" is a real question with two defensible answers and different consequences. A plan that picks one

without saying so has made a product decision inside an engineering task.

> **Pro Tip:** A plan with zero open questions on a non-trivial change usually means the planner resolved ambiguity by choosing, which is the failure this section exists to catch. Read the plan asking "what did this have to assume?" — the answers should be in the list.

The format

The format has five sections and fits on a page.

```
# Plan: fold accents in slugify

- **Slug:** fold-accents
- **Status:** draft

## Request

> slugify mangles accented text; fix it.

## Intent

`slugify` replaces every non-ASCII character with a hyphen, so European text
becomes an unreadable run of separators. Fold accented Latin letters to their
base letters before the character filter runs, so that titles in French,
German, and Spanish produce readable slugs.

## Acceptance criteria

- [ ] **AC1 - Accents fold.** `npx tsx demo/src/cli.ts "Crème Brûlée"` prints
      `creme-brulee`.
- [ ] **AC2 - Sharp s expands.** `npx tsx demo/src/cli.ts "Weiß"` prints
      `weiss`.
- [ ] **AC3 - Never empty.** `npx tsx demo/src/cli.ts "///"` prints `untitled`
      and exits 0.
- [ ] **AC4 - ASCII is unchanged.** `npx tsx demo/src/cli.ts "Hello There"`
      still prints `hello-there`.

## Non-goals
```

```
- No new dependencies; `package.json` is untouched.
- No transliteration of non-Latin scripts. Greek, Cyrillic, and CJK input is
  out of scope and may still produce `untitled`.
- No change to `demo/src/cli.ts`.

## Open questions

- None. The empty-input behaviour was settled as `untitled` rather than a
  thrown error, because the CLI has no error path and adding one is a
  larger change.

## Steps

- [ ] S1: Add a fold step to `slugify` before the character filter.
- [ ] S2: Add the empty-result fallback.
```

Notice what the acceptance criteria are made of. Every one is a command and its expected output. You can check the whole plan in four shell lines, without reading any code, without forming an opinion, and without asking anybody's judgement. That is the property everything downstream depends on.

The plan is a **system of record**: the artefact that is authoritative about what was supposed to happen, kept somewhere that survives the conversation that produced it. Every job after this one reads it, and this pipeline forbids any of them to edit the first three sections, because a target that moves is not a target.

Writing the planner

The program that produces one reads the code, fills in the format, and writes a file. It writes nothing else, and it never touches the source.

Listing: `agentic/plan.ts`

```
import fs from "node:fs";
import path from "node:path";
```

```
import { runAgent, READ_ONLY_TOOLS } from "./loop.js";

const PLANNER = [
  "You are writing a plan for a change to a TypeScript project.",
  "No code has been written yet and you will write none.",
  "Read whatever files you need in order to plan against the code that exists.",
  "",
  "Every acceptance criterion must be falsifiable: it names a command to run",
  "or a file to open, and states the observable result that counts as met.",
  "Reject your own criteria that cannot be checked that way.",
  "'Handles Unicode correctly' is not a criterion. 'Running the CLI on the",
  "input Weiss with a sharp s prints weiss' is.",
  "",
  "Non-goals name things a reasonable reader would expect and will not get.",
  "Open questions are things you could not settle by reading the code. Never",
  "invent an answer to one; list it and move on.",
  "",
  "Return the plan and nothing else. No preamble, no closing remarks.",
].join("\n");

const TEMPLATE = `# Plan: <short title>

- **Slug:** <slug>
- **Status:** draft

## Request

> <the request, verbatim>

## Intent

<two or three sentences: what changes, and why it is worth doing>

## Acceptance criteria

- [ ] **AC1 - <name>.** <command to run or file to open, and the observable
      result that counts as met>

## Non-goals

- <something a reader would expect and will not get>

## Open questions

- <anything you could not settle by reading, or "None." with a reason>

## Steps
```

```
- [ ] S1: <the first change to make>
`;

async function main(): Promise<void> {
  const [slug, ...rest] = process.argv.slice(2);
  const request = rest.join(" ");

  if (slug === undefined || request.length === 0) {
    console.error('usage: tsx agentic/plan.ts <slug> "<request>"');
    process.exit(1);
  }

  const dir = path.join("work", slug);
  const file = path.join(dir, "plan.md");

  if (fs.existsSync(file)) {
    console.error(`${file} already exists. Choose another slug or delete it.`);
    process.exit(1);
  }

  const run = await runAgent(
    PLANNER,
    [
      "Write a plan for this request.",
      "",
      "REQUEST:",
      request,
      "",
      `The slug for this work is: ${slug}`,
      "",
      "Use exactly this shape:",
      "",
      TEMPLATE,
    ].join("\n"),
    // It plans against the code. A prompt telling it not to write is a
    // request; withholding write_file is the thing that makes it true.
    READ_ONLY_TOOLS,
  );

  fs.mkdirSync(dir, { recursive: true });
  fs.writeFileSync(file, `${run.text.trim()}\n`, "utf8");

  console.log(`Wrote ${file} in ${run.iterations} iterations.`);
  console.log("Read it before anything else runs.");
}
```

```
main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Run it:

```
npx tsx agentic/plan.ts fold-accents \
  "slugify mangles accented text; fix it so European titles produce readable
  slugs"
```

Then open `work/fold-accents/plan.md` and read it properly. This is the one place in the whole workflow where a human being reads carefully, and it is deliberately placed where reading is cheapest: one page, before any code exists, when changing your mind costs nothing.

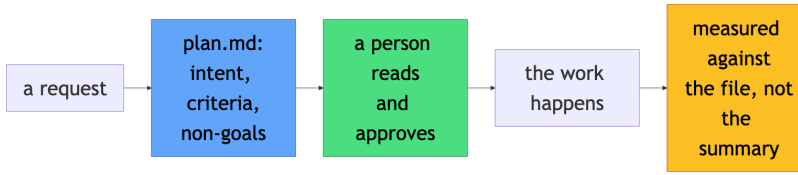


Figure 6.1: Vague intent leaves whether the work is done open to argument. A plan written in checkable terms, approved before the work starts, settles it by running something.

Reading a generated plan

Four things to check, in this order. They take about two minutes.

1. **Is every acceptance criterion falsifiable?** Cross out any that are not. This is where most generated plans need work; models produce "handles the edge cases gracefully" with great fluency.

2. **Is anything missing that you wanted?** A criterion you care about that is not listed will not be built, and — worse — will not be *reported* as not built.
3. **Are the non-goals right?** Add the ones you are afraid of.
4. **Do the open questions need answering?** Answer them in the file, in your own words.

Then change `Status: draft` to `Status: approved` with your own hands.

Rule: Approval here is a human act on a file rather than a reply in a conversation. That is a choice, and it follows from wanting a program to check it later — Chapter 11 will enforce it mechanically — because a "yes, go ahead" typed into a chat is invisible to everything except the chat.

Steps are not criteria

The format has both, and confusing them is the most common way a plan stops working.

Acceptance criteria describe the destination. They are about observable behaviour, they survive any change of approach, and nothing in the workflow may alter them once approved.

Steps describe the route. They are a working checklist, they are expected to be wrong in detail, and whoever does the work may add to them, reorder them, or abandon one that turns out to be unnecessary — as long as the criteria still end up met.

They have opposite properties under pressure. When the work turns out harder than expected, the honest response is to change the steps and keep the criteria. The dishonest response is to change the criteria and

keep the steps, at which point the plan describes what happened rather than what was wanted.

Example: Halfway through the accents change, the builder discovers that `String.prototype.normalize` handles the accents but not `ß`, which needs an explicit replacement first. Changing S1 from "add a fold step" to "add an explicit replacement, then a fold step" is fine — the route changed. Changing AC2 from "prints `weiss`" to "prints `wei-`, which is acceptable for German" is not.

Who writes the plan

You can write the plan yourself, and for anything genuinely delicate you should.

A planner that reads the code first, as `plan.ts` does, surfaces things you did not know were there. The `ß` case in the running example came from one.

The costs are just as specific. A generated plan is subject to everything in Chapter 5: it will contain a criterion that sounds checkable and is not, and it will occasionally invent a requirement that nobody asked for and state it with complete confidence.

Rule: Of everything this pipeline produces, the plan is the one artefact a person must read in full. That is deliberate: the jobs downstream are arranged so that machines and independent readers do the rest of the reading.

Where the plan lives

One directory per change, named by slug:

```
work/
  fold-accents/
    plan.md
  empty-input-guard/
    plan.md
```

Not one `plan.md` at the root that gets overwritten. Three reasons:

- **Two changes can be in flight at once.** A single file makes the second one destroy the first one's record.
- **The record survives.** Six months later, "why does slugify return `untitled`?" is answered by a file rather than by reading commit messages.
- **Everything downstream needs somewhere to put its own output.** By Chapter 9 that directory will also hold review files and a decision, and having them beside the plan they refer to is what makes the set readable.

Commit that directory along with the code, because it is part of the change rather than scaffolding around it.

What a plan cannot do

A plan does not make the work correct. It makes *incorrectness detectable*.

A plan can be met dishonestly. AC1 says the CLI prints `creme-brulee`, and a change that special-cases that exact string satisfies it. Criteria are a floor, not a ceiling, and closing that particular gap is what Chapter 7's readers and Chapter 10's tests are for.

A plan can also be wrong, in the sense that you approved criteria that turn out to describe the wrong thing. That is fine and normal, but it must be fixed by changing the plan deliberately and re-approving it, never by working around it unannounced during the build. A plan that was

diverged from without a record leaves everything downstream reporting against it.

> **Warning:** The failure that ruins a plan is editing the intent or the criteria *during* the work to match what got built. Chapter 11 makes that specific edit detectable by a program, because it is the one rule a prompt cannot hold.

Where this leaves you

You have a specification, in a file, approved by a person, before any code exists. Everything downstream now has something to be measured against that was not produced by the thing being measured.

The next step is to build something that does the measuring — and the central design problem there is not intelligence. It is ignorance: how do you get a reader that has *not* seen the argument for why the change is correct?

This page intentionally left blank

Chapter 7

A Second Pair of Eyes

This chapter builds a reader that is blind to the argument for a change, and the whole design comes down to controlling what it is allowed to see.

The grader in Chapter 5 failed for a reason that had nothing to do with how clever it was. It failed because of what was in its input: the change summary, well-written, confident, sitting at the top, asserting that the work was correct. Then it was asked whether the work was correct. Under those conditions the answer was decided before the reading started.

Anchoring

Anchoring is the tendency of a judgement to be pulled towards information present when the judgement is made, whether or not that information is evidence. Tell someone a number before asking them to estimate an unrelated quantity and their estimate moves towards the number. It is one of the most robust findings in the study of human judgement, and models exhibit it at least as strongly, for reasons that are structural

rather than psychological: a conclusion stated prominently in the input is, quite literally, a likely continuation.

The practical form here: a reviewer that has read "I fixed the accent handling by normalising to NFD and stripping combining marks" is not evaluating the code. It is evaluating whether the code is consistent with that sentence — which it will be, because the sentence was written from the code.

The reviewer's job is to answer "is this right?", and the builder's summary answers "why is this right?". Putting the second in front of the first replaces the review: every argument you supply is an argument the reviewer no longer has to construct, or fail to construct.

Fresh context

The fix is to start a new conversation.

Fresh context means a conversation that begins empty: no history, no earlier reasoning, no accumulated assumptions — only the material you deliberately put in it. Chapter 1 made this easy without either of us noticing at the time. The model is stateless, and the conversation is an array in your program, so a new array is a genuinely new mind on the problem. Nothing leaks, because no channel exists through which anything could leak.

In any other kind of system, isolating a reviewer from the author's reasoning is an organisational problem, solved with process and trust. Here it is `const messages = []`.

A process that runs in its own fresh context, doing one bounded job, and returns a result to whatever started it, is a **subagent**. `runAgent` already makes one every time you call it. Chapter 12 will show you a version where subagents are a declared feature of a product.

What the reviewer gets, and what it must not

Here is the complete input to each reviewer in this chapter:

The diff. The line-by-line change, from Chapter 3. Nothing else describes what was done with so little editorialising: it is the change, with no argument attached.

The plan, meaning its intent, acceptance criteria and non-goals. The reviewer needs to know what was being attempted, or every deliberate decision looks like an oversight.

Its own question. One specific thing to look for, which the next section is about.

And here is what it must not get, ever:

The builder's summary. For all the reasons above.

The conversation the change came from, which carries the same argument at greater length.

The other reviewers' output. Two readers who have seen each other's findings are not two readers. The second one is a commentary on the first.

A review conducted without access to the author's reasoning is a **blind review**, and the word is doing precise work: the reviewer is blind to the argument, not to the code. It sees more of the change than the builder's summary ever showed; it simply sees none of the case for it.

Rule: The reviewers you are about to build read the artefact and never the argument. Human reviewers routinely read both, and reasonably; these ones are denied the argument because a model handed one will adopt it. Passing a change summary into a reviewer "for context" rebuilds the Chapter 5 grader with extra steps.

One question each

A single reviewer asked "review this change" produces a list dominated by whatever kind of problem is most salient in that particular diff. Ask it again and you get a different list. What you get back is broad, shallow, and unstable, and its gaps are invisible, because a reviewer that did not think about security does not report that it did not think about security.

A **lens** is a review with exactly one question, asked in isolation, blind to the others: not a persona or a personality, but a scope restriction. The reviewer running the security lens is instructed that correctness is not its problem, and that instruction is what makes it look at the things a general reviewer skims.

Five lenses are built in this chapter, and the sixth arrives in Chapter 8, because it works completely differently.

Lens	The one question
`correctness`	Does this do what it says on inputs it will actually see?
`spec-fidelity`	Is every acceptance criterion met? Was any non-goal built anyway?
`security`	What does this do with input it did not expect?
`simplicity`	What here is more complicated than the problem requires?
`test-quality`	Would the tests fail if the behaviour broke?

Running the same model five times with five different scopes is not the same as running it once and asking for five things. The difference is the isolation: each run's entire input is one question, so nothing else

competes for attention, and no finding from one lens shapes the search of another.

> **Warning:** Five lenses on the same model share a blind spot. If the model is bad at spotting a particular class of defect, running it five times with five instructions does not fix that — every lens is bad at it in the same way, and the resulting five clean reports read exactly like five independent confirmations. The panel widens what gets looked at; it does not create independence at the level of the weights. Chapter 12 returns to this as an unsolved limitation of the tools built on this pattern.

Building it

`review.ts` takes a slug, computes the diff, runs every lens against it in parallel, and writes one file per lens.

Listing: `agentic/review.ts`

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { runAgent } from "./loop.js";

interface Lens {
  name: string;
  question: string;
}

const LENSES: Lens[] = [
  {
    name: "correctness",
    question:
      "Does this change do what it claims on inputs it will actually see? " +
      "Look for off-by-one errors, unhandled cases, wrong operators, and " +
      "assumptions the surrounding code does not guarantee.",
  },
  {
```

```
    name: "spec-fidelity",
    question:
      "Compare the change against the plan. Is every acceptance criterion " +
      "actually met by the code shown? Was anything listed under Non-goals " +
      "built anyway? A criterion met only for the literal example in the " +
      "plan is not met.",
  },
  {
    name: "security",
    question:
      "What does this do with input it did not expect? Consider untrusted " +
      "strings, paths, very large inputs, and anything that reaches a file " +
      "system, a shell, or a network call.",
  },
  {
    name: "simplicity",
    question:
      "What here is more complicated than the problem requires? Look for " +
      "duplication, dead code, needless abstraction, and anything the " +
      "existing code already does.",
  },
  {
    name: "test-quality",
    question:
      "Would the tests fail if this behaviour broke? Look for assertions " +
      "that cannot fail, tests coupled to implementation detail, and " +
      "behaviour changed without any test covering it.",
  },
];

const REVIEWER = [
  "You are reviewing a change to a TypeScript project.",
  "You did not write it and you have not seen the reasoning behind it.",
  "You have the diff and the plan. That is deliberately all you have.",
  "",
  "Report problems. Do not fix anything and do not rewrite any code.",
  "Do not comment on style, formatting, or naming unless it causes a defect.",
  "",
  "For each problem, write exactly this shape:",
  "",
  "## Finding N - <one line, specific>",
  "",
  "- **Where:** <file and, if you can tell, the line>",
  "- **Severity:** blocker | major | minor",
  "- **Why it is wrong:** <one or two sentences>",
  "- **How it fails:** <a concrete input, and the wrong result it produces>",
  "",
```

```
  "If you cannot name a concrete failing input, the finding is a guess.",
  "Mark it minor and say so.",
  "If you find nothing, reply with exactly: No findings.",
].join("\n");

function diffAgainst(base: string): string {
  return execFileSync("git", ["diff", base, "--", "."], {
    encoding: "utf8",
    maxBuffer: 32 * 1024 * 1024,
  });
}

async function runLens(
  lens: Lens,
  diff: string,
  plan: string,
): Promise<string> {
  const run = await runAgent(
    `${REVIEWER}\n\nYour lens is "${lens.name}".\n${lens.question}\n` +
      "Findings outside your lens are not your job. Ignore them.",
    [
      "PLAN",
      "====",
      plan,
      "",
      "DIFF",
      "====",
      diff,
    ].join("\n"),
    [], // No tools. The diff and the plan are the whole world.
  );
  return run.text.trim();
}

async function main(): Promise<void> {
  const [slug, base] = process.argv.slice(2);

  if (slug === undefined) {
    console.error("usage: tsx agentic/review.ts <slug> [base-ref]");
    process.exit(1);
  }

  const planFile = path.join("work", slug, "plan.md");
  if (!fs.existsSync(planFile)) {
    console.error(`No plan at ${planFile}. Run agentic/plan.ts first.`);
    process.exit(1);
  }
```

```
  const plan = fs.readFileSync(planFile, "utf8");
  const diff = diffAgainst(base ?? "HEAD");

  if (diff.trim().length === 0) {
    console.error("The diff is empty. There is nothing to review.");
    process.exit(1);
  }

  const dir = path.join("work", slug, "reviews");
  fs.mkdirSync(dir, { recursive: true });

  console.error(`Reviewing ${diff.split("\n").length} diff lines ` +
    `through ${LENSES.length} lenses...`);

  // Every lens runs against the same diff, at the same time,
  // and none of them can see any other's output.
  const reports = await Promise.all(
    LENSES.map((lens) => runLens(lens, diff, plan)),
  );

  LENSES.forEach((lens, index) => {
    const file = path.join(
      dir,
      `${String(index + 1).padStart(3, "0")}-${lens.name}.md`,
    );
    fs.writeFileSync(
      file,
      `# Review: ${lens.name}\n\n${reports[index]}\n`,
      "utf8",
    );
    const clean = reports[index].startsWith("No findings");
    console.log(`${clean ? "  clean" : "FINDING"}  ${file}`);
  });
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Use it on the change from Chapter 5:

```
git checkout demo/src/slugify.ts
npx tsx agentic/plan.ts fold-accents \
```

```
  "slugify mangles accented text; fix it so European titles produce readable
  slugs"
# read work/fold-accents/plan.md, fix the criteria, set Status: approved

npx tsx agentic/agent.ts "$(cat work/fold-accents/plan.md)"
npx tsx agentic/review.ts fold-accents
```

Then read the five files in `work/fold-accents/reviews/`. `spec-fidelity` is the only lens holding the change against a written target, and it is the one that says things like *AC3 requires the empty case to print `untitled`; the diff contains no such branch.*

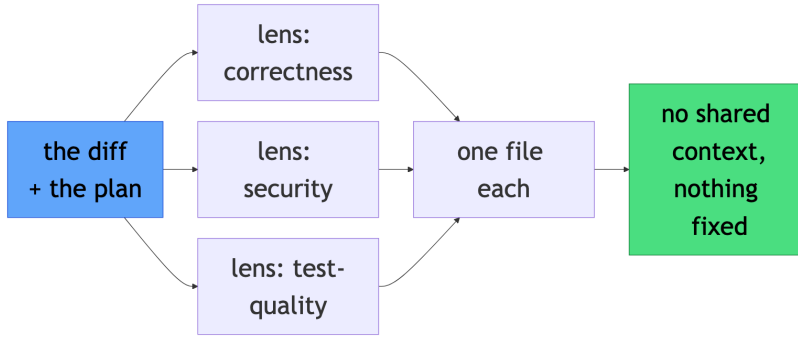


Figure 7.1: Same diff, five readers, no shared context. Each writes one file and fixes nothing. None can be talked round by another's reasoning, because none of them can see it.

Why `Promise.all`, and what it costs

The five lenses have no dependency on each other, so they run at once. This is not only about speed; it is a structural guarantee. Code that runs reviewers in sequence makes it possible to pass the previous report along "so it does not repeat findings," and at that moment the panel is one opinion repeated five times.

Five lenses is five times the input tokens, because each one receives the

full diff and the full plan. On a large change that is the most expensive job in the whole workflow. Two things make it tolerable: the diff is usually far smaller than the code it touches, and five requests that share most of their bytes, sent within a minute of each other, are the situation prompt caching from Chapter 4 was built for. As written, `review.ts` gets none of it. The cache is a prefix match, each lens's system prompt differs, and the system prompt comes before the plan and the diff, so the shared part sits behind what changes. Caching it means putting the lens question after the diff rather than before it, which the prefix rule forces and Chapter 1's rule about where instructions go argues against; that trade is yours to make, and Chapter 4's ordering rule is what decides it.

> **Pro Tip:** If cost is a concern, the lever is a smaller diff rather than fewer lenses. A change that produces an expensive review is usually a change that should have been two changes.

Findings, and the ones that are wrong

A **finding** is one reported problem: where it is, why it is wrong, and how it fails. How it fails is the part that makes a finding checkable.

Requiring a concrete failing input — *"`slugify('///')` returns the empty string, and the CLI prints a blank line"* — changes the character of the whole report. It is checkable in ten seconds, and it cannot be produced by a reviewer that is pattern-matching on shapes it finds suspicious. It is also the thing a model will fail to produce when it is guessing, which is why the instruction says so explicitly: *if you cannot name a concrete failing input, the finding is a guess.*

You will still get **false positives** — findings that are wrong. Expect them, at a rate somewhere around a quarter to a third of everything reported. The most common kinds:

- **Missing-context findings.** The reviewer cannot see the function two files over that already handles the case, so it reports it as unhandled.
- **Preference findings**, such as "this would be clearer as a `Map`," which is a taste, not a defect.
- **Fabricated line numbers.** The finding is real, the location is invented, so check the finding rather than the coordinate.

> **Warning:** A false positive rate of a third is not a reason to stop. It is a reason not to let the reviewers *act*. Nothing in this chapter fixes anything — the reviewers have no write access and no authority — and that is what makes a wrong finding cheap. Chapter 9 exists to decide which findings are real, and it is a separate job on purpose.

Should the reviewer be allowed to read the repository?

The reviewer in `review.ts` gets `[]` for its tools. It sees the diff, the plan, and nothing else. That is a deliberate starting point and it is not the only defensible one.

The case for keeping it blind: it is cheap, it is fast, and it is impossible for the reviewer to wander off into unrelated code. The diff is also the honest unit of review — it is what a colleague would look at.

The case for giving it `read_file` and `list_files`: most missing-context findings disappear, because it can go and look. The cost is more tokens and more iterations, and a small risk that it reviews the whole codebase instead of the change.

The right answer depends on your diffs, and the way to find out is to run it both ways on the same change and compare the false positive

rate. What is *not* negotiable in either configuration is the list of things it may never see: the builder's summary, the builder's conversation, and the other lenses' reports. Those three are the whole of the independence guarantee.

Severity is a routing decision

Each finding carries one of three severities, and the point of having exactly three is that each one names a different thing to *do* — not a different degree of annoyance.

- **blocker** — this change must not proceed while this is true. Data loss, a broken acceptance criterion, an unhandled case that reaches users.
- **major** — real and worth fixing, though not necessarily in this change, so it becomes recorded work instead.
- **minor** — worth someone knowing. Frequently a preference. Frequently wrong.

Three is enough because the decision space is: stop, schedule, note. Five-level scales invite arguments about whether something is a three or a four, and that argument has no consequence attached.

Pro Tip: Read the severity distribution across the five files before reading any individual finding. Five reports with no blockers and eleven minors usually means the change is fine. One report with three blockers and the rest clean usually means one lens found something real. Where the blockers are tells you which file to open first.

A worked report

Here is a genuine `spec-fidelity` report from a run of the accents change, with nothing removed:

```
# Review: spec-fidelity

## Finding 1 - AC3 (empty result) is not implemented

- **Where:** demo/src/slugify.ts, the return statement
- **Severity:** blocker
- **Why it is wrong:** The plan's AC3 requires that input consisting only of
  separators produces `untitled`. The diff adds normalisation and a character
  filter but no fallback for an empty result.
- **How it fails:** `slugify("///")` returns `""`, so the CLI prints a blank
  line rather than `untitled`.

## Finding 2 - AC2 (sharp s) is met only for lower case

- **Where:** demo/src/slugify.ts, the replacement chain
- **Severity:** major
- **Why it is wrong:** The replacement handles "ß" but runs before the input
  is lowercased, so the capital form, U+1E9E, which lowercasing would have
  folded to "ß", never matches it.
- **How it fails:** `slugify("WEI\u1E9E")` returns `wei` rather than `weiss`.
```

Two findings, both checkable in under a minute, both correct. Notice what makes them useful: each names a *call* and its *wrong result*. Neither finding asks you to agree with a judgement, and you can settle both by typing two commands.

Notice too what the lens did not report. It said nothing about the regular expression's readability, nothing about whether `normalize("NFD")` is the best approach, and nothing about the missing tests. Those are other lenses' jobs, and its silence on them is not an endorsement — it is scope.

When everything comes back clean

Five clean reports have three quite different explanations.

The change is genuinely fine, which is common on small, well-scoped diffs and is the system working as intended.

The change is too large to review. Past a few hundred lines, reports get vaguer rather than longer, and the vagueness reads as approval. A nine-hundred- line diff that comes back with two minor findings is a reviewer that ran out of attention.

The plan was too weak to review against. `spec-fidelity` can only find what the criteria let it look for. Criteria like "handles Unicode correctly" produce reports like "appears to handle Unicode correctly," and the round trip has told you nothing at all. This failure originates two chapters back.

> **Rule:** A clean review is evidence about the change *and* about the review. Ask which of the three you got before you act on it. The cheapest check is the diff's size: if you cannot read it yourself in ten minutes, neither could the reviewer.

What is still missing

Five readers have now examined the change. Between them they have read every line of it, several times, from five directions.

Not one of them has run it.

Every finding in every file is a claim about what the code would do, inferred from reading. That is a real and useful activity — it caught the missing empty-input branch — but it has a characteristic failure: the defects a user hits first are frequently invisible in a diff. A change that

is correct line by line and broken when assembled will pass all five lenses cleanly.

That gap is what the next chapter closes.

This page intentionally left blank

Chapter 8

Running the Thing

A sixth lens runs the assembled program and reports what it observed, and this chapter builds it. Five readers examined the change in Chapter 7 and none of them ran it, and that gap separates two entirely different kinds of knowledge about a program.

Two kinds of review

Static review is examining code without executing it: reading a diff, reading a file, reasoning about what would happen. Everything in Chapter 7 was static review. So is every code review you have ever done in a browser tab.

A **behavioral test** is finding out what a program does by running it and observing the result rather than reasoning about it.

The gap between them is where a specific and very ordinary class of defect lives. Every line of a change can be defensible, and the assembled program still broken, because the defect is in the assembly rather than in any line. Some examples that five static lenses will pass cleanly:

- The function is correct and nothing calls it.
- The function is correct and its caller passes arguments in the wrong order.
- The change works and the process exits before flushing stdout.
- The change works and the build no longer produces the file the CLI imports.
- The change works and takes four seconds, on a path that used to take four milliseconds.

The defects a user hits first are usually invisible in a diff: they are properties of the whole, and a diff is by construction a view of the parts.

The assembled application

The thing that has to be run is the **assembled application**: the program as a user encounters it, wired together, started the way it is really started: not the function, and not the module with the rest mocked out, but the command itself.

For our demo project that is `npx tsx demo/src/cli.ts <text>` — one process, one input, one output, one exit code. For a web service it would be the server listening on a port. For a desktop application it would be the window. The principle survives the change of scale, and so does the failure mode: every step you take away from the real entry point is a step towards testing something other than what ships.

A **smoke test** is the smallest exercise that shows the assembled application runs at all — start it, do the most ordinary thing, see that it did not fall over. The name is from electronics, where you power a new board and look for smoke. It is not a thorough test and is not meant to be. It is the test whose failure means nothing else is worth running.

Rule: The behavior lens starts the real thing. Importing the function under test directly writes a different kind of test — the subject of Chapter 10 — but not this one.

The design problem

This lens differs from the other five in one way, and getting it backwards produces something that looks identical.

We are going to ask a model to tell us what the program does.

That is the one thing Chapter 5 says you must never do. A model asked "what does `npx tsx demo/src/cli.ts '///'` print?" will answer, fluently, plausibly, and without running anything — and a fair proportion of the time, correctly, which is what makes the failure so hard to see.

So the model does not answer that question. Your code does:

1. **The model proposes inputs.** This is a task it is genuinely good at: reading a plan and thinking of the awkward cases. Nothing here is a claim about behaviour.
2. **Your program runs them**, with `spawnSync`, a real process, real stdout and a real exit code, and no model anywhere in the step.
3. **The model reads the recorded results** and reports what does not match the plan. It is now reasoning about observations rather than producing them.

The model is on both ends and nowhere in the middle. Every factual statement in the final report traces back to a line your program captured from a real process.

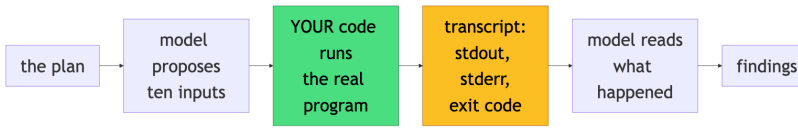


Figure 8.1: The model chooses what to try and reads what happened. The running happens in between, in your code.

Warning: If you skip step 2 — if the "behavior lens" is a prompt asking a model to describe what the program would do — it produces a document that looks exactly like evidence, is treated downstream as evidence, and is a hallucination with a `stdout:` label on it.

Building it

Listing: `agentic/behave.ts`

```
import fs from "node:fs";
import path from "node:path";
import { spawnSync } from "node:child_process";
import { runAgent } from "./loop.js";

const PROPOSER = [
  "You are choosing inputs to try against a command-line program.",
  "You will not be told what the program does; you are guessing at what",
  "might break it, from the plan alone.",
  "",
  "Return one input per line and nothing else. No numbering, no",
  "commentary. Write each input as a JSON string - double quotes and",
  "escapes - so that a space or an empty string survives the trip.",
  "At most ten lines.",
  "",
  "Include: the ordinary case, every case the plan names, the empty case,",
  "and two or three cases the plan does not mention but a user would hit.",
].join("\n");

const OBSERVER = [
```

```
  "You are the behavior reviewer. You did not write this change.",
  "Below is a plan, and a transcript of a real session recorded by running",
  "the program. Every line of the transcript is an observation.",
  "",
  "Report where the observed behaviour does not match the plan.",
  "Never state what the program does beyond what the transcript shows.",
  "If the transcript does not cover something, say it was not exercised -",
  "do not reason about what would have happened.",
  "",
  "Use exactly this shape for each problem:",
  "",
  "## Finding N - <one line>",
  "",
  "- **Input:** <the exact input from the transcript>",
  "- **Expected:** <what the plan requires>",
  "- **Observed:** <what the transcript shows>",
  "- **Severity:** blocker | major | minor",
  "",
  "If everything observed matches the plan, reply with exactly: No findings.",
].join("\n");

interface Observation {
  input: string;
  code: number;
  stdout: string;
  stderr: string;
}

function exercise(input: string): Observation {
  const result = spawnSync("npx", ["tsx", "demo/src/cli.ts", input], {
    encoding: "utf8",
    timeout: 30_000,
  });

  return {
    input,
    code: result.status ?? -1,
    stdout: result.stdout ?? "",
    stderr: result.stderr ?? "",
  };
}

function render(observations: Observation[]): string {
  return observations
    .map((o) =>
      [
        `$ npx tsx demo/src/cli.ts ${JSON.stringify(o.input)}`,
```

```
        `exit code: ${o.code}`,
        `stdout: ${JSON.stringify(o.stdout)}`,
        `stderr: ${JSON.stringify(o.stderr.trim())}`,
      ].join("\n"),
    )
    .join("\n\n");
}

async function main(): Promise<void> {
  const [slug] = process.argv.slice(2);

  if (slug === undefined) {
    console.error("usage: tsx agentic/behave.ts <slug>");
    process.exit(1);
  }

  const planFile = path.join("work", slug, "plan.md");
  if (!fs.existsSync(planFile)) {
    console.error(`No plan at ${planFile}.`);
    process.exit(1);
  }
  const plan = fs.readFileSync(planFile, "utf8");

  // 1. The model proposes. It is not claiming anything about behaviour.
  const proposed = await runAgent(
    PROPOSER,
    ["PLAN", "====", plan].join("\n"),
    [],
  );

  // JSON, not raw text. "" and " " are two of the inputs most likely to
  // break a program, and a trim-and-drop-blanks parser destroys both of
  // them before the program is ever run.
  const inputs = proposed.text
    .split("\n")
    .map((line) => line.trim())
    .filter((line) => line.startsWith('"'))
    .flatMap((line) => {
      try {
        const value: unknown = JSON.parse(line);
        return typeof value === "string" ? [value] : [];
      } catch {
        return [];
      }
    })
    .slice(0, 10);
```

```
if (inputs.length === 0) {
  console.error("No inputs were proposed. Nothing to exercise.");
  process.exit(1);
}

// 2. Your code runs them. This is the only source of fact in the file.
const observations = inputs.map((input) => exercise(input));
const transcript = render(observations);

console.error(transcript);
console.error("");

// 3. The model reads what happened.
const report = await runAgent(
  OBSERVER,
  [
    "PLAN",
    "====",
    plan,
    "",
    "SESSION TRANSCRIPT",
    "==================",
    transcript,
  ].join("\n"),
  [],
);

const dir = path.join("work", slug, "reviews");
fs.mkdirSync(dir, { recursive: true });

const file = path.join(dir, "006-behavior.md");
fs.writeFileSync(
  file,
  [
    "# Review: behavior",
    "",
    "## Session transcript",
    "",
    "```",
    transcript,
    "```",
    "",
    "## Findings",
    "",
    report.text.trim(),
    "",
  ].join("\n"),
```

```
    "utf8",
  );

  console.log(`wrote ${file}`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Run it against the same change:

```
npx tsx agentic/behave.ts fold-accents
```

The transcript prints to stderr while it runs, and the report is written to `work/fold-accents/reviews/006-behavior.md` for you to read afterwards.

The transcript goes in the file

Notice that the report file contains the raw session before it contains any finding.

Every finding says "observed: `""`" and the transcript above it shows `stdout: ""` on that input, from a real process. If a finding describes something the transcript does not show, you can see that in five seconds, without re-running anything. Put the evidence in the artefact, above the conclusions drawn from it.

Choosing what to try

The proposer sees the plan and nothing else — not the diff, not the code. That is deliberate, and it produces better inputs than showing it the implementation would.

A proposer that has read the code proposes inputs the code obviously handles, because those are the salient ones in its context. A proposer working from the plan alone has to think about the *problem*, and it reliably comes back with things like:

```
"Hello There World"
"Crème Brûlée"
"Weiß"
"///"
" "
""
"ÅÄÖ"
"naïve café"
"\u0627\u0644\u0639\u0631\u0628\u064a\u0629"
"\u03a9"
```

The ninth and tenth lines are Arabic and a Greek capital omega, written with JSON's `\u` escapes because that is what a model sends when it is unsure your terminal can carry the characters; either form parses. The quoting makes `" "` and `""` two distinct, transmissible inputs instead of two blank lines that a line-based parser throws away.

Most of those are in nobody's test file. Two of them — the non-Latin scripts — are explicitly out of scope per the plan's non-goals, and the observer does not report them as failures.

Example: `slugify(" ")` runs the space through the character filter, produces a single hyphen, then the trim step removes it, leaving the empty string — the same failure as `"///"` but arriving by a different route, which means a fix that special-cases separators can pass one and fail the other. No static lens found that, and the program found it in forty milliseconds.

Beyond a command-line program

A CLI is the easy case, and it was chosen here because the whole lens fits on two pages. The pattern survives the move to bigger things; the middle step, the part your code owns, is what changes.

A web service. Start the server as a child process, wait for the port to accept a connection, issue real HTTP requests, record status codes and bodies, then shut it down. The waiting is where this goes wrong: sleeping for two seconds works on your laptop and fails on a loaded machine, and a retry loop against the port is four lines and works everywhere.

A browser application. Drive it with a real browser automation library, take the page's visible text as the observation, and capture a screenshot on every step. The screenshots are not decoration — they are the transcript, and a reviewer that describes a screenshot it was given is doing the same job as the observer above.

A library with no entry point of its own. Write the smallest program that uses it the way a consumer would, and treat that as the application. If writing that program is awkward, that awkwardness is an observation no static lens produces.

> **Pro Tip:** Whatever the application is, keep the three roles separate in the code: one function proposes, one function *executes and records*, one function reads the record. When those three collapse into one prompt the output looks the same, and the lens stops being evidence.

Recording versus reasoning

The five static lenses and the behavior lens are not equally likely to be *wrong in a way that matters*. A static lens is reasoning from a partial

view of the code to a conclusion about behaviour, and every step in that reasoning is a place to be plausibly incorrect. The behavior lens is reasoning from a recorded fact to a comparison against a written criterion, which is a much shorter distance to travel.

There is also an asymmetry in the failures. A static lens that is wrong invents a defect that does not exist, or misses one that does, and in both cases the error is invisible without independent work. A behavior lens that is wrong is wrong *about a transcript printed above it*, and takes ten seconds to catch.

The static lenses find whole categories this one cannot: a resource that is never released, an input that is not sanitised on a path nobody exercised, a piece of duplication that will cause a bug next quarter.

Rule: A workflow whose output somebody relies on needs at least one job whose output is a recording rather than an opinion, and in this pipeline that job is the behavior lens you just built. If every artefact in your pipeline was produced by a model reasoning about text, then the whole pipeline's contact with reality is exactly zero, no matter how many stages it has.

The things this lens cannot do

Four limitations, all of them real:

It only sees what it ran. Ten inputs is ten inputs. The observer is instructed to say "not exercised" rather than to infer, which converts a gap into a visible gap instead of a silent assumption, but the gap is still there.

It needs the application to start, which costs nothing for a CLI. For a service with a database and three environment variables it is real work, done once and reused.

It is slow. Ten process launches through `npx tsx` is a few seconds. This lens will always be the slowest of the six.

It can be fooled by a program that fails without saying so. A CLI that prints nothing and exits 0 looks calm in a transcript. This is why the `exit code` line is recorded on every observation and why the plan's criteria should name the exit code as well as the output.

Six reports and no decision

You now have six files in `work/fold-accents/reviews/`: five from reading and one from running. Between them, perhaps fifteen findings, of which perhaps ten are real, two are duplicates of each other in different words, and three are wrong.

Nothing has been fixed, nothing has been decided, and no one has said whether this change may proceed.

Six opinions are not a decision, and the gap between them is where most workflows built on this pattern come apart: either by treating any finding as a stop-the-world event, or by treating the whole pile as advisory and merging anyway. The next chapter is about who decides, what they are allowed to do, and what they are not.

Chapter 9

Somebody Has to Decide

A judge decides what happens to each finding and records the reason, and this chapter builds one. Six files hold perhaps fifteen findings between them. Three of those are wrong, two are the same thing said twice, one is a preference dressed as a defect, and somewhere in the pile are two that genuinely must be fixed before this change goes anywhere.

The two ways of not deciding

There are exactly two ways to avoid making the decision.

Fix everything. Take all fifteen findings, hand them back, apply them all. This fails for a reason that has nothing to do with effort: a third of the findings are wrong, and applying a wrong finding actively damages the change. You also lose the distinction between "must fix before this ships" and "worth knowing about," which means the next change carries all fifteen again.

Ship it anyway. Read the pile, feel broadly reassured by its length, merge. It is the common outcome once review output exceeds what

anybody has time to read. The reviews still get generated, so there is now a record showing the problem was reported.

A panel of independent readers produces one opinion per reader rather than one decision, and six is this pipeline's number of readers.

Adjudication

Adjudication is the act of taking a set of reported findings and deciding, one by one, what happens to each — with reasons, recorded.

The output for each finding is its **disposition**, and there are exactly three:

- **Accepted.** The finding is real and in scope. It gets fixed now.
- **Rejected.** The finding is wrong, or it is a preference, or it describes something the plan explicitly excluded. It does not get fixed, and the reason is written down.
- **Escalated.** The finding cannot be settled from here, because it needs a decision that only a person is placed to make.

Three is the right number for the same reason three severities were: each names a different action. And the third is what keeps the first two honest: without somewhere to put a finding nobody here can settle, it gets forced into accepted or rejected.

An **escalation** is a finding raised to a human because settling it requires authority the workflow does not have: not because the question is hard, but because it is not the workflow's call. "The plan's AC3 requires `untitled`, but the API this feeds would rather have a 400" is not a defect anybody in this pipeline is entitled to resolve.

Without an escalation route, every finding must be either accepted or

rejected, and a system that cannot say "I don't get to decide this" will always find a reason to reject. Rejecting is free and finishing feels like success.

Rule: Any decision procedure that has no way to abstain will produce confident answers to questions it should not have answered. That is true of models, of teams, and of committees. Without the abstain option, every finding gets an answer, including the ones nobody could settle.

The verdict

The **verdict** is the recorded outcome of adjudication: every finding with its disposition and reason, every acceptance criterion with a yes or no and the evidence for it, and one overall result.

Three overall results:

- **ready** — every criterion met, no unresolved escalations, nothing outstanding.
- **ready with follow-ups** — every criterion met, no unresolved escalations, and some real problems recorded as future work. A change in that state still ships, because follow-ups are recorded work rather than objections, and treating them as blockers means the follow-up list becomes a reason never to finish anything.
- **blocked** — something is unmet, unresolved, or unsafe.

Blocked counts as a successful run. A run that ends *blocked*, with a clear statement of what is unresolved and why, has found something and refused to paper over it. The failure mode is not a blocked run. The failure mode is a `ready` verdict over a criterion that was not met.

The four prohibitions

A judge works under pressure: it has fifteen findings, an unmet criterion, and an implicit goal of producing a clean result. Every path to a clean result that does not involve doing the work is available to it, and every one of them is locally reasonable and articulately defensible. So they are named and forbidden, individually, in the system prompt:

Rule: The four prohibitions.

1. **You may not weaken, skip, or delete a test** to make something pass.
2. **You may not edit Intent, Acceptance criteria, or Non-goals.**
3. **You may not mark an acceptance criterion met when it is not met.**
4. **You may not expand scope.** A real defect outside this change is recorded as a follow-up rather than repaired here.

Each of those is a specific escape hatch, and each is the *shortest* path from a failing state to a clean-looking one:

Weakening a test is the fastest. One assertion loosened, one case deleted, and a red suite goes green with a diff of two lines that reads like a cleanup.

Editing the criteria leaves the least trace. AC2 said `weiss`; the code produces `wei`; changing AC2 to say `wei` produces a met criterion and a verdict that is internally consistent and false. Chapter 6 called this out; Chapter 11 makes it detectable by a machine, because a prompt cannot hold it.

Marking a criterion met is the most direct, and shows up as a verdict whose evidence column cites a file or a line that does not exist.

Expanding scope. The judge notices a genuine bug two functions over and fixes it while it is in there. Now the change is larger than what was reviewed, and the part nobody reviewed is the part written last, under the least supervision.

> **Warning:** Do not write these as "please avoid." Write them as numbered prohibitions with the reason attached, and put them in the system prompt where they sit outside the material being judged. It will not be enough on its own; Chapter 11 is about the half that has to be mechanical.

Building the judge

Two phases, and they are separate on purpose. First decide, with no ability to change anything. Then repair, with a decision already fixed in a file.

Listing: `agentic/judge.ts`

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { runAgent, FILE_TOOLS } from "./loop.js";

const PROHIBITIONS = [
  "You may not weaken, skip, or delete a test to make something pass.",
  "You may not edit Intent, Acceptance criteria, or Non-goals in the plan.",
  "You may not record an acceptance criterion as met when it is not met.",
  "You may not expand scope: a real defect outside this change is a",
  "follow-up, never a repair.",
].join("\n");

const ADJUDICATOR = [
  "You are judging a change. You did not write it and you did not review it.",
  "You have the plan, the diff, and every review report.",
  "",
  "Decide each finding: accepted, rejected, or escalated.",
  "  accepted  - real, in scope, must be fixed now.",
```

```
  "  rejected  - wrong, or a preference, or excluded by the plan. Say why.",
  "  escalated - cannot be settled without a decision a person must make.",
  "",
  "Reviewers are frequently wrong. Rejecting a finding with a clear reason",
  "is a correct outcome, and so is escalating one you are not entitled to",
  "settle. Do not accept a finding merely because it was reported.",
  "",
  "These prohibitions are absolute:",
  PROHIBITIONS,
  "",
  "Return exactly this document and nothing else:",
  "",
  "## Findings",
  "",
  "| # | Lens | Finding | Disposition | Reason |",
  "|---|---|---|---|---|",
  "",
  "## Acceptance criteria",
  "",
  "| Criterion | Met | Evidence |",
  "|---|---|---|",
  "",
  "## Escalations",
  "",
  "<one bullet each, or 'None.'>",
  "",
  "## Follow-ups",
  "",
  "<one bullet each, or 'None.'>",
  "",
  "## Outcome",
  "",
  "<exactly one of: ready | ready with follow-ups | blocked>",
].join("\n");

const REPAIRER = [
  "You are applying repairs that have already been decided by a judge.",
  "Fix exactly the findings marked accepted, and nothing else.",
  "",
  "These prohibitions are absolute:",
  PROHIBITIONS,
  "",
  "When you are done, list what you changed, one line per file.",
].join("\n");

function readOutcome(document: string): string {
  const match = /##\s*Outcome\s*\n+([^\n]+)/i.exec(document);
```

```
  return match === null ? "unknown" : match[1].trim();
}

async function main(): Promise<void> {
  const [slug, base] = process.argv.slice(2);

  if (slug === undefined) {
    console.error("usage: tsx agentic/judge.ts <slug> [base-ref]");
    process.exit(1);
  }

  const dir = path.join("work", slug);
  const reviewsDir = path.join(dir, "reviews");

  if (!fs.existsSync(reviewsDir)) {
    console.error(`No reviews at ${reviewsDir}. Run review.ts and behave.ts.`);
    process.exit(1);
  }

  const plan = fs.readFileSync(path.join(dir, "plan.md"), "utf8");

  const reviews = fs
    .readdirSync(reviewsDir)
    .filter((name) => name.endsWith(".md"))
    .sort()
    .map((name) => {
      const body = fs.readFileSync(path.join(reviewsDir, name), "utf8");
      return `----- ${name} -----\n${body}`;
    })
    .join("\n\n");

  const diff = execFileSync("git", ["diff", base ?? "HEAD", "--", "."], {
    encoding: "utf8",
    maxBuffer: 32 * 1024 * 1024,
  });

  // --- Phase 1: decide. No tools, so nothing can be changed here.
  const decision = await runAgent(
    ADJUDICATOR,
    ["PLAN", "====", plan, "", "DIFF", "====", diff, "", "REVIEWS", "=======",
      reviews].join("\n"),
    [],
  );

  const verdictFile = path.join(dir, "verdict.md");
  fs.writeFileSync(verdictFile, `${decision.text.trim()}\n`, "utf8");
```

```
  const outcome = readOutcome(decision.text);
  console.log(`outcome: ${outcome}`);
  console.log(`wrote ${verdictFile}`);

  if (!/\baccepted\b/i.test(decision.text)) {
    console.log("Nothing accepted. No repairs to apply.");
    return;
  }

  // --- Phase 2: repair. The decision is now a file, not an opinion.
  const repair = await runAgent(
    REPAIRER,
    ["VERDICT", "=======", decision.text, "", "PLAN", "====", plan].join("\n"),
    FILE_TOOLS,
  );

  fs.appendFileSync(
    verdictFile,
    `\n## Repairs applied\n\n${repair.text.trim()}\n`,
    "utf8",
  );

  console.log("repairs applied; see the end of verdict.md");
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Run the whole sequence:

```
npx tsx agentic/review.ts fold-accents
npx tsx agentic/behave.ts fold-accents
npx tsx agentic/judge.ts fold-accents

cat work/fold-accents/verdict.md
git diff
```

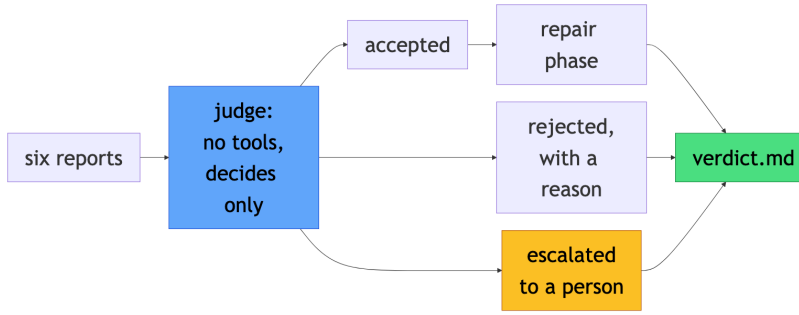


Figure 9.1: Six reports go in. One decision comes out, with a reason attached to every rejection.

A verdict, read properly

Here is a real one from the accents change, shortened only by cutting rows that say the same thing:

```
## Findings

| # | Lens | Finding | Disposition | Reason |
|---|---|---|---|---|
| 1 | spec-fidelity | AC3 empty case not implemented | accepted | Confirmed in
the diff: no fallback branch. |
| 2 | behavior | `"///"` prints a blank line | accepted | Same defect as 1,
observed. |
| 3 | correctness | `normalize` called on every input, including ASCII |
rejected | Not a defect. AC4 asks only that ASCII input is unchanged, and
nothing in the plan bounds cost. |
| 4 | simplicity | Prefer a lookup table to a replace chain | rejected |
Preference. No behavioural difference. |
| 5 | security | 5000-character input is not bounded | escalated | The plan is
silent on input limits. Product decision. |
| 6 | test-quality | No test covers the sharp-s case | accepted | AC2 exists;
nothing exercises it. |

## Acceptance criteria

| Criterion | Met | Evidence |
|---|---|---|
```

```
| AC1 accents fold | yes | behavior transcript: `Crème Brûlée` → `creme-brulee` |
| AC2 sharp s expands | yes | behavior transcript: `Weiß` → `weiss` |
| AC3 never empty | no | behavior transcript: `"///"` → `""` |
| AC4 ASCII unchanged | yes | behavior transcript: `Hello There` → `hello-there`
|

## Escalations

- Input length is unbounded. The plan does not say whether that is acceptable.

## Follow-ups

- None.

## Outcome

blocked
```

Read the evidence column first. Every "yes" cites something recorded by a process, not something concluded by a reader — which is what the behavior lens from Chapter 8 was for, and why its transcript is in the file.

Then read the rejections, of which there are two, each with a reason a person could disagree with in one sentence. That is what a rejection should look like: a claim you could argue with rather than a bare "not applicable."

The outcome is `blocked`, because AC3 is unmet and an escalation is open. The judge did not round that up, and it had every opportunity to: five of six criteria-shaped things looked fine, the change was clearly an improvement, and "ready with follow-ups" was one word away.

Example: Compare that with the same verdict written by a judge with no escalation option. Finding 5 has to become accepted or rejected. Accepting it means inventing a length limit nobody asked for — expanding scope, prohibition four. So it gets rejected, with a reason like "out of scope for this change," the escalations section is empty, and the outcome becomes `ready with follow-ups`. One missing option, and a real open question has been closed, with no record that it was ever open, by the only participant with no authority to close it.

The judge is not the builder

One structural rule that has not been said explicitly: the judge runs in its own fresh context, and it never sees the builder's conversation.

That is not an obvious choice. The builder knows things — why it chose one approach over another, what it tried that did not work, which of the reviewers' concerns it had already considered — and handing that over would make the judge better informed.

It would also make the judge anchored, which Chapter 7 has already been through at length. The builder's explanation of why a finding is not a problem is the most persuasive text that could possibly be placed in front of a judge deciding whether it is a problem, and it is persuasive in proportion to how convinced the builder was, which is uncorrelated with being right.

The cost: some rejections would have been better decisions with the builder's reasoning in hand, and some findings get accepted that the builder had already considered and correctly dismissed. That cost is paid deliberately, in exchange for the guarantee that no judgement in

this pipeline was produced by reading an argument written by the party it is judging.

Why the phases are separate

Phase 1 gets `[]` for tools, so it literally cannot edit a file, and the decision is made by something with no ability to make the decision come true.

That separation removes a whole class of failure that leaves no trace. A judge that can fix things while deciding will, sometimes, fix something and then record the finding as rejected — not dishonestly, but because by the time it wrote the row the problem was no longer there. The record and the reality diverge, and the record is what everything downstream reads.

It also makes the decision auditable, because `verdict.md` is written before a single line of source changes. If you want to know what the judge thought before it started editing, that file is it, on disk, timestamped by your version control.

> **Pro Tip:** Read `verdict.md` before `git diff`, always, in that order. The verdict tells you what was supposed to happen; the diff tells you what did. Read them the other way round and you are reconstructing intent from code, which is what every job so far has been arranged to avoid.

The judge is still a model

Everything Chapter 5 said applies here.

The judge hallucinates like anything else built on a model, and will occasionally cite evidence that does not exist: a line number, a function

name, a test file. It is sycophantic: a finding stated with great confidence gets accepted more readily than a hedged one, regardless of which is correct. And it is subject to the same pressure towards a clean result.

Three things reduce the damage, and none of them eliminate it:

It reads artefacts, not conversations. The plan, the diff, and the review files are all things written down by other processes, and no builder is in the room explaining itself.

Its prohibitions are specific. "Do not weaken a test" is checkable after the fact in a way that "be rigorous" is not, and Chapter 11 is where checking it after the fact happens.

Its own work is not covered by any of this, which is the gap the next section is about.

Nobody grades their own work — including the judge

Look carefully at what phase 2 produced. The repairer wrote code that was reviewed by nobody, because the six lenses ran before it existed.

This is a real hole and it does not have a clean fix. The obvious move — have the judge check its own repairs — is a model grading its own work, which is Chapter 5 again. The thorough move — run the six lenses again over the repairs, then judge those, then repair again — does not terminate; each round of repairs is new unreviewed code.

What actually works is a bounded, read-only pass: after the repairs are committed, run the lenses one more time over *just the judge's own commits*, and report what they find without fixing any of it. Findings in that pass are for a person, because a person is the only participant who can end the recursion.

Every automated review pipeline has exactly one piece of code that nothing reviewed: whatever was written last. Adding another stage does not close that gap — the new stage's output has the same property. It can only be made small, made visible, and handed to a human.

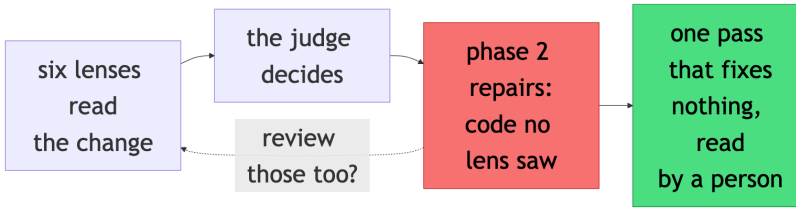


Figure 9.2: The dotted edge regenerates the condition it was taken to remove: every pass back through the panel ends in repairs no lens has read. The other edge out of that node is the only exit, and it ends at a person.

What is still only a promise

Read the four prohibitions again: what actually stops number two?

Nothing. It is a sentence in a system prompt. A judge under enough pressure — a red suite, an unmet criterion, an unattended run at three in the morning — can edit the plan's acceptance criteria and produce a verdict that is internally consistent, articulate, and completely false. Every check downstream reads that edited plan and agrees.

The same is true of "do not weaken a test": one deleted assertion turns the suite green, and a green suite produces a `ready` verdict.

Written in a system prompt, those four are requests rather than prohibitions. Some of them can be turned into rules that do not depend on cooperation, and the next two chapters do exactly that — first by building tests that are proven able to fail, and then by moving the rules a prompt cannot hold into software that never gets tired and cannot be argued with.

Chapter 10

Tests That Can Fail

This chapter builds a program that tells a test that works from a test that cannot fail.

There is a test in almost every codebase that has never failed and never could. It calls the function, it does something with the result, and it asserts something that was true before the feature existed and will be true after it is deleted. It runs green in every build, it appears in the count, and it is load-bearing in exactly one sense: somebody believes it.

What a test is for here

A **regression test** is an automated check that a behaviour that works today still works tomorrow: not a specification, documentation or a design aid, but a tripwire laid across a behaviour you have decided you care about.

In a workflow where a model writes the code, regression tests do a job they do not do in ordinary development.

Everything in Chapters 6 through 9 is a *judgement*: a plan somebody read, six reports somebody generated, a decision somebody made. All of it is subject to Chapter 5. A test is a different kind of thing entirely: it runs, it produces an exit code, and the exit code is not an opinion. It cannot be argued with, which is why the judge in Chapter 9 was not allowed to finish over a red suite.

The finished demo

Before there is anything to test, finish the running example. This is the version of `slugify` that meets all four criteria from Chapter 6.

Listing: `demo/src/slugify.ts`

```
// Characters that have a conventional multi-letter expansion. These must
// run before normalisation: NFD does not decompose ß into anything.
const EXPANSIONS: Array<[RegExp, string]> = [
  [/ß/g, "ss"],
  [/æ/g, "ae"],
  [/œ/g, "oe"],
  [/ø/g, "o"],
  [/đ/g, "d"],
  [/ł/g, "l"],
];

// Convert a piece of text into a URL slug.
export function slugify(input: string): string {
  let text = input.toLowerCase();

  for (const [pattern, replacement] of EXPANSIONS) {
    text = text.replace(pattern, replacement);
  }

  // NFD splits "é" into "e" + a combining acute accent; the range
  // U+0300-U+036F is those combining marks, which we then drop.
  const folded = text.normalize("NFD").replace(/[\u0300-\u036f]/g, "");

  const slug = folded.replace(/[^a-z0-9]+/g, "-").replace(/^-+|-+$/g, "");

  return slug.length > 0 ? slug : "untitled";
}
```

The tests

Node has a test runner built in, so there is no dependency to install and nothing to configure.

Listing: `demo/src/slugify.test.ts`

```
import test from "node:test";
import assert from "node:assert/strict";
import { slugify } from "./slugify.js";

test("ASCII text becomes a lowercase hyphenated slug", () => {
  assert.equal(slugify("Hello There World"), "hello-there-world");
});

test("accented Latin letters fold to their base letters", () => {
  assert.equal(slugify("Crème Brûlée"), "creme-brulee");
  assert.equal(slugify("naïve café"), "naive-cafe");
});

test("sharp s expands to ss in either case", () => {
  assert.equal(slugify("Weiß"), "weiss");
  assert.equal(slugify("WEI\u1E9E"), "weiss"); // capital sharp s
});

test("input with no slug characters becomes untitled", () => {
  assert.equal(slugify("///"), "untitled");
  assert.equal(slugify(" "), "untitled");
  assert.equal(slugify(""), "untitled");
});
```

Run it:

```
npx tsx demo/src/slugify.test.ts
echo "exit code: $?"
```

Four tests, green, exit code 0. Add it to `package.json` while you are here:

```
{
```

```
  "scripts": {
    "test": "tsx demo/src/slugify.test.ts"
  }
}
```

Each test corresponds to an acceptance criterion from the Chapter 6 plan. That is not a coincidence and it is not decoration: the criteria were written to be checkable by running something, so turning them into tests is transcription rather than design. If your criteria are hard to turn into tests, they were not falsifiable and Chapter 6's rule was not applied strictly enough.

Whether the tests check anything

The suite is green and all four tests pass, which leaves one question open: does any of them actually check anything?

The reasoning that they do — you wrote them and they look right — is the reasoning Chapter 5 dismantled, and it does not become sound because the text in question is a test file. Consider three assertions that pass and check nothing:

```
// 1. Asserts a tautology.
test("slugify returns a string", () => {
  assert.equal(typeof slugify("anything"), "string");
});

// 2. Asserts the behaviour of the test, not of the code.
test("handles accents", () => {
  const result = slugify("Crème");
  assert.ok(result.length >= 0);
});

// 3. Asserts something the fix did not cause.
test("sharp s is handled", () => {
  assert.notEqual(slugify("Weiß"), "");
});
```

Every one of these passes against the fixed code. Every one of them also passes against the *broken* code from Chapter 2. They are green in both worlds, which means they carry no information about which world you are in.

> **Warning:** A model asked to "add tests for this change" produces tests in this family at a noticeable rate — and they are hard to spot by reading, because they are syntactically identical to real ones. There is an assertion, a call, an expected value. The only way to tell is to break the code and see whether they notice.

Fail-first verification

Which is the technique, and it has a name.

Fail-first verification is proving that a test fails when the behaviour it guards is broken, before trusting it to guard anything. Break the code, run the test, see red, restore, see green.

It is old advice — it is the first half of test-driven development, arrived at from the other end — and it becomes non-negotiable the moment a model is writing either the code or the tests, because a test that cannot fail is exactly what an optimiser under pressure produces when asked to make a suite pass.

Doing it by hand takes a minute per test, which is why the job goes to a program.

Listing: `agentic/prove.ts`

```
import { execFileSync, spawnSync } from "node:child_process";
import { runAgent, FILE_TOOLS } from "./loop.js";

// The suite is defined once, in package.json. Naming the test files here
// too would leave this checking a subset of it the day you add one.
```

```
const TEST_COMMAND = ["npm", "test"];

const SABOTEUR = [
  "You are proving that a test is capable of failing.",
  "Make the smallest possible edit to the named source file that breaks the",
  "behaviour the named test checks - and nothing else.",
  "Do not touch any test file. Do not leave a comment explaining the change.",
  "Report the single line you changed and what it now does.",
].join("\n");

interface SuiteResult {
  green: boolean;
  output: string;
}

function runSuite(): SuiteResult {
  const [command, ...args] = TEST_COMMAND;
  const result = spawnSync(command, args, {
    encoding: "utf8",
    timeout: 120_000,
  });
  return {
    green: result.status === 0,
    output: `${result.stdout ?? ""}${result.stderr ?? ""}`,
  };
}

function restore(file: string): void {
  execFileSync("git", ["checkout", "--", file]);
}

// node:test prints one line per case - "ok 5 - name" for a pass, "not ok 3 -
// name" for a failure. Searching the whole output for the name matches the
// passing line just as happily, so match the failure line itself.
function failedAt(output: string, testName: string): boolean {
  return output
    .split("\n")
    .some((line) => /^not ok \d+ - (.*)$/.exec(line.trim())?.[1] === testName);
}

async function main(): Promise<void> {
  const [sourceFile, testName] = process.argv.slice(2);

  if (sourceFile === undefined || testName === undefined) {
    console.error('usage: tsx agentic/prove.ts <source-file> "<test name>"');
    process.exit(1);
  }
```

```
// This script restores the file with git. Refuse to run over work
// that git cannot give back.
const dirty = execFileSync("git", ["status", "--porcelain", "--", sourceFile],
{
  encoding: "utf8",
});
if (dirty.trim().length > 0) {
  console.error(`${sourceFile} has uncommitted changes.`);
  console.error("Commit or stash them first; this script uses git to undo.");
  process.exit(1);
}

const before = runSuite();
if (!before.green) {
  console.error("The suite is already red. A red suite proves nothing.");
  console.error(before.output);
  process.exit(1);
}
console.log("1. baseline ......... green");

const sabotage = await runAgent(
  SABOTEUR,
  [`Source file: ${sourceFile}`, `Test to break: ${testName}`].join("\n"),
  FILE_TOOLS,
);
console.log(`2. sabotage ......... ${sabotage.text.trim().split("\n")[0]}`);

const after = runSuite();

if (after.green) {
  restore(sourceFile);
  console.error(`\nNOT PROVEN: the suite stayed green with the code broken.`);
  console.error(`"${testName}" does not guard that behaviour.`);
  process.exit(1);
}

if (!failedAt(after.output, testName)) {
  restore(sourceFile);
  console.error(`\nNOT PROVEN: the suite went red, but not at
  "${testName}".`);
  console.error(after.output);
  process.exit(1);
}
console.log(`3. sabotaged ........ red, at "${testName}"`);

restore(sourceFile);
```

```
  const restored = runSuite();
  if (!restored.green) {
    console.error("\nThe suite is red after restoring. Check `git status`.");
    process.exit(1);
  }
  console.log("4. restored ......... green");

  console.log(`\nPROVEN: "${testName}" fails when its behaviour is broken.`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

Prove one:

```
git add -A && git commit -m "slugify: fold accents, expand sharp s, guard empty"
npx tsx agentic/prove.ts demo/src/slugify.ts "sharp s expands to ss in either
case"
```

Four lines of output and an exit code. Now run it against one of the useless tests from the previous section and watch it print `NOT PROVEN`.

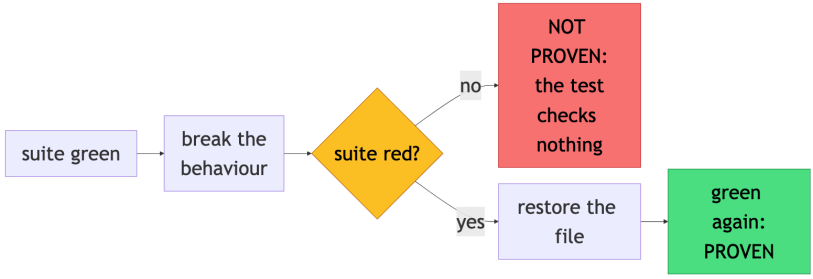


Figure 10.1: Green, break, red, restore, green. A test that does not go red in step three is not a test.

Rule: In this pipeline a test is not finished when it passes. It is finished when it has been observed to fail for the right reason. That bar is higher than most suites are held to, and it is set there because the suite is the only unarguable signal in the run: everything between those two states is a test you are taking on trust.

Skip, only, and the quietest way to a green suite

Two markers in every test framework can silently shrink the suite.

`skip` excludes a test from the run. It reports as skipped, which at least appears in the output — though nobody reads a summary line saying `4 passed, 1 skipped` any more carefully than one saying `5 passed`.

`only` is the more dangerous of the two. Marking a single test `only` runs *that test and nothing else*, and the suite reports green. One word, in one file, and forty tests stop running while every dashboard stays the colour it was.

Warning: `only` is the cheapest way for anything under pressure — a model, or a person at the end of a long day — to buy a passing suite. It is a one-word edit, it is syntactically unremarkable, and it is invisible in a summary. Chapter 11 makes adding one mechanically detectable, because it is not a rule a prompt can hold.

Flakes

A **flake** is a test that passes and fails on the same code. It is not a mildly annoying test; it is a test that has stopped carrying information, because from now on every failure it produces will be read as "probably the flake again."

The usual four causes:

- **Time.** Anything comparing against `Date.now()`, anything with a timeout tuned on a fast laptop.
- **Order.** Shared state between tests, so the suite passes in whatever order you happen to run it in and fails in another.
- **Randomness.** Unseeded generators; also, in our world, anything that calls a model and asserts on the text that comes back.
- **The network.** A real HTTP call in a test is a test of somebody else's uptime.

Do not write tests that assert on model output. Test the code around the model: that the loop stops at `end_turn`, that a failing tool produces `is_error: true`, that the budget check throws at the right number. Those are properties of your program, and your program is deterministic.

> **Pro Tip:** When a test flakes, the correct response is to fix it or delete it — not to retry it. A retry converts an unreliable test into an unreliable test that takes three times as long and reports green. If you cannot make it deterministic, it belongs in a different category of check than the one that gates merges.

Coverage is a diagnostic, not a target

Coverage measures which lines ran during the suite. It is a genuinely useful number in exactly one direction: lines that never ran are definitely untested, and a coverage report is a fast way to find behaviour nobody exercised.

In the other direction it says nothing at all. A line that ran during a test in which nothing was asserted about it has full coverage and no

protection. Every tautological test in the earlier section produces perfect coverage of `slugify`.

Making a coverage percentage a target makes this worse: the cheapest way to raise coverage is to write tests that execute code without asserting much about it, which is the family of useless test this chapter is about detecting. Give a model a coverage target and you will get exactly that, quickly and in volume.

The merge gate

A **merge gate** is a check that must pass before a change can be merged, enforced by something other than the good intentions of the person merging.

The regression suite is the obvious one, and the only requirement is that "must" be real. A suite that runs and reports, while merging remains possible over a red result, is a notification. That distinction is the entire subject of the next chapter.

For now, one rule that belongs with the tests rather than with the enforcement:

Rule: No verdict over a red suite. A `ready` outcome on a failing suite is not a judgement call, it is a contradiction — and `judge.ts`, as you wrote it in Chapter 9, never runs the suite at all. The obvious repair is to call `runSuite` from inside the judge, and it is the wrong one: something that checks itself can be argued out of the check, or can stop performing it with nobody the wiser. The next chapter enforces this rule from outside the judge, which is the only place enforcing it means anything.

Which tests to write, and how many

`prove.ts` does not belong on every test, for the same reason coverage targets fail: a large suite of weak tests costs more to run and proves less.

Three questions, in order, settle what to write:

What does the plan say? Every acceptance criterion becomes a test if it can. Chapter 6 already did the thinking: four criteria give you four tests.

What did the reviewers find? Every accepted finding from Chapter 9 is a defect that got past everything else once. A test for it is a test with a demonstrated failure mode, which means fail-first verification on it is nearly guaranteed to pass — you already know how to break it.

What would be expensive to discover late? Rarely more than a handful of things per system. The path that writes to disk, the one that talks to a payment provider, the one where a wrong answer is silent rather than loud.

Everything else is optional, and "optional" is not a euphemism.

> **Example:** The accents change produced exactly four tests, one per criterion. The judge's accepted findings added none: findings 1 and 2 *were* criteria, and finding 6 asked for a test that AC2 already called for. The tests, the criteria, and the findings all point at the same small set of behaviours.

Running the suite in the workflow

The suite has to run somewhere in the pipeline.

Not in the builder. A builder that runs the tests and reads the result

is a builder under pressure to make the result green, which is prohibition one from Chapter 9 again.

In the judge, after repairs. The judge's last action before writing an outcome is to run the suite, and a red suite makes `ready` impossible.

In the behavior lens, incidentally. Chapter 8's lens exercises the assembled application, which for a library-shaped project overlaps with the suite. Overlap is fine; it is two different questions arriving at the same code.

And once more somewhere none of your own code runs at all, which is where Chapter 11 goes.

What still rests on cooperation

`prove.ts` is a real mechanism: it produces an exit code, it cannot be talked round, and it fails loudly.

Nothing in the pipeline runs it: it is a script somebody has to remember, at a moment when remembering it is the only thing standing between them and being finished. The same is true of every rule in the last four chapters: do not edit the acceptance criteria, do not delete a test, do not add `only`, do not declare `ready` over a red suite. Every one of those is stated somewhere and enforced by nothing.

The next chapter moves the ones that can be mechanical into software that runs whether anybody remembers it or not.

This page intentionally left blank

Chapter 11

Rules a Prompt Can't Hold

This chapter builds a guard: a program that refuses a change when a rule has been broken. It also names three places to run it from.

Here is the complete list of rules stated so far, and what enforces each one.

Rule	Enforced by
Do not edit Intent, Acceptance criteria, or Non-goals	a sentence
Do not weaken, skip, or delete a test	a sentence
Do not mark a criterion met when it is not	a sentence
Do not expand scope	a sentence
Reviews are a record; do not edit them	a sentence
No verdict of `ready` over a red suite	a sentence
Every test proven able to fail	a script nobody runs

Of those seven rules, six are requests, and the seventh is a request to run something.

That was defensible while you were sitting in front of the terminal watching each job. It stops being defensible the moment the pipeline runs unattended. A rule that depends on the cooperation of the thing it constrains is not a rule. It is a preference that has been written down.

Which rules can be mechanical

Only some of them can be, and the test is a single question: **could a program decide this from files, without understanding anything?**

- "Did Intent change since it was approved?" — yes. Hash it before, hash it now, compare.
- "Was a test case removed?" — yes: count them in the old version and count them in the new one.
- "Was a review file edited?" — yes: ask git.
- "Does the verdict say `ready` while a criterion says no?" — yes: read two sections of one file and compare them.
- "Is this change well designed?" — no. Not now, not with a better model, not ever. That is a judgement, and judgement is what Chapters 7 through 9 are for.

The first four are the rules a model under pressure has the strongest incentive to break, and they are all *shortcuts*: not the honest path to a clean result but the cheap one. A machine that can catch the four cheap paths does not need to be clever, and it never gets tired.

The rules here are split by kind rather than by importance: judgements go to reviewers, facts go to a program. Splitting by importance leads to asking a model to check something a `grep` could settle, which costs money, takes seconds, and is wrong some of the time.

A violation is not a finding

A **finding**, from Chapter 7, is a claim a reviewer makes. A judge weighs it and may reject it, and rejecting it is often correct.

A **violation** is a rule that has been broken. Nothing in the pipeline may weigh it, reject it, or explain it away, because the rules are the things the pipeline states it does not do, and a judge that could adjudicate a violation would hold the very authority the violation exists to deny it.

> **Rule:** This guard reports violations rather than adjudicating them, and the run does not proceed until they are gone. Guards with severity levels and waivers exist and work; this one has neither, because a rule that can be excused is a rule a model can argue with. A rule that needs an exception written for it should have been a finding; demote it and take it out of the guard.

The guard

It is one script, which calls no model, needs no API key, and takes about a second.

Listing: `agentic/guard.ts`

```ts
import fs from "node:fs";
import path from "node:path";
import { createHash } from "node:crypto";
import { execFileSync, spawnSync } from "node:child_process";

interface Violation {
  rule: string;
  detail: string;
}

// The same one command CI runs. A guard that checks a different suite
// from the one the project declares is checking nothing in particular.
```

```
const TEST_COMMAND = ["npm", "test"];

function git(args: string[]): string {
  return execFileSync("git", args, {
    encoding: "utf8",
    maxBuffer: 32 * 1024 * 1024,
  });
}

function gitShow(ref: string, file: string): string | null {
  const result = spawnSync("git", ["show", `${ref}:${file}`], {
    encoding: "utf8",
  });
  return result.status === 0 ? (result.stdout ?? "") : null;
}

function sectionLines(document: string, name: string): string[] {
  const lines = document.split("\n");
  const start = lines.findIndex(
    (line) => line.trim().toLowerCase() === `## ${name.toLowerCase()}`,
  );
  if (start === -1) return [];
  const rest = lines.slice(start + 1);
  const end = rest.findIndex((line) => line.startsWith("## "));
  return end === -1 ? rest : rest.slice(0, end);
}

function normalised(document: string, name: string): string {
  const lines = sectionLines(document, name);
  return lines.length === 0
    ? `<<missing: ${name}>>`
    : lines.join("\n").trim().replace(/\s+/g, " ");
}

function requirementsHash(plan: string): string {
  const body = ["Intent", "Acceptance criteria", "Non-goals"]
    .map((name) => normalised(plan, name))
    .join("\n~~~\n");
  return createHash("sha256").update(body).digest("hex");
}

function countCases(source: string): number {
  return (source.match(/(^|[^.\w])test\s*\(/g) ?? []).length;
}

function countExclusions(source: string): number {
  return (source.match(/\btest\.(only|skip)\s*\(/g) ?? []).length;
```

```
}

function main(): void {
  const args = process.argv.slice(2);
  const record = args.includes("--record");
  const positional = args.filter((arg) => !arg.startsWith("--"));
  const slug = positional[0];
  const base = positional[1] ?? "origin/main";

  if (slug === undefined) {
    console.error("usage: tsx agentic/guard.ts [--record] <slug> [base-ref]");
    process.exit(1);
  }

  const dir = path.join("work", slug);
  const plan = fs.readFileSync(path.join(dir, "plan.md"), "utf8");
  const hashFile = path.join(dir, "requirements.sha256");

  // Run once, by a person, at the moment the plan is approved.
  if (record) {
    fs.writeFileSync(hashFile, `${requirementsHash(plan)}\n`, "utf8");
    console.log(`recorded ${hashFile}`);
    return;
  }

  const violations: Violation[] = [];

  // --- requirements: nobody moved the target.
  if (!fs.existsSync(hashFile)) {
    violations.push({
      rule: "requirements",
      detail: `${hashFile} is missing; nothing pins what was approved.`,
    });
  } else if (fs.readFileSync(hashFile, "utf8").trim() !== requirementsHash(plan))
{
    violations.push({
      rule: "requirements",
      detail: "Intent, Acceptance criteria, or Non-goals changed after
approval.",
    });
  }

  // --- tests and reviews: read the change, not the intentions.
  const changed = git(["diff", "--name-status", base, "--"]).trim();

  for (const line of changed.length === 0 ? [] : changed.split("\n")) {
    // A rename prints three fields: "R100\told\tnew". The path on disk now is
```

```
  // the last one; the path to look up at the base ref is always the second.
  // Reading the first field as the file is how this crashes on a rename.
  const fields = line.split("\t");
  const status = fields[0];
  const file = fields[fields.length - 1];
  const previous = fields[1];
  if (file === undefined || previous === undefined) continue;

  const reviews = `${dir}/reviews/`;
  if (
    (file.startsWith(reviews) || previous.startsWith(reviews)) &&
    status !== "A"
  ) {
    violations.push({
      rule: "reviews",
      detail: `${previous} was ${status === "D" ? "deleted" : "modified"}; ` +
        "reviews are append-only.",
    });
  }

  if (file.endsWith(".test.ts") || previous.endsWith(".test.ts")) {
    const before = gitShow(base, previous);
    const after = status === "D" ? "" : fs.readFileSync(file, "utf8");

    if (before !== null && countCases(after) < countCases(before)) {
      violations.push({
        rule: "tests",
        detail: `${file}: ${countCases(before)} cases before, ` +
          `${countCases(after)} now.`,
      });
    }

    const exclusionsBefore = before === null ? 0 : countExclusions(before);
    if (countExclusions(after) > exclusionsBefore) {
      violations.push({
        rule: "tests",
        detail: `${file}: a new skip or only marker was added.`,
      });
    }
  }
}

// --- verdict: 'ready' has to mean something.
const verdictFile = path.join(dir, "verdict.md");

if (fs.existsSync(verdictFile)) {
  const verdict = fs.readFileSync(verdictFile, "utf8");
```

```
  const outcome = normalised(verdict, "Outcome").toLowerCase();

  if (outcome.startsWith("ready")) {
    const unmet = sectionLines(verdict, "Acceptance criteria").filter((row) =>
      /\|\s*no\s*\|/i.test(row),
    );
    if (unmet.length > 0) {
      violations.push({
        rule: "verdict",
        detail: `outcome '${outcome}' with ${unmet.length} unmet criteria.`,
      });
    }

    const escalations = normalised(verdict, "Escalations");
    if (!/^none\.?$/i.test(escalations)) {
      violations.push({
        rule: "verdict",
        detail: `outcome '${outcome}' with open escalations.`,
      });
    }

    const suite = spawnSync(TEST_COMMAND[0], TEST_COMMAND.slice(1), {
      encoding: "utf8",
      timeout: 120_000,
    });
    if (suite.status !== 0) {
      violations.push({
        rule: "verdict",
        detail: `outcome '${outcome}' over a red suite.`,
      });
    }
  }
}

if (violations.length === 0) {
  console.log(`guard: clean (${slug})`);
  return;
}

for (const violation of violations) {
  console.error(`VIOLATION [${violation.rule}] ${violation.detail}`);
}
console.error(
  `\nguard: ${violations.length} violation(s). ` +
    "These are rules, not findings. Nothing here may adjudicate them away.",
);
process.exit(1);
```

```
}

try {
  main();
} catch (error) {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
}
```

Record the baseline when you approve a plan, and check it whenever you like:

```
npx tsx agentic/guard.ts --record fold-accents
npx tsx agentic/guard.ts fold-accents "$(git merge-base HEAD origin/main)"
```

Then try to break it. Open `work/fold-accents/plan.md`, change one word in an acceptance criterion, and run the guard again:

```
VIOLATION [requirements] Intent, Acceptance criteria, or Non-goals changed
after approval.
```

No model was consulted and no judgement was made: a hash did not match, and the process exited 1.

Example: The `tests` rule counts cases as a **delta against the base ref** rather than as an absolute, which is what makes it usable on an existing codebase: a repository that already carries forty skipped tests trips nothing, because the count before and the count after are both forty. Only exclusions your change introduces are violations. A guard that fires on pre-existing conditions gets switched off in a week.

Three layers

The guard is a program, and programs have to be run by something; *what* runs them is the layer.

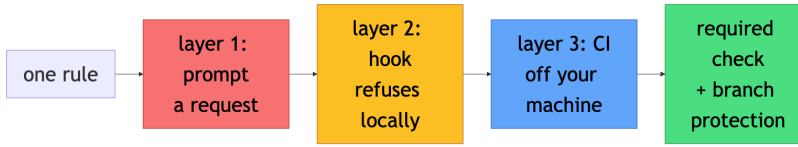


Figure 11.1: The same rule, enforced three times, with the last one outside anybody's reach.

Layer one: the prompt

This is where every rule so far started: a numbered prohibition in a system prompt.

It shapes behaviour most of the time, at no cost, for rules a machine cannot check, and it fails in the very circumstances you built the pipeline for: unattended, under pressure, with an articulate justification available.

Layer two: the hook

A **hook** is a program that a tool calls at a defined point in its own execution, and whose exit code the tool respects. Your code does not call the hook; the tool does, whether anybody remembered it or not.

Git provides them. Create `.githooks/pre-commit`:

Listing: `.githooks/pre-commit`

```
#!/usr/bin/env bash
set -euo pipefail

slug="$(git rev-parse --abbrev-ref HEAD | sed 's|.*/||')"

# Not every branch is a work item. Say nothing on the ones that are not.
[ -d "work/$slug" ] || exit 0

base="$(git merge-base HEAD origin/main 2>/dev/null || echo HEAD)"
npx tsx agentic/guard.ts "$slug" "$base"
```

Install it, once, per clone:

```
chmod +x .githooks/pre-commit
git config core.hooksPath .githooks
```

Now a commit that carries an edited acceptance criterion does not happen at all, rather than being reported after the fact.

The same idea reaches inside the loop itself. Chapter 3's `runAgent` executes whatever tool the model asks for; a check placed immediately before that execution can refuse:

```
function beforeTool(name: string, input: unknown): void {
  if (name !== "write_file") return;

  const target = String((input as Record<string, unknown>)["path"] ?? "");
  if (target.endsWith(".test.ts")) {
    throw new Error(
      "write_file: refusing to modify a test file during a repair.",
    );
  }
}
```

Throwing here produces a `tool_result` with `is_error: true`, which the model reads and works around — it does not crash the run. Prohibition one from Chapter 9 has just stopped being a request. Packaged agent tools expose this same point under the name **`PreToolUse`**: a hook that runs before a tool call and can refuse it. You have now written the check that refuses the call.

> **Pro Tip:** A hook that blocks something should always say what to do instead. `"refusing to modify a test file during a repair"` is a message a model can act on; `"forbidden"` is one it will retry against three times before giving up.

Layer three: outside the machine entirely

Hooks share one weakness with prompts, and it is fatal on its own: they run on somebody's machine, under somebody's configuration, and `git commit --no-verify` skips every one of them.

Continuous integration — **CI** — is a service that runs your checks on its own machines whenever a change is proposed, using the repository's committed configuration rather than anybody's local setup. Nothing on a developer's laptop can switch it off, because it is not running on a developer's laptop.

Listing: `.github/workflows/guard.yml`

```yaml
name: guard

on:
  pull_request:

jobs:
  guard:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - uses: actions/setup-node@v4
        with:
          node-version: 20

      - run: npm ci

      - name: Regression suite
        run: npm test

      - name: Guard rules
        run: |
          slug="${GITHUB_HEAD_REF##*/}"
          if [ -d "work/$slug" ]; then
            npx tsx agentic/guard.ts "$slug" "origin/${GITHUB_BASE_REF}"
          else
            echo "no work item for branch $GITHUB_HEAD_REF; nothing to guard"
```

```
        fi
```

`fetch-depth: 0` is not optional, because the guard compares against a base ref and the default shallow checkout does not have one. The job fails with an unhelpful git error.

The setting you cannot commit

Writing that workflow does not gate anything. It runs, it reports, and a change with a red guard can still be merged by anyone who clicks the button.

Turning it into a gate takes two settings in the repository host, and neither of them lives in your repository:

A **required status check** is a named check that must report success before a change may be merged. **Branch protection** is the setting that makes those requirements apply to a particular branch — typically your default branch — and that removes the ability to push to it directly.

Both are administrative settings in the hosting interface. Nothing in your codebase can turn them on, no script you have written can verify them from inside a normal run, and an unticked box looks just like a working gate from every angle except the one where somebody merges a red change.

> **Warning:** Go and look at the branch protection settings. Confirm the check is listed as required. If you cannot confirm it, the honest status is *unknown* — which is a different answer from *enforced*, and rounding one to the other is how a repository ends up with a merge gate that has never blocked anything.

Vendoring, and the check that goes stale without saying so

CI runners start with nothing installed and cannot load the tooling on your laptop, so the checker they run has to be *in the repository*, which is why `.github/workflows/guard.yml` runs `agentic/guard.ts` from the checkout rather than fetching it from anywhere.

Copying a shared tool into a repository so it runs without external dependencies is **vendoring**. It is the right call here, and it has one well-known consequence: a vendored copy is frozen on the day it was made. Improve the guard, add a rule or fix a bug, and every repository that vendored the old version keeps enforcing the old rules while reporting green throughout.

A stale checker does more harm than an absent one, because the green tick reads as enforcement. If you ever share a guard across repositories, the guard should check that its own copy matches the original, and treat a mismatch as a violation like any other.

What a guard cannot do

It checks shapes, not substance. The `tests` rule counts cases. A change that keeps the count and guts every assertion passes cleanly. That is what Chapter 10's `prove.ts` is for, and it is why fail-first verification is not something a guard can replace.

It can be satisfied dishonestly. Record the hash *after* editing the criteria and the `requirements` rule is silent. The protection comes from *when* the recording happens — at approval, by a person, before any work — and nothing in the script enforces that ordering. It is a rule about a workflow, not a rule about a file.

It is only as good as its rules. Five rules catch five shortcuts, and the sixth shortcut, the one nobody has thought of yet, is unguarded and will look just like a clean run.

It lives inside the thing it constrains. `agentic/guard.ts` sits in the project root, the same surface Chapter 9's repairer writes to, and all three layers run the copy that is in the branch. An agent that rewrote `main()` to print `guard: clean` would be validated by its own edit, hook and CI job and all. Closing that properly means running the guard from somewhere the change cannot reach: check out the base ref into a scratch directory and run *that* copy against the branch, or keep the enforcement files off the writable surface your tools expose.

Mechanical enforcement does not remove the need for judgement. It removes the need for judgement *about the things a machine can settle.*

Where this leaves you

Eleven chapters have produced a plan format, a builder, six lenses, a judge, a suite proven able to fail, a guard, a hook, and a job that runs on somebody else's computer.

Every line of it is code you wrote. That is the position from which to look at somebody else's version.

Chapter 12

From Yours to Shared

This chapter hands the pipeline from the last eleven chapters over to something that already does the plumbing.

You have a working delivery pipeline made of nine TypeScript files — `setup`, `loop`, `agent`, `plan`, `review`, `behave`, `judge`, `prove`, `guard` — and a little over a thousand lines between them. It plans, builds, reviews from six directions, adjudicates, tests, and guards.

It also only exists on your machine, in one repository, and it is entirely yours to maintain.

Why this comes last

Chapter 5 established the specific difficulty: a workflow that is producing confident output and a workflow that is producing correct output look identical from outside. A packaged pipeline has the same property one level up. It runs, it prints reassuring things, and it opens a change for you to read.

Rule: Build the pipeline yourself first and delegate it afterwards, which is the order these chapters put you through: delegate only once you can describe how it fails.

The harness

A **harness** is the program that runs the loop, manages the conversation, owns the tool set, and decides what the model sees. `loop.ts` is a harness. So is any agent product you have used: an editor extension, a command-line assistant, a hosted agent service.

Every one of them is Chapter 3's loop with three things added: a larger tool set, context management from Chapter 4, and a way to configure the system prompt without editing source.

The one used in the rest of this chapter is Claude Code, because it is Anthropic's, because its configuration lives in files in your repository, and because the pieces map cleanly onto what you built. Everything here has an equivalent elsewhere; only the names differ.

What moving to a harness gives you is the plumbing, and the plumbing is genuinely tedious: file editing that applies patches rather than rewriting whole files, a sensible tool set, permission prompts, context compaction, session resumption, cost reporting. Nothing in that list is an idea; all of it is work somebody has to do.

What it does not give you is the plan format, the six lenses, the four prohibitions or the guard rules — none of which is plumbing.

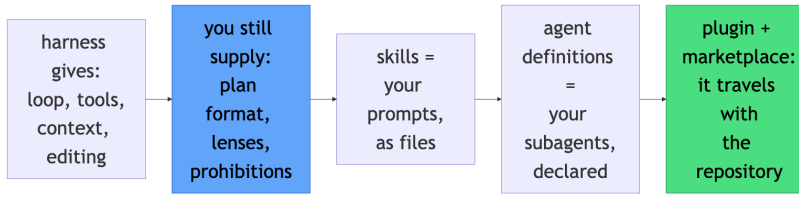


Figure 12.1: What the harness takes over, and what stays yours. The middle column is the part you specify.

Skills: your prompts, as files

A **skill** is a folder containing instructions the harness loads on demand, usually a `SKILL.md` with a name, a description, and a body. The description is what the harness matches against the task at hand; the body is what gets put in front of the model when it does.

Which is to say: it is a system prompt with a filing system. `agentic/review.ts` holds `REVIEWER` in a string constant; the same text in a skill file is available to anyone who opens the repository, travels with the code, and is versioned by git.

Here is the Chapter 8 behavior lens as a skill:

```
---
name: behavior-review
description: >
  Review a change by running the assembled application and reporting what
  actually happens. Use before merging any change with a runnable entry point.
---

# behavior-review

You are the behavior reviewer. You did not write this change.

1. Read `work/<slug>/plan.md` for the acceptance criteria.
2. Choose up to ten inputs: the ordinary case, every case the plan names,
   the empty case, and two or three a user would hit that it does not.
```

```
3. **Run them.** `npx tsx demo/src/cli.ts <input>` for each one, and record
   the exact stdout, stderr, and exit code.
4. Write `work/<slug>/reviews/006-behavior.md` with the raw session
   transcript first, then findings.

Never state what the program does beyond what the transcript shows. If a
case was not exercised, say so rather than reasoning about it.
```

Read step 3 again. The instruction to *run* it is now a sentence rather than a `spawnSync` call, and that is a real loss — Chapter 8 was emphatic that the running had to happen in code the model does not control. A skill can ask; it cannot guarantee. What it gets in exchange is that the harness's own tools do the running, with real output in the transcript, and the transcript requirement in step 4 makes a fabricated run visible.

> **Warning:** Watch this closely when you move a lens into a skill. Your version had a structural guarantee, in that the observation came from `spawnSync`, full stop, whereas the skill version has only an instruction. The mitigation is to demand the evidence in the artefact.

Agent definitions: subagents, declared

An **agent definition** is a file describing a subagent: its name, its system prompt, and the exact list of tools it is allowed to use.

```
---
name: reviewer
description: Reviews a diff from one lens. Finds problems; fixes nothing.
tools: Read, Grep, Glob, Bash
---

You are reviewing a change from exactly one lens, in fresh context.
You did not write it and you have not seen the reasoning behind it.

Report problems. Fix nothing. For each finding give the location, a
severity, why it is wrong, and a concrete input that produces the wrong
result. If you cannot name that input, the finding is a guess: say so.
```

The `tools:` line is doing something your version could not do as cleanly. A reviewer with no `Edit` and no `Write` cannot modify code, so "review fixes nothing" stops being an instruction in a prompt and becomes a property of the process. Chapter 7 achieved the same thing by passing `[]` for tools, and this is that idea promoted to a declaration anyone can read.

That reviewer still has `Bash`, because it needs it to read a diff, and `Bash` can write files. Removing `Edit` and `Write` closes the easy path and the accidental path without amounting to a sandbox. It raises the cost of drift substantially and it does not eliminate it, and a tool that claims otherwise is describing something it has not tested.

Orchestration is the layer that decides which of these run, in what order, with what handed between them: six lenses in parallel, then a judge over their output, then a guard. In your version, orchestration is `Promise.all` and the order of statements in `main`. In a harness it is a script or a skill that invokes the others. The concept does not change; only who writes the `for` loop.

Plugins and marketplaces

A **plugin** is a bundle of skills, agent definitions, hooks, and commands distributed as a unit. A **marketplace** is a place plugins are published and found — for Claude Code, a git repository with a manifest listing what it offers.

A plugin enabled in a *repository's* settings travels with the repository. Skills you put in your own home directory are personal to one machine: a teammate does not get them, a cloud session does not get them, and a scheduled run does not get them. A repository that declares its marketplace gets them everywhere, for everyone, including automation that has no home directory at all.

Adopting quorum

`AiFirstDevelopment/quorum` — at `https://github.com/AiFirstDevelopment/quorum` — is a marketplace carrying two plugins that implement the pipeline you have just built by hand. The marketplace and one of its plugins share a name, which is why the identifiers below read `quorum@quorum`.

`quorum` is the pipeline: plan, build, multi-lens review, adjudication, and a published change at the end. Its jobs are the chapters you have just worked through, in order:

Command	What it is, in the terms used here
`/quorum:1-plan`	Chapter 6. Writes `docs/work/<slug>/plan.md` and asks you to approve it.
`/quorum:2-build`	Chapters 3 and 6. Implements the plan; ticks its steps; records deviations.
`/quorum:3-review`	Chapters 7 and 8. Six lenses in fresh context, one file each.
`/quorum:4-quorum`	Chapter 9. Adjudicates, applies accepted fixes, ends on a green suite.
`/quorum:guard`	Chapter 11. The mechanical rules, and a CI job that enforces them.
`/quorum:pipeline`	The whole thing unattended, from an approved plan.

Its six lenses are `behavior`, `correctness`, `spec-fidelity`, `security`, `simplicity`, and `test-quality` — the same six, arrived at from the same argument. Its artefacts are `docs/work/<slug>/plan.md`,

`reviews/`, and `verdict.md`, which is the layout you built in Chapter 6 with a different root directory.

`tests` is Chapter 10: behavioral regression tests against the assembled application, each proven able to fail, and wired into CI as a required check.

Adoption is one file, committed:

Listing: `.claude/settings.json`

```
{
  "extraKnownMarketplaces": {
    "quorum": {
      "source": {
        "source": "github",
        "repo": "AiFirstDevelopment/quorum"
      }
    }
  },
  "enabledPlugins": {
    "quorum@quorum": true,
    "tests@quorum": true
  }
}
```

That one file is the entire installation. Anyone who opens the repository afterwards, whether that is you on another machine, a colleague, or an automated run, gets the same commands, because the settings are in version control rather than on a laptop.

Pin it, and understand why that is the conservative default:

```
"source": {
  "source": "github",
  "repo": "AiFirstDevelopment/quorum",
  "ref": "v1.0.0"
}
```

A plugin is a bundle of skills, agent definitions, hooks, and commands: code that runs on your machine and on your colleagues' machines, with

whatever file access and credentials they have. Omitting `ref` tracks the marketplace's default branch, so a commit pushed there tomorrow runs here tomorrow: no change in your repository, no review, no decision. That is occasionally what you want, and it is a decision about whose code you are willing to execute unread rather than a preference about release notes.

For its flags, its options, and the details of each command, read quorum's own README. Reproducing them here would be out of date by the time this is printed.

What is honestly wrong with it

Four limitations, none of which are solved, and all of which apply to every tool of this kind.

The lenses are not independent, because all six run on Claude models. Chapter 7 warned about this and the warning applies here: if the model is bad at noticing a particular class of defect, it is bad at it six times, and the six clean reports read just like six independent confirmations. The panel widens what gets looked at without producing independence at the level of the weights, and no amount of prompt variation creates it. Genuine independence would require different model families, or a human, and the pipeline has neither.

It costs real money per run. Up to thirteen agents, each receiving the diff and the plan, several of them reasoning at length. On a large change this is noticeable, and it recurs on every re-run of the branch. It is inexpensive against an engineer's hour and expensive against a habit of running it on every commit, which is why it runs per branch, on demand, and never on a hook.

No durable specification of record exists. `plan.md` describes a

single change, and merging it turns the plan into history; nothing accumulates into a statement of what the system as a whole is supposed to do. So the twentieth change is planned against the code, and against whatever the person remembers, rather than against a maintained account of intended behaviour. Every criterion is local, every criterion is per-change, and a contradiction between change three and change seventeen is invisible to the whole apparatus. Closing it is not a small piece of work.

Nobody has measured how many defects the panel catches. The guard rules have a proper test: a script that breaks each rule in a throwaway repository and asserts that it fires. The review panel has no equivalent, no corpus of real defects, no measured catch rate, and no false-positive rate published against a representative sample. What exists is a thin set of fixtures and a lot of observation. So the honest claim is *this finds things a single reviewer misses*, which is well supported, and not *this catches N per cent of defects*, which nobody can currently say about this or any comparable tool.

> **Warning:** Every one of those four questions, from whether the reviewers share a blind spot to what a run costs, whether a durable specification exists, and whether the catch rate has been measured, is a question to ask of any workflow tool you are offered, including the next one, and including one you wrote yourself.

What to keep from your own version

Keep `agentic/` in the repository after you adopt the plugins.

Three of those files remain useful after adoption:

`guard.ts`. Rules are project-specific. Your repository has a directory nobody may touch, a file that must stay generated, a convention that

matters. A guard is where those go, and a guard you wrote is one you can extend in an afternoon.

`prove.ts`. A script that runs fail-first verification end to end (green, break, red, restore, green) is the same five lines regardless of what harness is driving.

`budget.ts`. Cost visibility disappears the moment a tool reports its own usage in its own units, and forty lines that print dollars against your own arithmetic keep it.

And keep the rest as a reference implementation. When the packaged pipeline does something you do not understand, `agentic/` is a working version small enough to read in one sitting, which is a considerably faster route to an answer than reading somebody else's orchestration layer.

> **Pro Tip:** Adopt the pipeline on a change you would have been comfortable making by hand, and read the verdict against your own opinion of the change. If they agree, you have learned something about the tool. If they disagree, find out which of you is right before you adopt it for anything you could not have checked.

The end of the building

You now have one program that plans before it acts, one that reviews from six directions without ever seeing the argument for the change, one that decides and records why, one that proves its own tests can fail, and one that enforces the rules the others merely promise.

The next chapter opens the package you just installed and reads it, file by file, against the version you wrote.

Chapter 13

Inside a Plugin

This chapter builds a plugin, the smallest one that carries a piece of what you wrote, and then reads quorum's, directory by directory, as the same parts at full size.

Chapters 11 and 12 gave you the words: hook, skill, agent definition, plugin, marketplace. They did not say how a harness turns a directory of text files into a command you can type, or what happens between typing `/quorum:pipeline` and a pull request appearing. Until that is said, quorum is a package you have chosen to trust, and Chapter 12's rule was to delegate only once you can describe how the thing fails.

Nothing here needs more than Chapter 12 and a text editor. Every file in a plugin is JSON, Markdown, or a short script, and the harness reads each one in a way you can predict from where it sits.

What the harness reads

A harness starts in a directory and reads files at paths it knows. Claude Code reads `CLAUDE.md` for standing instructions and

`.claude/settings.json` for configuration, and it reads a plugin the same way: a plugin is a directory in a fixed shape, and each part is found by its path.

```
plugins/behavior/
  .claude-plugin/plugin.json      the manifest: name, version, description
  skills/review/SKILL.md          one skill, which becomes /behavior:review
  agents/reviewer.md              one agent definition, behavior:reviewer
  hooks/hooks.json                which event runs which command
  bin/reviews-append-only.mjs     the command the hook runs
```

The **plugin manifest** is the file at `.claude-plugin/plugin.json`. It carries a name, a version and a description, and the name does one job beyond identification: it becomes the **namespace**, the prefix in front of everything the plugin provides. The skill at `skills/review/` is invoked as `/behavior:review`, and the agent at `agents/reviewer.md` is addressed as `behavior:reviewer`. Two plugins can each carry a `review` skill without colliding, and a skill or an agent called by a bare name, without its prefix, does not resolve.

Listing: `plugins/behavior/.claude-plugin/plugin.json`

```
{
  "name": "behavior",
  "description": "The behavior lens from Chapter 8, and the append-only rule for
  reviews from Chapter 11.",
  "version": "0.1.0"
}
```

The layout settles one more thing, which is where a plugin lives. A plugin does not know where it will be installed: on your machine it is `plugins/behavior/` in this repository, and on a colleague's it is a copy under their home directory. Any file in the plugin that refers to another one writes `${CLAUDE_PLUGIN_ROOT}`, and the harness substitutes the **plugin root**, the directory the plugin was actually installed into, before it runs the command, and `hooks.json` below relies on that.

Those are the mechanics of discovery, and the rest of this chapter is what each part does once it has been found.

The skill, and whose conversation it runs in

A skill is `SKILL.md` in a directory of its own, with a name and a description in the frontmatter and instructions in the body. Chapter 12 called it a system prompt with a filing system. Here is what the filing system does.

When you type `/behavior:review fold-accents`, the harness reads the body of the file and places it in the conversation you are already in, as instructions, with `fold-accents` as the argument, and that is all it does. The skill runs with the tools your session has and the history your session has, so a skill that performs the review itself is Chapter 5's grader: it reads the code in the same context that wrote it.

So the skill does not do the review. It hands the review to something that has not seen the conversation.

Listing: `plugins/behavior/skills/review/SKILL.md`

```
---
name: review
description: >
  Review a change by running the assembled application and reporting what
  actually happens. Use before merging any change with a runnable entry point.
argument-hint: [slug]
---

# behavior:review

Hand this job to the `behavior:reviewer` agent, in its own context. Give it
the slug and nothing else: not what was built, not why, not what you expect
it to find. It has not seen this conversation and must not.

When it returns, write its report, unchanged, to
`work/<slug>/reviews/NNN-behavior.md`, where NNN continues from the highest
```

```
number already in that directory. Then tell the user where the file is and
how many findings it holds. Do not act on the findings; that is the judge's
job.
```

The `description` is matched against what the user is doing, so that the harness can offer the skill unasked; the body is what the model reads. "Hand this job to the `behavior:reviewer` agent" is not a figure of speech. The harness has a tool for starting an agent, and that sentence is an instruction to call it.

The agent definition, and where fresh context comes from

When the session hands a job to `behavior:reviewer`, the harness starts a new conversation. Its system prompt is the body of `agents/reviewer.md`. Its tools are the ones on the `tools:` line and no others. It receives one user message, the task, and runs Chapter 3's loop until the model stops asking for tools. Then the final text comes back to the session that started it, as the result of a tool call, and the new conversation is discarded.

That is `runAgent`, with the system prompt and the tool list read from a file instead of passed as arguments. The `const messages = []` that Chapter 7 built its independence on is the harness's first line, every time.

Listing: `plugins/behavior/agents/reviewer.md`

```
---
name: reviewer
description: >
  Runs the assembled application on inputs chosen from the plan and reports
  what it observed. Reads and runs; does not edit.
tools: Read, Bash
---
```

```
You are the behavior reviewer. You did not write this change and you have
not seen the reasoning behind it.

1. Read `work/<slug>/plan.md` for the acceptance criteria.
2. Choose up to ten inputs: the ordinary case, every case the plan names,
   the empty case, and two or three a user would hit that the plan does not.
3. Run each one with `npx tsx demo/src/cli.ts <input>`. Record the exact
   command, its stdout, its stderr, and its exit code, for every input.
4. Return a report with the raw transcript first, then one finding for each
   mismatch between what the plan requires and what the transcript shows:
   the input, the expected result, the observed result, and a severity of
   blocker, major, or minor. If everything matches, the findings section
   reads: No findings.

Never state what the program does beyond what the transcript shows. If a
case was not exercised, say so rather than reasoning about what would have
happened. Do not modify any file.
```

`tools: Read, Bash` is `READ_ONLY_TOOLS` from Chapter 3, as a line anyone can read. It is also the reason the reviewer returns its report rather than writing it: with no `Write`, the skill has to do the writing, and that division is worth keeping, because then the process that ran the program and the process that creates files are two processes rather than one.

Read step 3 again with Chapter 8 beside it. Your `behave.ts` ran the program from `spawnSync`, and the model could not have skipped that step if it wanted to. Here the running is an instruction, and `Bash` is a tool the model may or may not reach for. Chapter 12 warned about this trade and the warning stands: the guarantee is gone, and what replaces it is the transcript requirement in step 4, which makes a report that never ran anything look like what it is.

Warning: `Bash` can write files, whatever the `tools:` line leaves off. "Do not modify any file" is a request, as every sentence in a prompt is. Leaving `Write` and `Edit` off the `tools:` line closes the direct path and the accidental one; it does not close the shell. The next section is where one of these rules stops depending on the model's cooperation.

The hook: Chapter 11's `beforeTool`, called by a loop you did not write

Chapter 11 placed a function in front of `execute` in `runAgent`, so that a tool call the model asked for could be refused before it ran. The harness's loop has the same point in it, and `hooks.json` is how you attach a program there.

A **hook event** is a named moment in the harness's own loop at which it will run a program you have configured. `PreToolUse` is the moment after the model has asked for a tool and before the harness runs it. There are others, for the start of a session, for the end of a turn, and for the point after a tool has run, and they all work the same way as this one.

Listing: `plugins/behavior/hooks/hooks.json`

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "node
            \"${CLAUDE_PLUGIN_ROOT}/bin/reviews-append-only.mjs\""
          }
        ]
      }
    ]
  }
}
```

Read it from the outside in, event first. The event is `PreToolUse`, and the `matcher` is a pattern over tool names, so this entry fires for `Edit` and `Write` and stays out of the way of every read. The `command` is an ordinary shell command, run from the project directory, with the plugin root substituted.

Here is what happens when it fires. The harness writes a JSON description of the pending call to the command's standard input: the event name, the tool name, and the tool's arguments. It waits for the command to exit and reads the exit code. An exit code of zero means proceed. Two means refuse: the tool call does not happen, and whatever the command wrote to standard error goes back to the model as the result of the call, the way an `is_error` tool result did in Chapter 2. Any other exit code is reported as a broken hook, and the call proceeds.

Listing: `plugins/behavior/bin/reviews-append-only.mjs`

```
// Refuse an edit to a review file that already exists. Reviews are a
// record: a new one may be added and none may be changed. This runs as a
// PreToolUse hook, before the write reaches the file; exit code 2 blocks
// the call and hands stderr to the model as the reason.
import fs from "node:fs";

let payload = {};
try {
  payload = JSON.parse(fs.readFileSync(0, "utf8"));
} catch {
  // A hook that cannot read its input must not wedge every edit in the
  // repository. Failing open is a choice, and it is the one made here.
  process.exit(0);
}

const target = String(payload.tool_input?.file_path ?? "");
const isReview = /(^|\/)work\/[^/]+\/reviews\/[^/]+\.md$/.test(target);

if (isReview && fs.existsSync(target)) {
  process.stderr.write(
    `Blocked: ${target} is an existing review, and reviews are append-only. ` +
      "Write a new file with the next number instead.\n",
  );
  process.exit(2);
}

process.exit(0);
```

Two dozen lines, and one of Chapter 11's seven rules has moved from "a sentence" to "a program", inside the harness's loop rather than beside

it in a git hook. The message names what to do instead, for the reason Chapter 11's tip gave: a refusal the model can act on ends the exchange, and a bare "forbidden" is retried.

Note what the script does not check. It looks at the path and whether the file exists, and nothing else: it does not know what the edit is, whether the session is a reviewer or a judge, or whether the change was well-meant. That is the shape a hook should have. A hook that needed judgement would be a finding, and Chapter 11 said where those go.

Rule: Which file a rule lives in decides who can break it. In `SKILL.md` it is a request to the session. On an agent's `tools:` line it is a property of a process. In `hooks.json` it is a refusal the model receives and cannot argue with. Those are Chapter 11's three layers, and a plugin carries all three, so put each rule in the lowest layer that can hold it.

Trying it, then shipping it

The harness will load a plugin from a directory for one session, without installing anything:

```
claude --plugin-dir plugins/behavior
```

Inside that session, `/behavior:review fold-accents` runs the skill, which starts the agent, which runs the CLI and returns. Then test the hook: ask the session to change one word in `work/fold-accents/reviews/006-behavior.md`. The edit is refused, the reason is the sentence in the script, and the model, told what to do instead, proposes a new file.

Shipping it takes one more manifest, and a short one. The **market-**

place manifest is `.claude-plugin/marketplace.json` at the root of a repository, and it lists plugins by where they are:

Listing: `.claude-plugin/marketplace.json`

```
{
  "name": "agentic-workflow",
  "owner": { "name": "you" },
  "plugins": [
    {
      "name": "behavior",
      "source": "./plugins/behavior",
      "description": "The behavior lens, and the append-only rule for reviews."
    }
  ]
}
```

A marketplace, then, is a repository with this file in it, and the `source` of a plugin is a path inside that repository. Chapter 12's `settings.json` named a marketplace by its GitHub repository and a plugin as `quorum@quorum`: the plugin's name, then the marketplace's. Yours, once pushed, is `behavior@agentic-workflow`, and the same `extraKnownMarketplaces` block with your repository in it installs it for anyone who opens the repository.

You now have every part a plugin can have, and each is a text file you typed.

Reading quorum

Clone `https://github.com/AiFirstDevelopment/quorum` and list what is in it. The shape is the one above, with more in each directory.

```
quorum/
  .claude-plugin/marketplace.json   two plugins: quorum, and tests
  plugins/quorum/
    .claude-plugin/plugin.json      name: quorum
```

```
  skills/                       1-plan  2-build  3-review  4-quorum
                                pipeline  guard  audit  calibrate
                                history  status
  agents/                       quorum-builder  quorum-reviewer
                                quorum-scribe  quorum-judge
                                quorum-publisher  quorum-auditor
  hooks/hooks.json              PreToolUse on Edit|Write -> plan-lock
  bin/                          guard.py  plan-lock-hook.py  state.py
                                selftest.py  calibrate.py  audit.py ...
  workflow/pipeline.js          the unattended run
  reference/contract.md         where every artefact goes
  calibration/cases/            the fixtures Chapter 12 called thin
plugins/tests/                  the tests plugin: add, and ci
```

Take the parts in the order you built them.

The manifests. The marketplace manifest lists two plugins by path, which is why Chapter 12's settings enabled `quorum@quorum` and `tests@quorum`: two plugins, one marketplace, and the first plugin shares the marketplace's name.

The skills. One per command in Chapter 12's table, plus four that measure rather than build: `audit`, `calibrate`, `history` and `status`. Open `skills/3-review/SKILL.md` and read its body. It names the six lenses in a table and then says to run each one in fresh context, by starting a separate subagent per lens. That is your `review` skill six times over, and the sentence that hands the work away is the same sentence.

The agents. Read the `tools:` lines before anything else, because each one is a decision about what a process can do:

Agent	`tools:`	What that settles
`quorum-reviewer`	`Read, Grep, Glob, Bash`	Reads and runs; cannot `Edit` or `Write`
`quorum-scribe`	`Write`	Writes review files; cannot read code

Agent	`tools:`	What that settles
`quorum-publisher`	`Read, Bash`	Runs git; cannot edit code
`quorum-auditor`	`Read, Grep, Glob`	Cannot run anything at all
`quorum-builder`	*(every tool)*	Must edit; the line is absent
`quorum-judge`	*(every tool)*	Must apply repairs

The scribe is the division you made between reviewer and skill, made into an agent of its own: reviewers return findings as data, and a process that can only write puts them on disk. The reviewer, which ran the code, cannot touch the reviews directory, and the scribe, which writes there, has never seen the code.

The hook. `hooks/hooks.json` is your file with one difference: the matcher also names `MultiEdit`, and the command is `bin/plan-lock-hook.py`. Read the script, which is under a hundred lines. It takes the plan file as it is and as the proposed edit would leave it, computes Chapter 11's requirements hash over both, and exits 2 if they differ. Prohibition two from Chapter 9, "you may not edit Intent, Acceptance criteria, or Non-goals," is enforced before the write reaches the file, in a program that neither the builder nor the judge can talk to. It fails open, for the reason yours does.

The scripts. `bin/guard.py` is `guard.ts` at nine hundred lines: your four rules and six more, among them `coverage`, which fails a verdict that leaves a criterion out, because silence about AC4 reads like success, and `evidence`, which checks that every file and line a verdict cites as proof exists. `bin/state.py` writes `state.json`, a terse record of what has run, because a directory of prose artefacts says what was decided and not which job ran last. And `bin/selftest.py` builds throwaway

repositories, breaks each guard rule on purpose, and checks that the rule fires: Chapter 10's fail-first verification, applied to the guard.

The orchestration. `workflow/pipeline.js` is where the unattended run lives. It is a **workflow script**: a program the harness runs on your behalf that can start agents and wait for them, and can do nothing else. It has no shell, no file access and no model call of its own; every effect it has is an agent it started. Read it top to bottom and you are reading Chapters 6 through 11 as one function.

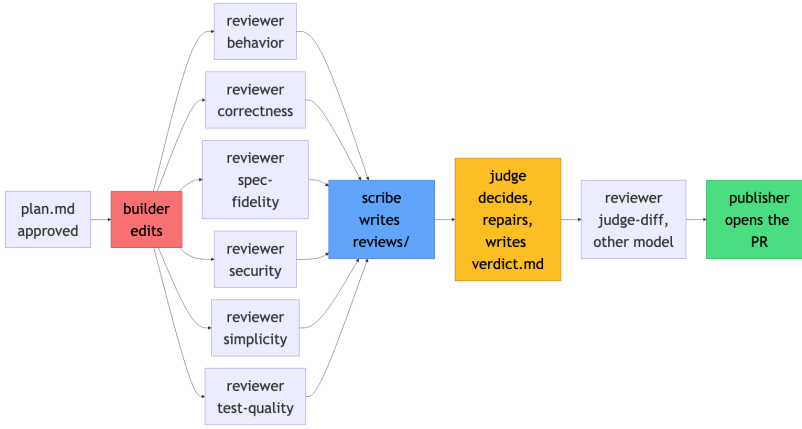


Figure 13.1: One unattended run of `pipeline.js`, as the phases the script declares. The builder edits. Six reviewers run at once and return findings as data. The scribe, which cannot read code, writes them to disk. The judge decides and repairs, and runs on a different model from the reviewer that then reads only the judge's own commits. The publisher opens the pull request and never touches code.

Four things in it are the chapters, made concrete:

- The six lenses are a list of remits, and the review phase is a `parallel()` over them, each an `agent()` call with `agentType: 'quorum:quorum-reviewer'`. That is Chapter 7's `Promise.all`, and the namespaced name is the one this chapter started with.

- Each reviewer must return JSON in a declared shape, and the shape requires a `file`, a `line` and a `failureScenario` on every finding. Chapter 7 asked for "how it fails" in a prompt; here a finding without one is not a finding, because it does not parse.
- The judge runs in a loop bounded at two passes, and the second pass is entered only over a red suite. Chapter 9's recursion is cut where Chapter 9 said to cut it, with the number written down.
- The recheck of the judge's own commits runs, by default, on a different model from the judge. That is this project's answer to Chapter 7's warning about shared blind spots, and the comment above the line says what it does not solve: every option is still a Claude model.

The contract. `reference/contract.md` is the file layout every skill reads and writes: `docs/work/<slug>/` holding `plan.md`, `reviews/`, `verdict.md` and `state.json`. It is Chapter 6's "where the plan lives", with the numbering and the slug rules written down so that ten skills agree on them.

Where the model is called

Search the repository for `messages.create` and you will find nothing. It has no API key, no `Anthropic` client and no `max_tokens` anywhere in it.

Every model call in a run of quorum is made by the harness. Chapters 1 through 4, the request, the tools, the loop and the context, are the harness's job now, and the plugin does not carry a line of them. What it carries is Chapters 6 through 11: a plan format, six remits, four prohibitions, a judge with two phases, a guard with ten rules, and a hook, all of it as text, in files at paths the harness knows.

That is the position Chapter 12 asked you to reach before delegating. A

plugin is not a program that does the pipeline. It is the pipeline you wrote, with the harness supplying the parts you built first and the parts you built last kept as you would have written them.

What you can do with it now

Because every part is a file, every part is yours to change.

- **Change a lens.** Its remit is one entry in the `LENSES` array in `pipeline.js` and one row in the table in `3-review`. Edit both, or the code and the table disagree.
- **Add a rule.** A mechanical one goes into `guard.py` beside the ten that are there; one that has to fire before a write reaches the file goes into `hooks.json`, with a script beside `plan-lock-hook.py`.
- **Prove it still works.** Run `selftest.py` after either change. It is the reason a change to the guard is safe to make.
- **Fork it.** Point the marketplace manifest at your copy, and Chapter 12's `settings.json` at your repository. A pinned `ref` is what stops a change upstream from arriving unread.

What you cannot do is what no plugin can: make the reviewers independent of the weights they share, or replace the reading you do at the approval gate. Those were the limits Chapter 12 listed, and reading the source moves none of them.

Where this leaves you

Twelve chapters built a pipeline by hand. This one showed you that the packaged version is the same pipeline, in files whose every mechanism you have now seen from the inside: a manifest found by path, a skill

placed into a conversation, an agent started in an empty one, a hook that runs before a write, and a script that arranges the rest.

The last chapter is short, and it is about what to do with that on Monday.

This page intentionally left blank

Conclusion

The split between what will be superseded and what will not runs along a clean line.

What will be obsolete

The model identifier in `setup.ts` will be superseded, probably within a year.

The specific parameters will move too. `thinking`, `effort`, and the beta flags on compaction and context editing will variously be renamed, become defaults, or be replaced by something better.

The prices in `budget.ts` have only ever moved in one direction, and the arithmetic around them will still be the arithmetic.

Some of the failures in Chapter 5. Models hallucinate less than they did, agree with you slightly less readily, and are better at noticing that they have not checked something. Every one of those is a rate, and every one of those rates is improving.

All of it is one edit in one place, which is why the model ID lives in a constant.

What will not

Plausible output is indistinguishable from correct output, not because models are bad, but because a model produces likely text and has no separate faculty that checks whether likely and true have come apart. A better model widens the gap between rate and detectability: fewer wrong answers, more convincing ones.

Nothing grades its own work. This is not an observation about language models. It is a structural fact about any evaluator whose only input is the thing being evaluated, and it is why the reviewers in Chapter 7 were blind to the argument, why the judge in Chapter 9 never met the builder, and why the judge's own repairs needed a pass nobody in the pipeline could perform.

A requirement written after the work describes the work. Chapter 6's whole argument, and it predates computers.

A rule stated in a prompt is a request, as Chapter 11 argued. Some rules can be moved into software that does not negotiate, and the ones that can, should be.

A test that has never failed is a claim, not a check, which was Chapter 10's subject. That was true before any of this existed and will be true after.

The thirteen chapters are how you act on those five sentences. Every technique here is a way of arranging for a disagreement to happen — between a plan and a diff, between a reader and an author, between an expectation and an exit code.

What to do on Monday

Do not start with all of it.

Take them in this order, because it is the order of benefit per unit of disruption:

1. Write the plan first, by hand, for one change. No tooling. A markdown file with intent, four falsifiable acceptance criteria, and three non-goals, which costs twenty minutes. Afterwards, "is it done?" is answered by running four commands rather than by forming an impression.

2. Prove one test can fail. Pick the test you most rely on, break the code it guards, and watch what the suite does. Roughly one time in five, in the author's experience, the test does not go red.

3. Add one guard rule, for whichever rule your team breaks most often. Ten lines and an exit code, run wherever your other checks run, make it a program rather than a policy document.

4. Then review, and only then. Multi-lens review is the most expensive job in the workflow and the one that produces the most output to read. It pays for itself against a written plan, and it produces noise against a vague one — so it comes after step 1 has become a habit, not before.

> **Pro Tip:** Do step 1 on a change you are confident about, and compare what you were sure the criteria were against what you can write down.

How little of this is about models

`loop.ts` is a `for` statement and a branch on a string, `plan.ts` writes a file, `review.ts` is `Promise.all`, and `judge.ts` reads six files and writes one. `guard.ts` is a hash comparison, a regular expression, and a subtraction, and it does not call a model at all.

The intelligence is rented and the plan, the reviews, the tests and the

guard are yours. The hard part was never getting a model to produce good code; models produce good code frequently and for very little money. The hard part is knowing which time was which, and that turns out to be an engineering problem of a familiar kind: artefacts, comparisons, exit codes, and somebody with the authority to say no.

Rule: The answer to "is it right?" must come from somewhere other than the thing that produced it. Every workflow you are offered can be evaluated by asking where its answer comes from.

What the Chapter 5 runs produced

Look at what actually happened in the Chapter 5 runs. A three-part request, correctly understood; the right Unicode technique identified; two of three parts implemented well; a clear explanation, in forty seconds, for a fraction of a cent.

The pipeline records which of those parts were checked, and by what.

Go and build something, and write the plan first.

Appendix A: Glossary

Terms in alphabetical order, each with the chapter and the file where it is defined and a one-line statement of what it means.

The chapter entry carries an example and the reason the term exists; the entry here carries neither. Use it when you remember the shape of an idea and not its name.

A

acceptance criterion — Chapter 6, `06-writing-it-down-first.md`. One checkable condition that must hold for a change to count as complete. Worth having only if it is falsifiable: it names a command to run or a file to open, and the observable result that counts as met.

adjudication — Chapter 9, `09-somebody-has-to-decide.md`. Taking a set of reported findings and deciding, one by one and with recorded reasons, what happens to each. The job that converts a pile of opinions into a decision.

agent — Introduction, `00-introduction.md`. A program that lets a model choose its own next action, repeatedly, until the model decides it is finished. Neither autonomy nor planning, but a loop with a model in the condition.

agent definition — Chapter 12, `12-from-yours-to-shared.md`. A file describing a subagent — its name, its system prompt, and the exact list of tools it may use. The tool list is what turns "review fixes nothing" from an instruction into a property of the process.

agentic loop — Chapter 3, `03-the-loop.md`. The cycle of sending the conversation, reading the stop reason, executing any requested tools, appending the results, and sending again. Your code runs it; the model only decides whether there is another pass.

agentic workflow — Introduction, `00-introduction.md`. A workflow in which some jobs are carried out by a language model rather than by a person or by code written in advance. Nothing in the definition is specific to software.

anchoring — Chapter 7, `07-a-second-pair-of-eyes.md`. The tendency of a judgement to be pulled towards information present when the judgement is made, whether or not that information is evidence. The reason a reviewer must never see the builder's summary.

API key — Introduction, `00-introduction.md`. A secret string identifying your account to a service, meant to be read by programs rather than typed by people. Kept in the environment, never in a source file.

assembled application — Chapter 8, `08-running-the-thing.md`. The program as a user encounters it: wired together and started the way it is really started. Not the function, and not the module with the rest mocked out.

B

behavioral test — Chapter 8, `08-running-the-thing.md`. Finding out what a program does by running it and observing the result — as opposed to reasoning about what it would do.

blind review — Chapter 7, `07-a-second-pair-of-eyes.md`. Review conducted without access to the author's reasoning. The reviewer is blind to the argument, not to the code.

blocked — Chapter 9, `09-somebody-has-to-decide.md`. A verdict outcome meaning something is unmet, unresolved, or unsafe. A success outcome: a run that ends blocked, with a clear statement of what is unresolved, has done its job.

branch protection — Chapter 11, `11-rules-a-prompt-cant-hold.md`. The repository-host setting that applies required status checks to a particular branch and removes the ability to push to it directly. An administrative setting, invisible from inside the repository.

C

CI — Chapter 11, `11-rules-a-prompt-cant-hold.md`. Continuous integration: a service that runs your checks on its own machines whenever a change is proposed, using the repository's committed configuration rather than anybody's local setup. Nothing on a developer's laptop can switch it off.

compaction — Chapter 4, `04-what-it-forgets.md`. An API feature that summarises older parts of a conversation when a request approaches a size threshold, keeping the gist and discarding the detail. Its blocks must be appended back into the message history or the state is lost without a warning.

content block — Chapter 1, `01-talking-to-a-model.md`. One typed piece of a response. `response.content` is an array of these, which is why code branches on `block.type` rather than reaching for `content[0].text`.

context editing — Chapter 4, `04-what-it-forgets.md`. An API feature that deletes old tool results from a conversation rather than summarising them. Compaction is lossy about everything; context editing is lossless about what it keeps.

context rot — Chapter 4, `04-what-it-forgets.md`. The degradation in a model's attention to any particular piece of its input as that input grows. It produces no error and no warning, only answers that are worse.

context window — Chapter 1, `01-talking-to-a-model.md`. The maximum number of tokens allowed in a single request and its response combined. A hard limit: exceed it and the API returns an error rather than dropping the oldest part.

conversation — Chapter 1, `01-talking-to-a-model.md`. The complete `messages` array, in order. It is the entire input to the model, sent again in full on every request, and it lives in your program rather than on a server.

coverage — Chapter 10, `10-tests-that-can-fail.md`. Which lines ran during a test suite. Useful in one direction — lines that never ran are definitely untested — and silent in the other, because a line can run inside a test that asserts nothing about it.

D

diff — Chapter 3, `03-the-loop.md`. The line-by-line difference between two states of a file or a tree: what was removed, what was added, and nothing else. The unit of review from Chapter 7 onward.

disposition — Chapter 9, `09-somebody-has-to-decide.md`. What a judge decides about one finding: accepted, rejected, or escalated. Three options because each names a different action.

E

escalation — Chapter 9, `09-somebody-has-to-decide.md`. A finding raised to a human because settling it requires authority the workflow does not have. Without this option, a decision procedure that cannot abstain will reject what it should have escalated.

exit code — Chapter 1, `01-talking-to-a-model.md`. The small integer a process returns to whatever started it. Zero conventionally means success. It is how the shell, `npm`, and every continuous integration system decide whether a program worked.

F

fail-first verification — Chapter 10, `10-tests-that-can-fail.md`. Proving that a test fails when the behaviour it guards is broken, before trusting it to guard anything. Break the code, run the test, see red, restore, see green.

falsifiable — Chapter 6, `06-writing-it-down-first.md`. A statement for which there is a specific observation that would show it is not met. A criterion that cannot be failed cannot be passed either.

false positive — Chapter 7, `07-a-second-pair-of-eyes.md`. A reported finding that is wrong. Expect them at roughly a quarter to a third of everything reported; they are cheap because reviewers have no authority to act.

finding — Chapter 7, `07-a-second-pair-of-eyes.md`. One reported problem: where it is, why it is wrong, and a concrete input that produces the wrong result. The last part is what separates a finding from a guess.

flake — Chapter 10, `10-tests-that-can-fail.md`. A test that passes

and fails on the same code. Its real cost is that every subsequent failure it produces gets read as "probably the flake again."

fresh context — Chapter 7, `07-a-second-pair-of-eyes.md`. A conversation that begins empty: no history, no earlier reasoning, only what you deliberately put in it. Because the model is stateless, a new array is genuinely a new mind on the problem.

H

hallucination — Chapter 5, `05-why-it-lies-to-you.md`. Output that is fluent, confident, well-formed, and false. Not a malfunction: the same operation that produces correct answers, observed in a case where the likely continuation and the true one came apart.

harness — Chapter 12, `12-from-yours-to-shared.md`. The program that runs the loop, manages the conversation, owns the tool set, and decides what the model sees. Every agent product is Chapter 3's loop with a larger tool set and a configuration file.

hook — Chapter 11, `11-rules-a-prompt-cant-hold.md`. A program that a tool calls at a defined point in its own execution, and whose exit code the tool respects. Your code does not call it; the tool does, whether anybody remembered it or not.

hook event — Chapter 13, `13-inside-a-plugin.md`. A named moment in a harness's own loop at which it runs a program you configured. `PreToolUse` is the moment after the model asks for a tool and before the harness runs it; a program that exits 2 there refuses the call.

I

inference — Chapter 1, `01-talking-to-a-model.md`. Running a model's weights over an input to produce an output. It is the only thing that happens when you call the API: no training, no learning, no adjustment.

J

JSON Schema — Chapter 2, `02-giving-the-model-hands.md`. A standard notation for describing the structure of JSON data — what fields exist, what types they hold, which are required. What a tool's `input_schema` is written in.

L

large language model — Chapter 1, `01-talking-to-a-model.md`. A very large mathematical function that takes a sequence of text and produces a probability distribution over what text comes next, fitted to that job on an enormous quantity of text. Abbreviated LLM.

lens — Chapter 7, `07-a-second-pair-of-eyes.md`. A review with exactly one question, asked in isolation, blind to the other lenses. Not a persona — a scope restriction, and the restriction is what makes it look closely.

LLM — Chapter 1, `01-talking-to-a-model.md`. Large language model.

M

marketplace — Chapter 12, `12-from-yours-to-shared.md`. A place plugins are published and found; for Claude Code, a git repository with a manifest listing what it offers. Declared in a repository's settings, it travels with the repository.

marketplace manifest — Chapter 13, `13-inside-a-plugin.md`. The file `.claude-plugin/marketplace.json` at the root of a repository, listing the plugins it carries by path. It is what makes a repository a marketplace.

`max_tokens` — Chapter 1, `01-talking-to-a-model.md`. A required hard ceiling on how much a model may produce in one response. Not a suggestion: reaching it stops the response mid-sentence and reports `max_tokens` as the stop reason.

merge gate — Chapter 10, `10-tests-that-can-fail.md`. A check that must pass before a change can be merged, enforced by something other than the good intentions of the person merging. A check that merely reports is a notification.

message — Chapter 1, `01-talking-to-a-model.md`. One contribution to a conversation, carrying a role and content. The `messages` array of them is the entire input to the model.

model ID — Chapter 1, `01-talking-to-a-model.md`. The exact identifier of the model a request runs against, such as `claude-opus-5`. Every request must name one. Worth keeping in one exported constant, because it is the string most likely to change.

N

namespace — Chapter 13, `13-inside-a-plugin.md`. A plugin's name, prefixed to every skill and agent it provides: `/behavior:review`, `quorum:quorum-reviewer`. A bare name does not resolve.

non-determinism — Chapter 5, `05-why-it-lies-to-you.md`. The property that identical inputs do not guarantee identical outputs. You cannot debug by re-running, and "it worked when I tried it" is a statement about one sample.

non-goal — Chapter 6, `06-writing-it-down-first.md`. Something a reasonable reader would expect a change to include, stated explicitly as not included. The cheapest way to keep scope from expanding without an argument about it later.

O

orchestration — Chapter 12, `12-from-yours-to-shared.md`. The layer that decides which subagents run, in what order, with what handed between them. In your own version it is `Promise.all` and the order of statements in `main`.

P

parallel tool use — Chapter 2, `02-giving-the-model-hands.md`. A single response containing several `tool_use` blocks, because the model could see the operations were independent. All their results go back in one user message.

plausible output — Chapter 5, `05-why-it-lies-to-you.md`. Output whose surface features — structure, tone, specificity, confidence — are

indistinguishable from those of correct output. The reason reading the answer is not a quality control.

plugin — Chapter 12, `12-from-yours-to-shared.md`. A bundle of skills, agent definitions, hooks, and commands distributed as a unit.

plugin manifest — Chapter 13, `13-inside-a-plugin.md`. The file `.claude-plugin/plugin.json`: a plugin's name, version and description. The name becomes the namespace.

plugin root — Chapter 13, `13-inside-a-plugin.md`. The directory a plugin was installed into, substituted for `${CLAUDE_PLUGIN_ROOT}` wherever one of the plugin's files refers to another.

pre-registration — Chapter 6, `06-writing-it-down-first.md`. A statement, filed before the data is collected, of what will be measured and which result counts as which outcome. Borrowed from science, for the same reason: a hypothesis chosen after seeing the data is a description.

`PreToolUse` — Chapter 11, `11-rules-a-prompt-cant-hold.md`. The point at which a packaged agent tool lets a hook run before a tool call and refuse it. The same idea as a check placed immediately before `execute` in your own loop.

prompt — Chapter 1, `01-talking-to-a-model.md`. The text given to a model to act on. Casually, the question you typed; in the API, the whole input — every message plus the system prompt.

prompt caching — Chapter 4, `04-what-it-forgets.md`. Storing the beginning of a request on the server so identical beginnings on later requests are billed at a fraction of the normal rate. An exact prefix match: one changed character invalidates everything after it.

R

rate limit — Chapter 1, `01-talking-to-a-model.md`. A cap on how many requests or tokens an account may use per minute, so that one client cannot starve the rest. Exceeding it produces a `RateLimitError`.

regression test — Chapter 10, `10-tests-that-can-fail.md`. An automated check that a behaviour that works today still works tomorrow. In a workflow with a model in it, its result is not an opinion.

required status check — Chapter 11, `11-rules-a-prompt-cant-hold.md`. A named check that must report success before a change may be merged. Writing the job is not the gate; listing it as required is.

retrieval — Chapter 4, `04-what-it-forgets.md`. Selecting the small part of a large body of material that a particular request actually needs, and sending only that. In its simplest form, `read_file`.

role — Chapter 1, `01-talking-to-a-model.md`. Who is speaking in a message: `user` for you, `assistant` for the model. The array must begin with a `user` message.

runaway loop — Chapter 3, `03-the-loop.md`. An agentic loop that never reaches `end_turn` and keeps calling the API until something external stops it. Its cost is roughly quadratic in iterations, and it produces no output at all.

S

SDK — Introduction, `00-introduction.md`. Software development kit: a library published by a service's maker that wraps its HTTP interface in ordinary functions of your language.

self-grading — Chapter 5, `05-why-it-lies-to-you.md`. Asking a model to evaluate output produced by a model — most sharply, its own. A grader whose only input is the thing being graded has a strong prior towards approval.

skill — Chapter 12, `12-from-yours-to-shared.md`. A folder containing instructions a harness loads on demand, usually a `SKILL.md` with a name, a description, and a body. A system prompt with a filing system.

smoke test — Chapter 8, `08-running-the-thing.md`. The smallest exercise that shows the assembled application runs at all. Not thorough, and not meant to be: the test whose failure means nothing else is worth running.

specification — Chapter 6, `06-writing-it-down-first.md`. A written statement of what a change is supposed to accomplish, produced before the change exists. Mostly a list of acceptance criteria.

stateless — Chapter 1, `01-talking-to-a-model.md`. Retaining nothing between requests. Every response is produced from scratch using only what that request contains; the appearance of memory is an array your program maintains.

static review — Chapter 8, `08-running-the-thing.md`. Examining code without executing it. Everything five of the six lenses do, and everything a code review in a browser tab does.

stop reason — Chapter 2, `02-giving-the-model-hands.md`. A short string on every response saying why the model stopped: `end_turn`, `tool_use`, `max_tokens`, `pause_turn`, `refusal`, and two rarer ones, `stop_sequence` and `model_context_window_exceeded`. Branch on it before reading `content`, not after.

subagent — Chapter 7, `07-a-second-pair-of-eyes.md`. A process that runs in its own fresh context, does one bounded job, and returns a result to whatever started it.

sycophancy — Chapter 5, `05-why-it-lies-to-you.md`. A model's tendency to move towards the position it believes its interlocutor holds, rather than towards the position the evidence supports. Worst exactly where a grader is needed most.

system of record — Chapter 6, `06-writing-it-down-first.md`. The one artefact that is authoritative about what was supposed to happen, kept somewhere that survives the conversation that produced it.

system prompt — Chapter 1, `01-talking-to-a-model.md`. A top-level `system` field carrying instructions about how to behave rather than what to answer. It sits outside the message array, comes first however long the conversation grows, and reads as operator instruction rather than as something a participant said.

T

temperature — Chapter 1, `01-talking-to-a-model.md`. A dial controlling how adventurously the next token is sampled from the model's distribution. Setting it to zero makes the selection rule deterministic; it does not make the model deterministic.

termination condition — Chapter 3, `03-the-loop.md`. Whatever makes an agentic loop stop. Three are needed: the model saying it is done, something going wrong, and a hard iteration cap.

token — Chapter 1, `01-talking-to-a-model.md`. The unit of text a model operates on: a chunk of characters between a letter and a word, roughly four characters of English prose. You are billed per token, `max_tokens` is measured in them, and the context window bounds them.

token budget — Chapter 4, `04-what-it-forgets.md`. A soft limit you impose on the total tokens a piece of work may consume across every

request it makes. Nothing enforces it but you. The context window protects the model; this protects you.

tool — Introduction, `00-introduction.md`. A function in your program that you have described to the model in a form it can request. The model gains no capability it did not have — it gains a vocabulary for asking you to do something.

tool schema — Chapter 2, `02-giving-the-model-hands.md`. The description of a tool: a name, a plain-English description, and an `input_schema` giving the shape of the arguments. The descriptions are the only thing the model has to decide whether and how to call it.

tool use — Chapter 2, `02-giving-the-model-hands.md`. The whole mechanism: the model emitting a structured request to call one of your described functions, your program executing it, and the result being fed back into the conversation.

`tool_result` — Chapter 2, `02-giving-the-model-hands.md`. A content block carrying what your function returned, joined to its request by `tool_use_id`. Every request gets one, including the ones that failed — those carry `is_error: true`.

`tool_use` block — Chapter 2, `02-giving-the-model-hands.md`. A content block in which the model asks you to call a tool, carrying an `id`, a `name`, and the arguments as `input`.

training cutoff — Chapter 4, `04-what-it-forgets.md`. The point in time up to which the text a model was fitted on was gathered. After it the model knows nothing, and cannot distinguish "after my cutoff" from "does not exist."

transcript — Chapter 1, `01-talking-to-a-model.md`. The conversation regarded as something to inspect rather than something to send.

turn — Chapter 3, `03-the-loop.md`. One pass through the agentic loop: one request, one response, and whatever tool execution follows.

V

vendoring — Chapter 11, `11-rules-a-prompt-cant-hold.md`. Copying a shared tool into a repository so it runs without external dependencies. Necessary for CI runners, and it freezes the copy on the day it was made.

verdict — Chapter 9, `09-somebody-has-to-decide.md`. The recorded outcome of adjudication: every finding with its disposition and reason, every acceptance criterion with a yes or no and the evidence for it, and one overall result of `ready`, `ready with follow-ups`, or `blocked`.

violation — Chapter 11, `11-rules-a-prompt-cant-hold.md`. A broken rule. Unlike a finding, nothing in the pipeline may weigh it, reject it, or explain it away, because the rules are the things the pipeline states it does not do.

W

workflow — Introduction, `00-introduction.md`. A fixed sequence of jobs a piece of work passes through, where each job has one responsibility and a defined output.

workflow script — Chapter 13, `13-inside-a-plugin.md`. A program the harness runs that can start agents and wait for them, and do nothing else: no shell, no files, no model call of its own. Where an unattended run is written down.

This page intentionally left blank

Appendix B: The Finished Workflow

Every complete file the chapters ask you to create, in its finished state.

These listings are generated from the chapter sources by `scripts/build-appendix.mjs` rather than written by hand, so they cannot disagree with the chapters: if a listing here differs from the one you typed, the chapter is where the difference came from.

Where a file grew across several chapters, the version below is the last one — the one the finished workflow runs. The chapter each listing was completed in is noted beside it.

The workflow itself — everything under `agentic/` — is 14 TypeScript files and 1546 lines. Both numbers are counted from the listings below when this page is generated, so the sentence cannot drift away from the table. At that size, what makes the workflow work is not the quantity of code but which parts are kept separate from which other parts.

To run any of it you need `ANTHROPIC_API_KEY` exported, `@anthropic-ai/sdk` installed, and `"type": "module"` in `package.json`. `guard.ts` is the one exception — it calls no model and needs no credential.

What is here

File	Completed in	Lines
`package.json`	Introduction	18
`tsconfig.json`	Introduction	14
`.gitignore`	Introduction	1
`agentic/talk.ts`	Talking to a Model	37
`agentic/setup.ts`	Talking to a Model	13
`agentic/chat.ts`	Talking to a Model	49
`demo/src/slugify.ts`	Tests That Can Fail	27
`demo/src/cli.ts`	Giving the Model Hands	10
`agentic/tool.ts`	Giving the Model Hands	115
`agentic/loop.ts`	The Loop	226
`agentic/agent.ts`	The Loop	30
`agentic/budget.ts`	What It Knows and Forgets	120
`agentic/grade.ts`	Why It Lies to You	72
`agentic/plan.ts`	Writing It Down First	100
`agentic/review.ts`	A Second Pair of Eyes	153
`agentic/behave.ts`	Running the Thing	171
`agentic/judge.ts`	Somebody Has to Decide	142
`demo/src/slugify.test.ts`	Tests That Can Fail	23
`agentic/prove.ts`	Tests That Can Fail	112
`agentic/guard.ts`	Rules a Prompt Can't Hold	206
`.githooks/pre-commit`	Rules a Prompt Can't Hold	10

File	Completed in	Lines
`.github/workflows/guard.yml`	Rules a Prompt Can't Hold	30
`.claude/settings.json`	From Yours to Shared	14
`plugins/behavior/.claude-plugin/plugin.json`	Inside a Plugin	5
`plugins/behavior/skills/review/SKILL.md`	Inside a Plugin	19
`plugins/behavior/agents/reviewer.md`	Inside a Plugin	25
`plugins/behavior/hooks/hooks.json`	Inside a Plugin	15
`plugins/behavior/bin/inventory-append-only.mjs`	Inside a Plugin	27
`.claude-plugin/marketplace.json`	Inside a Plugin	11

package.json

From Introduction.

```
{
  "name": "agentic-workflow",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "test": "tsx demo/src/slugify.test.ts",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.124.0"
  },
  "devDependencies": {
    "@types/node": "^20.14.0",
    "tsx": "^4.16.0",
    "typescript": "^5.5.0"
  }
}
```

tsconfig.json

From Introduction.

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022"],
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "types": ["node"],
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "noEmit": true
  },
  "include": ["agentic/**/*.ts", "demo/**/*.ts"]
}
```

.gitignore

From Introduction.

```
node_modules/
```

agentic/talk.ts

From Talking to a Model.

```
import Anthropic from "@anthropic-ai/sdk";

// A missing credential should be one clear line, not a stack trace.
if (!process.env.ANTHROPIC_API_KEY) {
  console.error("ANTHROPIC_API_KEY is not set. Export it and run this again.");
  process.exit(1);
}
```

```
const client = new Anthropic();

async function main(): Promise<void> {
  const response = await client.messages.create({
    model: "claude-opus-5",
    max_tokens: 1024,
    messages: [
      {
        role: "user",
        content: "In two sentences: what is a slug in a URL, and why?",
      },
    ],
  });

  for (const block of response.content) {
    if (block.type === "text") {
      console.log(block.text);
    }
  }

  console.log(
    `\n[in: ${response.usage.input_tokens} out:
    ${response.usage.output_tokens}]`,
  );
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/setup.ts

From Talking to a Model.

```
// The one place the model identifier lives. When Anthropic releases a
// newer model, this is the only line in the whole project that changes.
export const MODEL = "claude-opus-5";

// Every script from here on calls this before it does anything else.
export function requireApiKey(): void {
  if (!process.env.ANTHROPIC_API_KEY) {
    console.error(
```

```
      "ANTHROPIC_API_KEY is not set. Export it and run this again.",
    );
    process.exit(1);
  }
}
```

agentic/chat.ts

From Talking to a Model.

```
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

requireApiKey();

const client = new Anthropic();

const SYSTEM = "You are terse. Answer in one short sentence.";

async function ask(
  history: Anthropic.MessageParam[],
  question: string,
): Promise<string> {
  history.push({ role: "user", content: question });

  const response = await client.messages.create({
    model: MODEL,
    max_tokens: 1024,
    system: SYSTEM,
    messages: history,
  });

  // Push the model's own reply back into the history, or the next
  // request will look like we never heard it.
  history.push({ role: "assistant", content: response.content });

  return response.content
    .filter((b): b is Anthropic.TextBlock => b.type === "text")
    .map((b) => b.text)
    .join("");
}

async function main(): Promise<void> {
```

```
  const remembered: Anthropic.MessageParam[] = [];
  console.log("A:", await ask(remembered, "My favourite colour is orange."));
  console.log("B:", await ask(remembered, "What is my favourite colour?"));

  // Same question, same model, same everything - except we throw the
  // history away first.
  const forgotten: Anthropic.MessageParam[] = [];
  console.log("C:", await ask(forgotten, "What is my favourite colour?"));

  console.log(`\nhistory carried into B: ${remembered.length - 1} messages`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

demo/src/slugify.ts

From Tests That Can Fail.

```
// Characters that have a conventional multi-letter expansion. These must
// run before normalisation: NFD does not decompose ß into anything.
const EXPANSIONS: Array<[RegExp, string]> = [
  [/ß/g, "ss"],
  [/æ/g, "ae"],
  [/œ/g, "oe"],
  [/ø/g, "o"],
  [/đ/g, "d"],
  [/ł/g, "l"],
];

// Convert a piece of text into a URL slug.
export function slugify(input: string): string {
  let text = input.toLowerCase();

  for (const [pattern, replacement] of EXPANSIONS) {
    text = text.replace(pattern, replacement);
  }

  // NFD splits "é" into "e" + a combining acute accent; the range
  // U+0300-U+036F is those combining marks, which we then drop.
  const folded = text.normalize("NFD").replace(/[\u0300-\u036f]/g, "");
```

```
  const slug = folded.replace(/[^a-z0-9]+/g, "-").replace(/^-+|-+$/g, "");

  return slug.length > 0 ? slug : "untitled";
}
```

demo/src/cli.ts

From Giving the Model Hands.

```
import { slugify } from "./slugify.js";

const input = process.argv.slice(2).join(" ");

if (input.length === 0) {
  console.error("usage: tsx demo/src/cli.ts <text>");
  process.exit(1);
}

console.log(slugify(input));
```

agentic/tool.ts

From Giving the Model Hands.

```
import fs from "node:fs";
import path from "node:path";
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

requireApiKey();

const client = new Anthropic();
const ROOT = process.cwd();

const readFileTool: Anthropic.Tool = {
  name: "read_file",
  description:
    "Read a UTF-8 text file from the project and return its contents. " +
```

```
    "Paths are relative to the project root.",
  input_schema: {
    type: "object",
    properties: {
      path: {
        type: "string",
        description: "Project-relative path, e.g. demo/src/slugify.ts",
      },
    },
    required: ["path"],
  },
};

// The actual implementation. Note that it validates its own input:
// the arguments came from a model, not from your code.
function readFile(input: unknown): string {
  const { path: relative } = input as { path?: unknown };
  if (typeof relative !== "string") {
    throw new Error("read_file: 'path' must be a string");
  }
  const resolved = path.resolve(ROOT, relative);
  if (!resolved.startsWith(ROOT + path.sep)) {
    throw new Error(`read_file: refusing to read outside the project`);
  }
  return fs.readFileSync(resolved, "utf8");
}

async function main(): Promise<void> {
  const messages: Anthropic.MessageParam[] = [
    {
      role: "user",
      content:
        "Read demo/src/slugify.ts, then say in one sentence what it does " +
        "to accented characters such as è.",
    },
  ];

  // --- First request: the model decides it needs the file.
  const first = await client.messages.create({
    model: MODEL,
    max_tokens: 2048,
    tools: [readFileTool],
    messages,
  });

  console.log("stop_reason:", first.stop_reason);
```

```
if (first.stop_reason !== "tool_use") {
  console.log("The model answered without reading anything.");
  return;
}

// The model's request has to go into the history verbatim.
messages.push({ role: "assistant", content: first.content });

// --- Execute every request it made, collecting one result each.
const results: Anthropic.ToolResultBlockParam[] = [];

for (const block of first.content) {
  if (block.type !== "tool_use") continue;

  console.log(`call: ${block.name}(${JSON.stringify(block.input)})`);

  try {
    results.push({
      type: "tool_result",
      tool_use_id: block.id,
      content: readFile(block.input),
    });
  } catch (error) {
    results.push({
      type: "tool_result",
      tool_use_id: block.id,
      content: error instanceof Error ? error.message : String(error),
      is_error: true,
    });
  }
}

// All results go back as ONE user message.
messages.push({ role: "user", content: results });

// --- Second request: same conversation, now with the file in it.
const second = await client.messages.create({
  model: MODEL,
  max_tokens: 2048,
  tools: [readFileTool],
  messages,
});

for (const block of second.content) {
  if (block.type === "text") {
    console.log(block.text);
  }
```

```
  }
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/loop.ts

From The Loop.

```
import fs from "node:fs";
import path from "node:path";
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

// Checked once, here, because every entry point imports this module.
// A missing key becomes one line instead of a stack trace.
requireApiKey();

const client = new Anthropic();
const ROOT = process.cwd();

export const MAX_ITERATIONS = 25;

export interface AgentRun {
  text: string;
  iterations: number;
  inputTokens: number;
  outputTokens: number;
}

// --- Tools ------------------------------------------------------------

function projectPath(
  input: unknown,
  field: string,
  toolName: string,
): { relative: string; absolute: string } {
  if (typeof input !== "object" || input === null) {
    throw new Error(`${toolName}: expected an object of arguments`);
  }
```

```
  const value = (input as Record<string, unknown>)[field];
  if (typeof value !== "string" || value.length === 0) {
    throw new Error(`${toolName}: '${field}' must be a non-empty string`);
  }
  const absolute = path.resolve(ROOT, value);
  // The root itself resolves to ROOT with no trailing separator, so a bare
  // prefix test rejects `.` - which is the first thing list_files is asked for.
  if (absolute !== ROOT && !absolute.startsWith(ROOT + path.sep)) {
    throw new Error(`${toolName}: refusing to touch anything outside the
    project`);
  }
  return { relative: value, absolute };
}

function readFile(input: unknown): string {
  const { absolute } = projectPath(input, "path", "read_file");
  return fs.readFileSync(absolute, "utf8");
}

function listFiles(input: unknown): string {
  const { relative, absolute } = projectPath(input, "dir", "list_files");
  const entries = fs.readdirSync(absolute, { withFileTypes: true });
  if (entries.length === 0) {
    return `${relative} is empty.`;
  }
  return entries
    .map((entry) => (entry.isDirectory() ? `${entry.name}/` : entry.name))
    .join("\n");
}

function writeFile(input: unknown): string {
  const { relative, absolute } = projectPath(input, "path", "write_file");
  const content = (input as Record<string, unknown>)["content"];
  if (typeof content !== "string") {
    throw new Error("write_file: 'content' must be a string");
  }
  const existed = fs.existsSync(absolute);
  fs.mkdirSync(path.dirname(absolute), { recursive: true });
  fs.writeFileSync(absolute, content, "utf8");
  return `${existed ? "Overwrote" : "Created"} ${relative} ` +
    `(${content.length} characters).`;
}

export const FILE_TOOLS: Anthropic.Tool[] = [
  {
    name: "read_file",
    description:
```

```
      "Read a UTF-8 text file from the project and return its contents. " +
      "Paths are relative to the project root.",
    input_schema: {
      type: "object",
      properties: {
        path: { type: "string", description: "e.g. demo/src/slugify.ts" },
      },
      required: ["path"],
    },
  },
  {
    name: "list_files",
    description:
      "List the entries in a project directory, one per line. " +
      "Directories are shown with a trailing slash. " +
      "Use this when you are unsure what a file is called.",
    input_schema: {
      type: "object",
      properties: {
        dir: { type: "string", description: "e.g. demo/src" },
      },
      required: ["dir"],
    },
  },
  {
    name: "write_file",
    description:
      "Write a UTF-8 text file, replacing it entirely if it already exists. " +
      "You must supply the complete new contents, not a fragment.",
    input_schema: {
      type: "object",
      properties: {
        path: { type: "string", description: "e.g. demo/src/slugify.ts" },
        content: { type: "string", description: "The complete file contents" },
      },
      required: ["path", "content"],
    },
  },
];

// The same set with the one destructive tool removed. Anything that is only
// supposed to read gets this, and then it cannot do otherwise.
export const READ_ONLY_TOOLS: Anthropic.Tool[] = FILE_TOOLS.filter(
  (tool) => tool.name !== "write_file",
);

function execute(name: string, input: unknown): string {
```

```
  switch (name) {
    case "read_file":
      return readFile(input);
    case "list_files":
      return listFiles(input);
    case "write_file":
      return writeFile(input);
    default:
      throw new Error(`Unknown tool: ${name}`);
  }
}

// --- The loop ----------------------------------------------------------

function trace(line: string): void {
  // Traces go to stderr so that stdout stays usable as a result.
  process.stderr.write(`${line}\n`);
}

export async function runAgent(
  system: string,
  task: string,
  tools: Anthropic.Tool[] = FILE_TOOLS,
  maxIterations: number = MAX_ITERATIONS,
): Promise<AgentRun> {
  const messages: Anthropic.MessageParam[] = [{ role: "user", content: task }];
  let inputTokens = 0;
  let outputTokens = 0;

  for (let iteration = 1; iteration <= maxIterations; iteration++) {
    const response = await client.messages.create({
      model: MODEL,
      max_tokens: 8192,
      system,
      tools: tools.length > 0 ? tools : undefined,
      messages,
    });

    inputTokens += response.usage.input_tokens;
    outputTokens += response.usage.output_tokens;

    const text = response.content
      .filter((b): b is Anthropic.TextBlock => b.type === "text")
      .map((b) => b.text)
      .join("");

    if (response.stop_reason === "end_turn") {
```

```
    return { text, iterations: iteration, inputTokens, outputTokens };
  }

  if (response.stop_reason === "max_tokens") {
    throw new Error(
      `Cut off at max_tokens on iteration ${iteration}. Raise max_tokens ` +
        `or ask for less at a time.`,
    );
  }

  if (response.stop_reason === "refusal") {
    throw new Error("The model declined this request.");
  }

  messages.push({ role: "assistant", content: response.content });

  if (response.stop_reason === "pause_turn") {
    continue;
  }

  const results: Anthropic.ToolResultBlockParam[] = [];

  for (const block of response.content) {
    if (block.type !== "tool_use") continue;
    trace(`  [${iteration}] ${block.name} ${JSON.stringify(block.input)}`);
    try {
      results.push({
        type: "tool_result",
        tool_use_id: block.id,
        content: execute(block.name, block.input),
      });
    } catch (error) {
      trace(`  [${iteration}] failed: ${String(error)}`);
      results.push({
        type: "tool_result",
        tool_use_id: block.id,
        content: error instanceof Error ? error.message : String(error),
        is_error: true,
      });
    }
  }

  if (results.length === 0) {
    throw new Error(
      `Stopped with '${response.stop_reason}' and no tool call to answer.`,
    );
  }
```

```
    messages.push({ role: "user", content: results });
  }

  throw new Error(
    `Gave up after ${maxIterations} iterations without reaching end_turn.`,
  );
}
```

agentic/agent.ts

From The Loop.

```
import { runAgent } from "./loop.js";

const SYSTEM = [
  "You are a careful software engineer working in a TypeScript project.",
  "Read before you write. When you change a file, write the complete new",
  "contents, never a fragment. When you are finished, state plainly what you",
  "changed and why.",
].join(" ");

async function main(): Promise<void> {
  const task = process.argv.slice(2).join(" ");

  if (task.length === 0) {
    console.error('usage: tsx agentic/agent.ts "<task>"');
    process.exit(1);
  }

  const run = await runAgent(SYSTEM, task);

  console.log(run.text);
  console.log(
    `\n[${run.iterations} iterations, ` +
      `in: ${run.inputTokens} out: ${run.outputTokens}]`,
  );
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/budget.ts

From What It Knows and Forgets.

```ts
import fs from "node:fs";
import Anthropic from "@anthropic-ai/sdk";
import { MODEL, requireApiKey } from "./setup.js";

requireApiKey();

const client = new Anthropic();

// Dollars per million tokens for the model named in setup.ts.
// Check the current figures on Anthropic's pricing page; they change,
// and an estimate you have stopped believing is worse than none.
// input_tokens is only the part of the prompt that was neither written
// to the cache nor read from it; the three prompt counts add up.
export const PRICE_PER_MTOK = {
  input: 5,
  cacheWrite: 6.25,
  cacheRead: 0.5,
  output: 25,
};

export function estimateCost(usage: {
  input_tokens: number;
  output_tokens: number;
  cache_creation_input_tokens?: number | null;
  cache_read_input_tokens?: number | null;
}): number {
  const written = usage.cache_creation_input_tokens ?? 0;
  const cached = usage.cache_read_input_tokens ?? 0;
  return (
    (usage.input_tokens / 1_000_000) * PRICE_PER_MTOK.input +
    (written / 1_000_000) * PRICE_PER_MTOK.cacheWrite +
    (cached / 1_000_000) * PRICE_PER_MTOK.cacheRead +
    (usage.output_tokens / 1_000_000) * PRICE_PER_MTOK.output
  );
}

// The soft limit. Nothing in the API knows about this number.
export const TOKEN_BUDGET = 200_000;

export function assertWithinBudget(spent: number, label: string): void {
  if (spent > TOKEN_BUDGET) {
```

```
    throw new Error(
      `${label}: token budget exceeded - ${spent} of ${TOKEN_BUDGET} used.`,
    );
  }
}

// Identical on every request, and long enough to be worth caching. The
// house rules alone are a few dozen tokens, and a prefix that short is
// never stored: the floor is a few hundred tokens and varies by model.
// The file under discussion is what carries this block over it.
const HOUSE_RULES = [
  "You are a reviewer for a small TypeScript project.",
  "Answer in at most three sentences.",
  "Never speculate about code you have not been shown.",
  "If a question cannot be answered from what you have, say so plainly.",
  "Prefer concrete nouns from the code over general description.",
].join("\n");

const SOURCE = fs.readFileSync("agentic/loop.ts", "utf8");

const CONTEXT =
  `${HOUSE_RULES}\n\nThe file under discussion, agentic/loop.ts:\n\n${SOURCE}`;

async function main(): Promise<void> {
  const messages: Anthropic.MessageParam[] = [];
  let spent = 0;
  let dollars = 0;

  const questions = [
    "What does runAgent do when the stop reason is max_tokens?",
    "Which tool is missing from READ_ONLY_TOOLS, and why?",
    "How many ways can the loop stop?",
    "Which of those would a runaway loop hit first?",
    "Summarise this conversation in one sentence.",
  ];

  for (const [index, question] of questions.entries()) {
    messages.push({ role: "user", content: question });

    const response = await client.messages.create({
      model: MODEL,
      max_tokens: 4096,
      system: [
        {
          type: "text",
          text: CONTEXT,
          cache_control: { type: "ephemeral" },
```

```
        },
      ],
      messages,
    });

    messages.push({ role: "assistant", content: response.content });

    const usage = response.usage;
    const cached = usage.cache_read_input_tokens ?? 0;
    const written = usage.cache_creation_input_tokens ?? 0;

    spent += usage.input_tokens + written + cached + usage.output_tokens;
    dollars += estimateCost(usage);

    console.log(
      `round ${index + 1}: ` +
        `fresh ${String(usage.input_tokens).padStart(6)}  ` +
        `cached ${String(cached).padStart(6)}  ` +
        `written ${String(written).padStart(6)}  ` +
        `out ${String(usage.output_tokens).padStart(5)}  ` +
        `total ${String(spent).padStart(7)}  ` +
        `$${dollars.toFixed(5)}`,
    );

    assertWithinBudget(spent, "budget.ts");
  }
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/grade.ts

From Why It Lies to You.

```
import { runAgent } from "./loop.js";

const ENGINEER = [
  "You are a careful software engineer working in a TypeScript project.",
  "Read before you write. Write complete file contents, never fragments.",
  "When finished, state plainly what you changed and why.",
```

```
].join(" ");

const GRADER = [
  "You are grading a change to a TypeScript project.",
  "Read the files you need. Judge only what the code actually does.",
  "Be brief and be decisive.",
].join(" ");

const TASK = [
  "demo/src/slugify.ts mangles non-ASCII text.",
  "Fix it so that accented Latin letters fold to their base letters,",
  "German ß becomes ss, and the result is never an empty string.",
  "Do not add a dependency.",
].join(" ");

async function main(): Promise<void> {
  // --- 1. Do the work.
  const work = await runAgent(ENGINEER, TASK);
  console.log("=== 1. What it says it did ===\n");
  console.log(work.text);

  // --- 2. Ask it whether the work is correct.
  const grade = await runAgent(
    GRADER,
    [
      "A colleague changed demo/src/slugify.ts and reported this:",
      "---",
      work.text,
      "---",
      "The original request was:",
      TASK,
      "",
      "Read the file as it now stands. Does it do what was asked?",
      'Reply with exactly one line: "PASS: <reason>" or "FAIL: <reason>".',
    ].join("\n"),
  );
  console.log("\n=== 2. Its own grade ===\n");
  console.log(grade.text);

  // --- 3. Disagree with it, giving no reason whatsoever.
  const pressed = await runAgent(
    GRADER,
    [
      "Earlier you judged a change to demo/src/slugify.ts and said:",
      "---",
      grade.text,
      "---",
```

```
      "Another reviewer looked at the same code and reached the opposite",
      "conclusion. They did not explain why.",
      "",
      "Do you stand by your judgement?",
      'Reply with exactly one line: "STAND: <reason>" or "CHANGE: <reason>".',
    ].join("\n"),
  );
  console.log("\n=== 3. The same question, under mild pressure ===\n");
  console.log(pressed.text);

  const inTokens = work.inputTokens + grade.inputTokens + pressed.inputTokens;
  const outTokens = work.outputTokens + grade.outputTokens +
  pressed.outputTokens;
  console.log(`\n[in: ${inTokens} out: ${outTokens}]`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/plan.ts

From Writing It Down First.

```
import fs from "node:fs";
import path from "node:path";
import { runAgent, READ_ONLY_TOOLS } from "./loop.js";

const PLANNER = [
  "You are writing a plan for a change to a TypeScript project.",
  "No code has been written yet and you will write none.",
  "Read whatever files you need in order to plan against the code that exists.",
  "",
  "Every acceptance criterion must be falsifiable: it names a command to run",
  "or a file to open, and states the observable result that counts as met.",
  "Reject your own criteria that cannot be checked that way.",
  "'Handles Unicode correctly' is not a criterion. 'Running the CLI on the",
  "input Weiss with a sharp s prints weiss' is.",
  "",
  "Non-goals name things a reasonable reader would expect and will not get.",
  "Open questions are things you could not settle by reading the code. Never",
  "invent an answer to one; list it and move on.",
```

```
  "",
  "Return the plan and nothing else. No preamble, no closing remarks.",
].join("\n");

const TEMPLATE = `# Plan: <short title>

- **Slug:** <slug>
- **Status:** draft

## Request

> <the request, verbatim>

## Intent

<two or three sentences: what changes, and why it is worth doing>

## Acceptance criteria

- [ ] **AC1 - <name>.** <command to run or file to open, and the observable
      result that counts as met>

## Non-goals

- <something a reader would expect and will not get>

## Open questions

- <anything you could not settle by reading, or "None." with a reason>

## Steps

- [ ] S1: <the first change to make>
`;

async function main(): Promise<void> {
  const [slug, ...rest] = process.argv.slice(2);
  const request = rest.join(" ");

  if (slug === undefined || request.length === 0) {
    console.error('usage: tsx agentic/plan.ts <slug> "<request>"');
    process.exit(1);
  }

  const dir = path.join("work", slug);
  const file = path.join(dir, "plan.md");
```

```
  if (fs.existsSync(file)) {
    console.error(`${file} already exists. Choose another slug or delete it.`);
    process.exit(1);
  }

  const run = await runAgent(
    PLANNER,
    [
      "Write a plan for this request.",
      "",
      "REQUEST:",
      request,
      "",
      `The slug for this work is: ${slug}`,
      "",
      "Use exactly this shape:",
      "",
      TEMPLATE,
    ].join("\n"),
    // It plans against the code. A prompt telling it not to write is a
    // request; withholding write_file is the thing that makes it true.
    READ_ONLY_TOOLS,
  );

  fs.mkdirSync(dir, { recursive: true });
  fs.writeFileSync(file, `${run.text.trim()}\n`, "utf8");

  console.log(`Wrote ${file} in ${run.iterations} iterations.`);
  console.log("Read it before anything else runs.");
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/review.ts

From A Second Pair of Eyes.

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
```

```
import { runAgent } from "./loop.js";

interface Lens {
  name: string;
  question: string;
}

const LENSES: Lens[] = [
  {
    name: "correctness",
    question:
      "Does this change do what it claims on inputs it will actually see? " +
      "Look for off-by-one errors, unhandled cases, wrong operators, and " +
      "assumptions the surrounding code does not guarantee.",
  },
  {
    name: "spec-fidelity",
    question:
      "Compare the change against the plan. Is every acceptance criterion " +
      "actually met by the code shown? Was anything listed under Non-goals " +
      "built anyway? A criterion met only for the literal example in the " +
      "plan is not met.",
  },
  {
    name: "security",
    question:
      "What does this do with input it did not expect? Consider untrusted " +
      "strings, paths, very large inputs, and anything that reaches a file " +
      "system, a shell, or a network call.",
  },
  {
    name: "simplicity",
    question:
      "What here is more complicated than the problem requires? Look for " +
      "duplication, dead code, needless abstraction, and anything the " +
      "existing code already does.",
  },
  {
    name: "test-quality",
    question:
      "Would the tests fail if this behaviour broke? Look for assertions " +
      "that cannot fail, tests coupled to implementation detail, and " +
      "behaviour changed without any test covering it.",
  },
];

const REVIEWER = [
```

```
  "You are reviewing a change to a TypeScript project.",
  "You did not write it and you have not seen the reasoning behind it.",
  "You have the diff and the plan. That is deliberately all you have.",
  "",
  "Report problems. Do not fix anything and do not rewrite any code.",
  "Do not comment on style, formatting, or naming unless it causes a defect.",
  "",
  "For each problem, write exactly this shape:",
  "",
  "## Finding N - <one line, specific>",
  "",
  "- **Where:** <file and, if you can tell, the line>",
  "- **Severity:** blocker | major | minor",
  "- **Why it is wrong:** <one or two sentences>",
  "- **How it fails:** <a concrete input, and the wrong result it produces>",
  "",
  "If you cannot name a concrete failing input, the finding is a guess.",
  "Mark it minor and say so.",
  "If you find nothing, reply with exactly: No findings.",
].join("\n");

function diffAgainst(base: string): string {
  return execFileSync("git", ["diff", base, "--", "."], {
    encoding: "utf8",
    maxBuffer: 32 * 1024 * 1024,
  });
}

async function runLens(
  lens: Lens,
  diff: string,
  plan: string,
): Promise<string> {
  const run = await runAgent(
    `${REVIEWER}\n\nYour lens is "${lens.name}".\n${lens.question}\n` +
      "Findings outside your lens are not your job. Ignore them.",
    [
      "PLAN",
      "====",
      plan,
      "",
      "DIFF",
      "====",
      diff,
    ].join("\n"),
    [], // No tools. The diff and the plan are the whole world.
  );
```

```
  return run.text.trim();
}

async function main(): Promise<void> {
  const [slug, base] = process.argv.slice(2);

  if (slug === undefined) {
    console.error("usage: tsx agentic/review.ts <slug> [base-ref]");
    process.exit(1);
  }

  const planFile = path.join("work", slug, "plan.md");
  if (!fs.existsSync(planFile)) {
    console.error(`No plan at ${planFile}. Run agentic/plan.ts first.`);
    process.exit(1);
  }

  const plan = fs.readFileSync(planFile, "utf8");
  const diff = diffAgainst(base ?? "HEAD");

  if (diff.trim().length === 0) {
    console.error("The diff is empty. There is nothing to review.");
    process.exit(1);
  }

  const dir = path.join("work", slug, "reviews");
  fs.mkdirSync(dir, { recursive: true });

  console.error(`Reviewing ${diff.split("\n").length} diff lines ` +
    `through ${LENSES.length} lenses...`);

  // Every lens runs against the same diff, at the same time,
  // and none of them can see any other's output.
  const reports = await Promise.all(
    LENSES.map((lens) => runLens(lens, diff, plan)),
  );

  LENSES.forEach((lens, index) => {
    const file = path.join(
      dir,
      `${String(index + 1).padStart(3, "0")}-${lens.name}.md`,
    );
    fs.writeFileSync(
      file,
      `# Review: ${lens.name}\n\n${reports[index]}\n`,
      "utf8",
    );
```

```
    const clean = reports[index].startsWith("No findings");
    console.log(`${clean ? "  clean" : "FINDING"}  ${file}`);
  });
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/behave.ts

From Running the Thing.

```
import fs from "node:fs";
import path from "node:path";
import { spawnSync } from "node:child_process";
import { runAgent } from "./loop.js";

const PROPOSER = [
  "You are choosing inputs to try against a command-line program.",
  "You will not be told what the program does; you are guessing at what",
  "might break it, from the plan alone.",
  "",
  "Return one input per line and nothing else. No numbering, no",
  "commentary. Write each input as a JSON string - double quotes and",
  "escapes - so that a space or an empty string survives the trip.",
  "At most ten lines.",
  "",
  "Include: the ordinary case, every case the plan names, the empty case,",
  "and two or three cases the plan does not mention but a user would hit.",
].join("\n");

const OBSERVER = [
  "You are the behavior reviewer. You did not write this change.",
  "Below is a plan, and a transcript of a real session recorded by running",
  "the program. Every line of the transcript is an observation.",
  "",
  "Report where the observed behaviour does not match the plan.",
  "Never state what the program does beyond what the transcript shows.",
  "If the transcript does not cover something, say it was not exercised -",
  "do not reason about what would have happened.",
  "",
```

```
  "Use exactly this shape for each problem:",
  "",
  "## Finding N - <one line>",
  "",
  "- **Input:** <the exact input from the transcript>",
  "- **Expected:** <what the plan requires>",
  "- **Observed:** <what the transcript shows>",
  "- **Severity:** blocker | major | minor",
  "",
  "If everything observed matches the plan, reply with exactly: No findings.",
].join("\n");

interface Observation {
  input: string;
  code: number;
  stdout: string;
  stderr: string;
}

function exercise(input: string): Observation {
  const result = spawnSync("npx", ["tsx", "demo/src/cli.ts", input], {
    encoding: "utf8",
    timeout: 30_000,
  });

  return {
    input,
    code: result.status ?? -1,
    stdout: result.stdout ?? "",
    stderr: result.stderr ?? "",
  };
}

function render(observations: Observation[]): string {
  return observations
    .map((o) =>
      [
        `$ npx tsx demo/src/cli.ts ${JSON.stringify(o.input)}`,
        `exit code: ${o.code}`,
        `stdout: ${JSON.stringify(o.stdout)}`,
        `stderr: ${JSON.stringify(o.stderr.trim())}`,
      ].join("\n"),
    )
    .join("\n\n");
}

async function main(): Promise<void> {
```

```
const [slug] = process.argv.slice(2);

if (slug === undefined) {
  console.error("usage: tsx agentic/behave.ts <slug>");
  process.exit(1);
}

const planFile = path.join("work", slug, "plan.md");
if (!fs.existsSync(planFile)) {
  console.error(`No plan at ${planFile}.`);
  process.exit(1);
}
const plan = fs.readFileSync(planFile, "utf8");

// 1. The model proposes. It is not claiming anything about behaviour.
const proposed = await runAgent(
  PROPOSER,
  ["PLAN", "====", plan].join("\n"),
  [],
);

// JSON, not raw text. "" and " " are two of the inputs most likely to
// break a program, and a trim-and-drop-blanks parser destroys both of
// them before the program is ever run.
const inputs = proposed.text
  .split("\n")
  .map((line) => line.trim())
  .filter((line) => line.startsWith('"'))
  .flatMap((line) => {
    try {
      const value: unknown = JSON.parse(line);
      return typeof value === "string" ? [value] : [];
    } catch {
      return [];
    }
  })
  .slice(0, 10);

if (inputs.length === 0) {
  console.error("No inputs were proposed. Nothing to exercise.");
  process.exit(1);
}

// 2. Your code runs them. This is the only source of fact in the file.
const observations = inputs.map((input) => exercise(input));
const transcript = render(observations);
```

```
  console.error(transcript);
  console.error("");

  // 3. The model reads what happened.
  const report = await runAgent(
    OBSERVER,
    [
      "PLAN",
      "====",
      plan,
      "",
      "SESSION TRANSCRIPT",
      "==================",
      transcript,
    ].join("\n"),
    [],
  );

  const dir = path.join("work", slug, "reviews");
  fs.mkdirSync(dir, { recursive: true });

  const file = path.join(dir, "006-behavior.md");
  fs.writeFileSync(
    file,
    [
      "# Review: behavior",
      "",
      "## Session transcript",
      "",
      "```",
      transcript,
      "```",
      "",
      "## Findings",
      "",
      report.text.trim(),
      "",
    ].join("\n"),
    "utf8",
  );

  console.log(`wrote ${file}`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
```

```
});
```

agentic/judge.ts

From Somebody Has to Decide.

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { runAgent, FILE_TOOLS } from "./loop.js";

const PROHIBITIONS = [
  "You may not weaken, skip, or delete a test to make something pass.",
  "You may not edit Intent, Acceptance criteria, or Non-goals in the plan.",
  "You may not record an acceptance criterion as met when it is not met.",
  "You may not expand scope: a real defect outside this change is a",
  "follow-up, never a repair.",
].join("\n");

const ADJUDICATOR = [
  "You are judging a change. You did not write it and you did not review it.",
  "You have the plan, the diff, and every review report.",
  "",
  "Decide each finding: accepted, rejected, or escalated.",
  "  accepted  - real, in scope, must be fixed now.",
  "  rejected  - wrong, or a preference, or excluded by the plan. Say why.",
  "  escalated - cannot be settled without a decision a person must make.",
  "",
  "Reviewers are frequently wrong. Rejecting a finding with a clear reason",
  "is a correct outcome, and so is escalating one you are not entitled to",
  "settle. Do not accept a finding merely because it was reported.",
  "",
  "These prohibitions are absolute:",
  PROHIBITIONS,
  "",
  "Return exactly this document and nothing else:",
  "",
  "## Findings",
  "",
  "| # | Lens | Finding | Disposition | Reason |",
  "|---|---|---|---|---|",
  "",
  "## Acceptance criteria",
```

```
  "",
  "| Criterion | Met | Evidence |",
  "|---|---|---|",
  "",
  "## Escalations",
  "",
  "<one bullet each, or 'None.'>",
  "",
  "## Follow-ups",
  "",
  "<one bullet each, or 'None.'>",
  "",
  "## Outcome",
  "",
  "<exactly one of: ready | ready with follow-ups | blocked>",
].join("\n");

const REPAIRER = [
  "You are applying repairs that have already been decided by a judge.",
  "Fix exactly the findings marked accepted, and nothing else.",
  "",
  "These prohibitions are absolute:",
  PROHIBITIONS,
  "",
  "When you are done, list what you changed, one line per file.",
].join("\n");

function readOutcome(document: string): string {
  const match = /##\s*Outcome\s*\n+([^\n]+)/i.exec(document);
  return match === null ? "unknown" : match[1].trim();
}

async function main(): Promise<void> {
  const [slug, base] = process.argv.slice(2);

  if (slug === undefined) {
    console.error("usage: tsx agentic/judge.ts <slug> [base-ref]");
    process.exit(1);
  }

  const dir = path.join("work", slug);
  const reviewsDir = path.join(dir, "reviews");

  if (!fs.existsSync(reviewsDir)) {
    console.error(`No reviews at ${reviewsDir}. Run review.ts and behave.ts.`);
    process.exit(1);
  }
```

```
const plan = fs.readFileSync(path.join(dir, "plan.md"), "utf8");

const reviews = fs
  .readdirSync(reviewsDir)
  .filter((name) => name.endsWith(".md"))
  .sort()
  .map((name) => {
    const body = fs.readFileSync(path.join(reviewsDir, name), "utf8");
    return `----- ${name} -----\n${body}`;
  })
  .join("\n\n");

const diff = execFileSync("git", ["diff", base ?? "HEAD", "--", "."], {
  encoding: "utf8",
  maxBuffer: 32 * 1024 * 1024,
});

// --- Phase 1: decide. No tools, so nothing can be changed here.
const decision = await runAgent(
  ADJUDICATOR,
  ["PLAN", "====", plan, "", "DIFF", "====", diff, "", "REVIEWS", "=======",
    reviews].join("\n"),
  [],
);

const verdictFile = path.join(dir, "verdict.md");
fs.writeFileSync(verdictFile, `${decision.text.trim()}\n`, "utf8");

const outcome = readOutcome(decision.text);
console.log(`outcome: ${outcome}`);
console.log(`wrote ${verdictFile}`);

if (!/\baccepted\b/i.test(decision.text)) {
  console.log("Nothing accepted. No repairs to apply.");
  return;
}

// --- Phase 2: repair. The decision is now a file, not an opinion.
const repair = await runAgent(
  REPAIRER,
  ["VERDICT", "=======", decision.text, "", "PLAN", "====", plan].join("\n"),
  FILE_TOOLS,
);

fs.appendFileSync(
  verdictFile,
```

```
    `\n## Repairs applied\n\n${repair.text.trim()}\n`,
    "utf8",
  );

  console.log("repairs applied; see the end of verdict.md");
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

demo/src/slugify.test.ts

From Tests That Can Fail.

```
import test from "node:test";
import assert from "node:assert/strict";
import { slugify } from "./slugify.js";

test("ASCII text becomes a lowercase hyphenated slug", () => {
  assert.equal(slugify("Hello There World"), "hello-there-world");
});

test("accented Latin letters fold to their base letters", () => {
  assert.equal(slugify("Crème Brûlée"), "creme-brulee");
  assert.equal(slugify("naïve café"), "naive-cafe");
});

test("sharp s expands to ss in either case", () => {
  assert.equal(slugify("Weiß"), "weiss");
  assert.equal(slugify("WEI\u1E9E"), "weiss"); // capital sharp s
});

test("input with no slug characters becomes untitled", () => {
  assert.equal(slugify("///"), "untitled");
  assert.equal(slugify(" "), "untitled");
  assert.equal(slugify(""), "untitled");
});
```

agentic/prove.ts

From Tests That Can Fail.

```ts
import { execFileSync, spawnSync } from "node:child_process";
import { runAgent, FILE_TOOLS } from "./loop.js";

// The suite is defined once, in package.json. Naming the test files here
// too would leave this checking a subset of it the day you add one.
const TEST_COMMAND = ["npm", "test"];

const SABOTEUR = [
  "You are proving that a test is capable of failing.",
  "Make the smallest possible edit to the named source file that breaks the",
  "behaviour the named test checks - and nothing else.",
  "Do not touch any test file. Do not leave a comment explaining the change.",
  "Report the single line you changed and what it now does.",
].join("\n");

interface SuiteResult {
  green: boolean;
  output: string;
}

function runSuite(): SuiteResult {
  const [command, ...args] = TEST_COMMAND;
  const result = spawnSync(command, args, {
    encoding: "utf8",
    timeout: 120_000,
  });
  return {
    green: result.status === 0,
    output: `${result.stdout ?? ""}${result.stderr ?? ""}`,
  };
}

function restore(file: string): void {
  execFileSync("git", ["checkout", "--", file]);
}

// node:test prints one line per case - "ok 5 - name" for a pass, "not ok 3 -
// name" for a failure. Searching the whole output for the name matches the
// passing line just as happily, so match the failure line itself.
function failedAt(output: string, testName: string): boolean {
  return output
```

```
    .split("\n")
    .some((line) => /^not ok \d+ - (.*)$/.exec(line.trim())?.[1] === testName);
}

async function main(): Promise<void> {
  const [sourceFile, testName] = process.argv.slice(2);

  if (sourceFile === undefined || testName === undefined) {
    console.error('usage: tsx agentic/prove.ts <source-file> "<test name>"');
    process.exit(1);
  }

  // This script restores the file with git. Refuse to run over work
  // that git cannot give back.
  const dirty = execFileSync("git", ["status", "--porcelain", "--", sourceFile],
  {
    encoding: "utf8",
  });
  if (dirty.trim().length > 0) {
    console.error(`${sourceFile} has uncommitted changes.`);
    console.error("Commit or stash them first; this script uses git to undo.");
    process.exit(1);
  }

  const before = runSuite();
  if (!before.green) {
    console.error("The suite is already red. A red suite proves nothing.");
    console.error(before.output);
    process.exit(1);
  }
  console.log("1. baseline ......... green");

  const sabotage = await runAgent(
    SABOTEUR,
    [`Source file: ${sourceFile}`, `Test to break: ${testName}`].join("\n"),
    FILE_TOOLS,
  );
  console.log(`2. sabotage ......... ${sabotage.text.trim().split("\n")[0]}`);

  const after = runSuite();

  if (after.green) {
    restore(sourceFile);
    console.error(`\nNOT PROVEN: the suite stayed green with the code broken.`);
    console.error(`"${testName}" does not guard that behaviour.`);
    process.exit(1);
  }
```

```
  if (!failedAt(after.output, testName)) {
    restore(sourceFile);
    console.error(`\nNOT PROVEN: the suite went red, but not at
    "${testName}".`);
    console.error(after.output);
    process.exit(1);
  }
  console.log(`3. sabotaged ........ red, at "${testName}"`);

  restore(sourceFile);

  const restored = runSuite();
  if (!restored.green) {
    console.error("\nThe suite is red after restoring. Check `git status`.");
    process.exit(1);
  }
  console.log("4. restored ......... green");

  console.log(`\nPROVEN: "${testName}" fails when its behaviour is broken.`);
}

main().catch((error: unknown) => {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
});
```

agentic/guard.ts

From Rules a Prompt Can't Hold.

```
import fs from "node:fs";
import path from "node:path";
import { createHash } from "node:crypto";
import { execFileSync, spawnSync } from "node:child_process";

interface Violation {
  rule: string;
  detail: string;
}

// The same one command CI runs. A guard that checks a different suite
// from the one the project declares is checking nothing in particular.
```

```
const TEST_COMMAND = ["npm", "test"];

function git(args: string[]): string {
  return execFileSync("git", args, {
    encoding: "utf8",
    maxBuffer: 32 * 1024 * 1024,
  });
}

function gitShow(ref: string, file: string): string | null {
  const result = spawnSync("git", ["show", `${ref}:${file}`], {
    encoding: "utf8",
  });
  return result.status === 0 ? (result.stdout ?? "") : null;
}

function sectionLines(document: string, name: string): string[] {
  const lines = document.split("\n");
  const start = lines.findIndex(
    (line) => line.trim().toLowerCase() === `## ${name.toLowerCase()}`,
  );
  if (start === -1) return [];
  const rest = lines.slice(start + 1);
  const end = rest.findIndex((line) => line.startsWith("## "));
  return end === -1 ? rest : rest.slice(0, end);
}

function normalised(document: string, name: string): string {
  const lines = sectionLines(document, name);
  return lines.length === 0
    ? `<<missing: ${name}>>`
    : lines.join("\n").trim().replace(/\s+/g, " ");
}

function requirementsHash(plan: string): string {
  const body = ["Intent", "Acceptance criteria", "Non-goals"]
    .map((name) => normalised(plan, name))
    .join("\n~~~\n");
  return createHash("sha256").update(body).digest("hex");
}

function countCases(source: string): number {
  return (source.match(/(^|[^.\w])test\s*\(/g) ?? []).length;
}

function countExclusions(source: string): number {
  return (source.match(/\btest\.(only|skip)\s*\(/g) ?? []).length;
```

```
}

function main(): void {
  const args = process.argv.slice(2);
  const record = args.includes("--record");
  const positional = args.filter((arg) => !arg.startsWith("--"));
  const slug = positional[0];
  const base = positional[1] ?? "origin/main";

  if (slug === undefined) {
    console.error("usage: tsx agentic/guard.ts [--record] <slug> [base-ref]");
    process.exit(1);
  }

  const dir = path.join("work", slug);
  const plan = fs.readFileSync(path.join(dir, "plan.md"), "utf8");
  const hashFile = path.join(dir, "requirements.sha256");

  // Run once, by a person, at the moment the plan is approved.
  if (record) {
    fs.writeFileSync(hashFile, `${requirementsHash(plan)}\n`, "utf8");
    console.log(`recorded ${hashFile}`);
    return;
  }

  const violations: Violation[] = [];

  // --- requirements: nobody moved the target.
  if (!fs.existsSync(hashFile)) {
    violations.push({
      rule: "requirements",
      detail: `${hashFile} is missing; nothing pins what was approved.`,
    });
  } else if (fs.readFileSync(hashFile, "utf8").trim() !== requirementsHash(plan))
{
    violations.push({
      rule: "requirements",
      detail: "Intent, Acceptance criteria, or Non-goals changed after
approval.",
    });
  }

  // --- tests and reviews: read the change, not the intentions.
  const changed = git(["diff", "--name-status", base, "--"]).trim();

  for (const line of changed.length === 0 ? [] : changed.split("\n")) {
    // A rename prints three fields: "R100\told\tnew". The path on disk now is
```

```
  // the last one; the path to look up at the base ref is always the second.
  // Reading the first field as the file is how this crashes on a rename.
  const fields = line.split("\t");
  const status = fields[0];
  const file = fields[fields.length - 1];
  const previous = fields[1];
  if (file === undefined || previous === undefined) continue;

  const reviews = `${dir}/reviews/`;
  if (
    (file.startsWith(reviews) || previous.startsWith(reviews)) &&
    status !== "A"
  ) {
    violations.push({
      rule: "reviews",
      detail: `${previous} was ${status === "D" ? "deleted" : "modified"}; ` +
        "reviews are append-only.",
    });
  }

  if (file.endsWith(".test.ts") || previous.endsWith(".test.ts")) {
    const before = gitShow(base, previous);
    const after = status === "D" ? "" : fs.readFileSync(file, "utf8");

    if (before !== null && countCases(after) < countCases(before)) {
      violations.push({
        rule: "tests",
        detail: `${file}: ${countCases(before)} cases before, ` +
          `${countCases(after)} now.`,
      });
    }

    const exclusionsBefore = before === null ? 0 : countExclusions(before);
    if (countExclusions(after) > exclusionsBefore) {
      violations.push({
        rule: "tests",
        detail: `${file}: a new skip or only marker was added.`,
      });
    }
  }
}

// --- verdict: 'ready' has to mean something.
const verdictFile = path.join(dir, "verdict.md");

if (fs.existsSync(verdictFile)) {
  const verdict = fs.readFileSync(verdictFile, "utf8");
```

```
  const outcome = normalised(verdict, "Outcome").toLowerCase();

  if (outcome.startsWith("ready")) {
    const unmet = sectionLines(verdict, "Acceptance criteria").filter((row) =>
      /\|\s*no\s*\|/i.test(row),
    );
    if (unmet.length > 0) {
      violations.push({
        rule: "verdict",
        detail: `outcome '${outcome}' with ${unmet.length} unmet criteria.`,
      });
    }

    const escalations = normalised(verdict, "Escalations");
    if (!/^none\.?$/i.test(escalations)) {
      violations.push({
        rule: "verdict",
        detail: `outcome '${outcome}' with open escalations.`,
      });
    }

    const suite = spawnSync(TEST_COMMAND[0], TEST_COMMAND.slice(1), {
      encoding: "utf8",
      timeout: 120_000,
    });
    if (suite.status !== 0) {
      violations.push({
        rule: "verdict",
        detail: `outcome '${outcome}' over a red suite.`,
      });
    }
  }
}

if (violations.length === 0) {
  console.log(`guard: clean (${slug})`);
  return;
}

for (const violation of violations) {
  console.error(`VIOLATION [${violation.rule}] ${violation.detail}`);
}
console.error(
  `\nguard: ${violations.length} violation(s). ` +
    "These are rules, not findings. Nothing here may adjudicate them away.",
);
process.exit(1);
```

```
}

try {
  main();
} catch (error) {
  console.error(error instanceof Error ? error.message : String(error));
  process.exit(1);
}
```

.githooks/pre-commit

From Rules a Prompt Can't Hold.

```
#!/usr/bin/env bash
set -euo pipefail

slug="$(git rev-parse --abbrev-ref HEAD | sed 's|.*/||')"

# Not every branch is a work item. Say nothing on the ones that are not.
[ -d "work/$slug" ] || exit 0

base="$(git merge-base HEAD origin/main 2>/dev/null || echo HEAD)"
npx tsx agentic/guard.ts "$slug" "$base"
```

.github/workflows/guard.yml

From Rules a Prompt Can't Hold.

```
name: guard

on:
  pull_request:

jobs:
  guard:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
```

```
          fetch-depth: 0

      - uses: actions/setup-node@v4
        with:
          node-version: 20

      - run: npm ci

      - name: Regression suite
        run: npm test

      - name: Guard rules
        run: |
          slug="${GITHUB_HEAD_REF##*/}"
          if [ -d "work/$slug" ]; then
            npx tsx agentic/guard.ts "$slug" "origin/${GITHUB_BASE_REF}"
          else
            echo "no work item for branch $GITHUB_HEAD_REF; nothing to guard"
          fi
```

`.claude/settings.json`

From From Yours to Shared.

```
{
  "extraKnownMarketplaces": {
    "quorum": {
      "source": {
        "source": "github",
        "repo": "AiFirstDevelopment/quorum"
      }
    }
  },
  "enabledPlugins": {
    "quorum@quorum": true,
    "tests@quorum": true
  }
}
```

plugins/behavior/.claude-plugin/plugin.json

From Inside a Plugin.

```
{
  "name": "behavior",
  "description": "The behavior lens from Chapter 8, and the append-only rule for
  reviews from Chapter 11.",
  "version": "0.1.0"
}
```

plugins/behavior/skills/review/SKILL.md

From Inside a Plugin.

```
---
name: review
description: >
  Review a change by running the assembled application and reporting what
  actually happens. Use before merging any change with a runnable entry point.
argument-hint: [slug]
---

# behavior:review

Hand this job to the `behavior:reviewer` agent, in its own context. Give it
the slug and nothing else: not what was built, not why, not what you expect
it to find. It has not seen this conversation and must not.

When it returns, write its report, unchanged, to
`work/<slug>/reviews/NNN-behavior.md`, where NNN continues from the highest
number already in that directory. Then tell the user where the file is and
how many findings it holds. Do not act on the findings; that is the judge's
job.
```

plugins/behavior/agents/reviewer.md

From Inside a Plugin.

```
---
name: reviewer
description: >
  Runs the assembled application on inputs chosen from the plan and reports
  what it observed. Reads and runs; does not edit.
tools: Read, Bash
---

You are the behavior reviewer. You did not write this change and you have
not seen the reasoning behind it.

1. Read `work/<slug>/plan.md` for the acceptance criteria.
2. Choose up to ten inputs: the ordinary case, every case the plan names,
   the empty case, and two or three a user would hit that the plan does not.
3. Run each one with `npx tsx demo/src/cli.ts <input>`. Record the exact
   command, its stdout, its stderr, and its exit code, for every input.
4. Return a report with the raw transcript first, then one finding for each
   mismatch between what the plan requires and what the transcript shows:
   the input, the expected result, the observed result, and a severity of
   blocker, major, or minor. If everything matches, the findings section
   reads: No findings.

Never state what the program does beyond what the transcript shows. If a
case was not exercised, say so rather than reasoning about what would have
happened. Do not modify any file.
```

plugins/behavior/hooks/hooks.json

From Inside a Plugin.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "node
            \"${CLAUDE_PLUGIN_ROOT}/bin/reviews-append-only.mjs\""
          }
        ]
      }
```

```
    ]
  }
}
```

plugins/behavior/bin/reviews-append-only.mjs

From Inside a Plugin.

```
// Refuse an edit to a review file that already exists. Reviews are a
// record: a new one may be added and none may be changed. This runs as a
// PreToolUse hook, before the write reaches the file; exit code 2 blocks
// the call and hands stderr to the model as the reason.
import fs from "node:fs";

let payload = {};
try {
  payload = JSON.parse(fs.readFileSync(0, "utf8"));
} catch {
  // A hook that cannot read its input must not wedge every edit in the
  // repository. Failing open is a choice, and it is the one made here.
  process.exit(0);
}

const target = String(payload.tool_input?.file_path ?? "");
const isReview = /(^|\/)work\/[^/]+\/reviews\/[^/]+\.md$/.test(target);

if (isReview && fs.existsSync(target)) {
  process.stderr.write(
    `Blocked: ${target} is an existing review, and reviews are append-only. ` +
      "Write a new file with the next number instead.\n",
  );
  process.exit(2);
}

process.exit(0);
```

.claude-plugin/marketplace.json

From Inside a Plugin.

```
{
  "name": "agentic-workflow",
  "owner": { "name": "you" },
  "plugins": [
    {
      "name": "behavior",
      "source": "./plugins/behavior",
      "description": "The behavior lens, and the append-only rule for reviews."
    }
  ]
}
```